University of Padova

Department of Mathematics "Tullio Levi-Civita"

Master Thesis in Computer Science

CrossDem

Embedding Italian Prime Ministers' Speeches to Track the Evolution of Political Rhetoric Against V-Dem Indices

Master Candidate
Marco Bellò

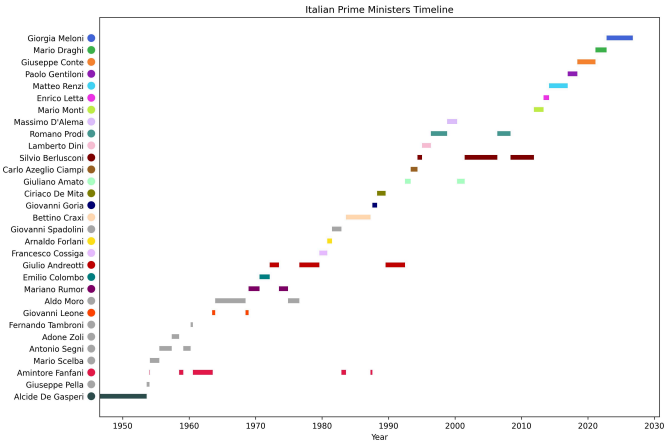
Supervisor
Prof. Alessandro Galeazzi
University of Padova

Student ID
2122196

Academic Year
2026-2027

Marco Bellò, *Crossdem — Embedding Italian Prime Ministers' Speeches to Track the Evolution of Political Rhetoric Against V-Dem Indices*
Master Thesis in Computer Science , University of Padova, © 2026-2027

This document was prepared using Typst, based on Davide Spada's template. In addition, LLMs (Claude Sonnet 5, Gemini 3.6 Flash, and Copilot Free) were used to check syntax and semantics only. All source code (Python and Typst), as well as the whole dataset, is available in the repository:
https://github.com/mhetacc/crossdem



Look again at that dot. That's here. That's home. That's us.

On it everyone you love, everyone you know, everyone you ever heard of, every human being who ever was, lived out their lives.

The aggregate of our joy and suffering, thousands of confident religions, ideologies and economic doctrines, every hunter and forager, every hero and coward, every creator and destroyer of civilization, every king and peasant, every young couple in love, every mother and father, hopeful child, inventor and explorer, every teacher of morals, every corrupt politician, every "superstar", every "supreme leader", every saint and sinner in the history of our species lived there–on a mote of dust suspended in a sunbeam.

— Carl Sagan, *Pale Blue Dot*, 1994

Abstract

Politicians are some of the most influential people in history: from the Greeks' demagogues to our Presidents and Prime Ministers, leaders have always tried to shape society. In this work I collect public speeches from Italian Prime Ministers authored between 1945 and 2025, scraping them from YouTube and Radio Radicale. The speeches are then transcribed into text with OpenAI's speech recognition model "Whisper", and annotated with the LLM "Gemma 4" for their degrees of hate speech, aggressiveness, and negativity, as well as whether they target a specific group or not (possible target groups are political adversaries, gender minorities, ethnic minorities, and religious minorities). The resulting dataset contains 6,038 speeches distributed among twenty-four Prime Ministers. I then measure the evolution of lexical complexity over time, comparing it against V-Dem democratic indices. The same comparison is performed on the evolution of speeches' sentiment over time. I also embed the speeches into vectors to visualize and measure polarization among politicians and political leanings. Overall, results show little correlation between complexity, sentiment, and democratic indices, while there is strong evidence of polarization.

Contents

Abstract v

List of Tables viii

List of Figures x

List of Codes xiii

List of Acronyms xiv

1 Introduction 1

1.1 Research Questions ... 4

2 State of the Art 6

2.1 Textual Complexity ... 6

2.2 Polarization ... 7

2.3 Sentiment Analysis ... 9

3 The Corpus 12

3.1 Similar Corpora ... 12

3.2 Corpus Structure ... 14

3.3 Building the Corpus ... 18

3.3.1 Alcide De Gasperi's Corpus ... 18

3.3.2 Scraping ... 19

3.3.3 Corpus Annotation ... 33

3.3.4 Varieties of Democracy ... 38

4 Results 41

4.1 Textual Complexity ... 41

4.1.1 Measure of Textual Lexical Diversity ... 41

4.1.2 Prime Minister's MTLDs ... 42

4.1.3 VDEM ... 44

4.1.4 Conclusions ... 46

4.2 Polarization ... 47

4.2.1 Prime Minister's Lexical Similarity ... 47

4.2.2 Political Leanings Lexicons Over Time 48
4.2.3 t-SNE 49
4.2.4 Conclusions 50
4.3 Sentiment Analysis 51
4.3.1 Democratic Indices Against Sentiment Scoring 51
4.3.2 Toxicity (TODO: change name) 51
4.3.3 Targets 52
4.3.4 Conclusions 53

5 Discussion & Future Works 57

A Appendix 58
1.1 Tables 58
1.1.1 Not Colored 58
1.1.2 Colored 59
1.1.3 Half Colored Half Not 59
1.1.4 Stacked 60
1.2 Figures 60
1.2.1 Single Figure 60
1.2.2 Double Figures 61
1.3 Code Listings 61

References 71

List of Tables

Table 3.1 Overall corpus size and pooled median speech length. First row shows how many total speeches were collected, second row shows how many words (and tokens) were collected, and third row shows the overall median speech length, in words (and tokens). Tokenization done via `spacy.load("it_core_news_sm")`. 15

Table 3.2 Corpus overview per prime minister. In order: Prime Minister's names, time period from which their speeches are pooled, how many speeches were pooled, how many words (and tokens) were pooled in total, and median speech length, in words (and tokens). Tokenization done via `spacy.load("it_core_news_sm")`. 15

Table 3.3 Per-PM classification results, ordered by their first term in office. In order: Prime Minister's name, percentage of speeches above threshold, number of speeches below the similarity threshold. Mean similarity represents how much the speeches resemble each other (lower values mean more variety in style). Colombo and Gentiloni were skipped (only one usable speech each, minimum three required). In total, 567 flagged speeches were exported to `outlier_speeches.csv`. 32

Table 3.4 Agreement between candidate models' predictions and gold labels on the 207 validation speeches, measured by Cohen's κ. Quadratic-weighted κ is used for the ordinal labels (hate speech, negativity, aggressiveness) and unweighted κ for target. Values in bold mark the best results per label. Models are identified by their Ollama tags. Thinking mode does not apply to Qwen 2.5 and Mistral (-). 34

Table 3.5 Best result per label and metric across all models and thinking settings on the 207 validation speeches. Each cell reports the best value and the model that achieved it. If a model was tested with thinking enabled and with thinking disabled, the setting is given in brackets. Cohen's κ is quadratic-weighted for hate speech, negativity and aggressiveness, and unweighted for target. 35

Table 3.6 Model `gemma4:e4b` compared on all the metrics where it does not perform the best. The difference is around around 5% with models that fit in VRAM, and around 13% with models too big to fit. On every single other metric, `gemma4:e4b` beats all other candidate models. 35

Table A.7 10-level topic hierarchy structure 58

Table A.8 10-level topic hierarchy structure 59

Table A.9 10-level topic hierarchy structure 59

Table A.10 Sample rows from `iot_records_tenant_01` (top) and `iot_records_tenant_02` (bottom). 60

Table A.11 Full validation results on the 207 speeches for all models, identified by their Ollama tags, with thinking mode off and on (- where thinking mode does not apply). In order: model, thinking mode, label, Cohen's κ (quadratic-weighted for hate speech, negativity and aggressiveness; unweighted for target), accuracy, and macro-averaged precision (P), recall

(R) and F1. Rows in bold mark the configuration adopted for each label (`gemma4:e4b`).
*Evaluated on 206/207 speeches, as one failed to be classified. 64

List of Figures

Figure 2.1 (a) 1.5 million tweets authored in the US in 2021, colored by estimated political alignment (blue is left-leaning, red is right-leaning). Tweets close together are semantically similar. Topical keywords have been plotted over dense clusters (colored similarly, by the share of red vs blue user tweets in the cluster). Some topics like food and birthdays are discussed regardless of political alignment. The blue areas stand out with everyday life topics (keywords like sleep, car, birthday). The top left blue corner are mostly bilingual tweets containing Spanish. Some political figures, religion and vaccination related topics appear more popular in the right-leaning subcorpus. The inset (b) is a heatmap of the same UMAP, colored by the average estimated sentiment of the tweets (purple negative through gray neutral to green positive). The political tweet cluster in the bottom right again stands out as notably more negative. This map illustrates how groups of people of opposing political alignment in the US, while sharing some topics of conversation, noticeably diverge in others. *Image and caption thanks to Karjus and Cuskley* [1] 9

Figure 3.2 Italian Prime Ministers from 1946 to 2025 and their time in office. In grey Prime Ministers not included in the corpus. Colors will remain consistent throughout this work. 14

Figure 3 Figure 3a show the number of words collected for the dataset over the span of eighty years. In green are words pertaining to center-leaning Prime Ministers, in red are words pertaining to left-leaning Prime Ministers, and in blue are words pertaining to right-leaning Prime Ministers. Figure 3b, on the other hand, shows the same data but in terms of tokens instead of whole words. 17

Figure 3.4 Italian Prime Ministers for each Italian Republic, from the first (1946) to the latest (2025). 39

Figure 5 5a and 5b show MTLD values for each Italian Prime Minster, ordered by their first time in office. This allows for not only a direct comparison between Prime Ministers, but also a comparison of the evolution of MTLD values over time. Prime Minister with fewer than 100 speeches are omitted. 42

Figure 6 6a and 6b show MTLD values for each Italian Prime Minster over their careers span. The LOWESS curve allows was chosen to aid in the interpretation of the values, and the overall medians allow for a comparison between Prime Ministers. Prime Minister with fewer than 100 speeches are omitted. 43

Figure 7 MTLD values in a Boxplot by 5-year periods. There seem to be a reduction in MTLD variance starting from 1989 all the way to 2019, whith an abrupt inversion in the 2020-2025 period. If this is a correlation with the inflection of some VDEM indices, it is not clear. 44

Figure 8 MTLD values in a Boxplot by 10-year periods. There seem to be a reduction in MTLD variance starting from 1989 all the way to 2025. If this is a correlation with the inflection of some VDEM indices, it is not clear. 45

Figure 9 MTLD values aggregated for each year, plotted against VDEM indices. There does not seem to be a correlation between a decrease in complexity and a decrease in democratic indices. Instead, the graph seems to suggest the opposite. 46

Figure 10 Heatmaps showing lexical similarity between Prime Ministers' speeches. 10a was made by pooling the 1000 most used words by each Prime Minister, then calculating how many words are shared between each of them. 10b, on the other hand, is made by calculating the Jaccard index between each pair of Prime Ministers, by pooling *all* words from all speeches used by each Prime Minister. 47

Figure 11 All speeches are pooled in 5-year time frames. Then, for each 5-year time frame, the top-N most frequently used words are extracted for each political leaning[1] (one thousands words for graph at 11a, and ten thousands words for graph at 11b). This results in three distinct sets[2] of words for left, right, and center leaning speeches respectively.

The graph shows, for each 5-year time frame, words in common among all leanings (in grey), and words unique for each leaning (in red, blue, and green respectively). For the years with most data (from 1985 to 2015), 11a shows a progressive increase in unique vocabularies compared to a decrease in shared ones, which can be interpreted as symptoms of increasing polarization. 48

Figure 4.12 Top-1000 words[3] from each Prime Minster. The embedding vectors are pretrained Italian word vectors (CBOW, 300 dimensions) taken from *fastText*. Words shared by all Prime Ministers are displayed as black rhombuses. The graph show a lot of clustering, suggesting a certain similarity among Prime Ministers. That being said, the graph is also too hard to read to gain any conclusive evidence. 49

Figure 13 Using `TfidfVectorizer` from `sklearn` library, I computed one embedding for each speech[4]. The embeddings can then be projected in two dimensions. 13a shows the projected speech vectors grouped by Prime Minsters. We can see clustering, suggesting that most prime ministers have a lexical style and wording that are unique to them. 13b groups the vectors by political leaning, showing that different leanings occupy different and distinct regions in the vector space. Both graphs strongly hints to a polarized political landscape.

On the other hand, 13c shows two main clustering regions: on the top, speeches from 1945 to 1955 gather, while speeches from 1991 to 2025 are more uniformly scattered across the whole vector space. This suggests that earlier speeches were significantly different compared to modern ones, while speeches from the nineties onward are quite similar. 50

Figure 14 Sentiment analysis results for *hate speech*, *aggressiveness*, and *negativity* plotted against VDEM indices. Its hart to tell wether there is any correlation between them. 51

[1] A speech is considered "right-leaning" if it pertains a right-leaning Prime Minister.

[2] When speeches for each leaning are present. Otherwise only two, one, or no set at all can be extracted from that 5-year time frame.

[3] Top-1000 words means pooling the one thousand most used words of a specific Prime Minister. Stopwords are removed and the lexicon is lemmatized.

[4] Each speech counts as a document.

Figure 15 Sentiment analysis results for *negativity* and *aggressiveness*. Values go from zero to one, and the classification task considers three possibile values for each speech: *low* (0), *mid* (0.5), and *high* (1).

Size of bubbles depends on the amount of speeches. 15a project results for single Prime Ministers: Berlusconi stands out as the most aggressive by far. 15b project results aggregated by leanings. The right appears to be more aggressive, while the left more negative. The center seems to be more positive than both. The last two figures show the sentiment scores aggregated by years (with 5-year pool range). While 15c shows an outlier for the period $1976 - 1980$, 15d shows that overall the sentiment scores are quite close to each other, without great variance especially for periods with a lot of speeches. 52

Figure 16 All graphs show the percentage of speeches that target political adversaries. 16a shows which Prime Minister speaks more often about the opposition (prime ministers with less than 100 speeches are omitted). 16b shows in which 5-year periods there are more speeches targeting political adversaries (ordered by year). Lastly, 16c shows which of the three political fields is more likely to speak about the opposing front. The results show how, across the board, more than one speech out of two targets the opposition. This is true regardless of time period or affiliation. 53

Figure 17 Overall, an extremely low percentage of speeches target minority groups. Therefore, I will ignore this analysis. (TODO explain better.. within error yada yada). 53

Figure A.18 Architecture of IoT (A: 3-layers) (B: 5-layers) 60

Figure A.19 Comparison between (a) Request-Response and (b) Publish-Subscribe paradigms 61

List of Codes

Code 3.1 An example of how a single speech in De Gasperi's Corpus is formatted in XML 19
Code 3.2 Example of the *"run"* pipeline to scrape and transcribe all Prime Ministers from Radio Radicale. 20
Code 3.3 Scraping the audio URLs of a single category 21
Code 3.4 Get all URLs of all speeches of a single Prime Minister, without duplicates. 22
Code 3.5 Helper functions to normalise a recording URL and extract its ID 22
Code 3.6 Downloading, merging and renaming the audio of a recording 23
Code 3.7 Parsing dates in the formats found on the pages 24
Code 3.8 Extracting the date of the event from the page 24
Code 3.9 Extracting the date of the #pm's intervention 25
Code 3.10 Parsing the identifier, start time and duration of an intervention 26
Code 3.11 Extracting date, location and timestamps of a #pm's speech 27
Code 3.12 Trimming a recording to the segments of the speaker 28
Code 3.13 Transcribing a speech and saving it with its metadata in a CSV file 30
Code 3.14 Downloading audio and metadata from a YouTube video 31
Code 3.15 Transcribe the audio file into text using Whisper 31
Code 3.16 Label values, model settings and output schema 37
Code 3.17 Classifying a single speech. 37
Code 3.18 Labelling the corpus in two passes. 38
Code A.19 Example of a Permify tuple 61
Code A.20 Example of a Permify tuple 61
Code A.21 Example of the HTML in Radio Radicale that contains a list of links pointing to the speeches of a Prime Minister. 61
Code A.22 Example of the HTML in Radio Radicale that contains a list of interventions in a conference. 63
Code A.23 Prompt to generate 100 synthetic speeches with high hostility and diverse targets. The speeches were generated with Gemini 3.6 Flash. 65
Code A.24 Prompts to be fed into Gemma 4 to classify the corpus and their respective JSON output schemas. One prompt to classify negativity and aggressiveness with thinking disabled , and one prompt to classify hate speech and target with thinking enabled. 67

List of Acronyms

MQTT.....Message Queuing Telemetry Transport

CoAP.....Constrained Application Protocol

IoT.....Internet of Things

BLE.....Bluetooth Low Energy

LoRa.....Long Range

LoRaWAN.....Long Range Wide Area Network

NB-IoT.....Narrowband Internet of Things

AES-CCM.....Advanced Encryption Standard - Counter with CBC-MAC

MAC.....Message Authentication Code

TCP.....Transmission Control Protocol

mTLS.....Mutual Transport Layer Security

M2M.....Machine to Machine

IAM.....Identity Access Management

HA.....High Availability

IDS.....Intrusion Detection Systems

ReBAC.....Relationship-Based Access Control

1 Introduction

Politicians are some of the most influential people in history: from the Greeks' demagogues to our Presidents and Prime Ministers, leaders have always tried to shape society. Shapiro and Page (1984) [2] showed how nearly all of the 20th century U.S. Presidents sought to steer public opinion, with varying degrees of success. As societies became more democratic, so grew the leaders' responsibility to convince potential followers that their policies are beneficial. Thus, **speeches** play a vital role in the functioning of society [3]: leaders depend on the verbal power to persuade people [4].

Moreover, democratic stability depends on citizens on the losing side accepting election outcomes. Clayton et al. (2021) [5] evaluated the effects of exposure to multiple statements from (at the time) former president Donald Trump[5] attacking the legitimacy of the 2020 US presidential election. While they found no evidence indicating that elites can erode democratic norms easily, elite rhetoric can shape people's views about core democratic values such as confidence in elections and support for peaceful transfers of power. This was especially evident amongst people who approved of Trump's performance in office.

Speeches can be differentiated between two types: *internal speeches*, aimed at other institutional bodies, such as parliamentary speeches, and *external speeches*, aimed at the population, such as public speeches, interviews and social network posts. Social networks are especially interesting since they afford politicians a direct and high-volume communication channel to their potential followers: U.S. President Donald Trump tweeted 57,000 times between May 2009 and January 2021 [6], while Italy's Prime Minister Giorgia Meloni totalled more than eighty million social network interactions in 2025 [7].

While television is still the first source of information for the majority of the population, online platforms are steadily growing and are already the preferred media for young people [8,9]. There is evidence of an increase in political participation due to social media usage, as well as risks for the functioning of democracy [10,11], and there have been attempts to predict election results with analyses on data from

[5] Former president when Clayton et al. wrote their paper, current president at the time of writing this Master Thesis.

social media. In Belcastro et al.'s 2022 study [12] a real-time analysis was carried out during the 2020 US presidential election campaign, correctly identifying the leading candidate before Election Day in 10 out of 11 swing states.

Opposite results can be seen in Rita et al.'s work (2023) [13], which measures sentiment polarity on Twitter, and concludes that tweets sentiment is not a reliable election results predictor. Additionally, results also show that it is impossible to state that social media impacts voting decisions.

Silva et al. (2022) [14], on the other hand, managed to match tweets against parliamentary speeches to measure politicians' sentiment on a specific topic. They find that parliament members who participate less in parliamentary debate tend to have larger differences with their party on Twitter, suggesting that a certain level of self-censoring is taking place in the parliamentary arena.

Circling back to the matter of speeches, external speeches often need to be scraped. Social networks can simplify this process through their exposed APIs and the availability of community-developed tools such as Python's library `ytp-dl`[6]. These resources open the possibility to automate speech collection (section 3.3.2).

On the other hand, internal speeches are usually already available in easy to access, *processing-ready* format[7], yet their technical and codified nature makes their interpretation harder for both humans and automated agents. Moreover, until 1985 the Italian stenographic documentation did not report speeches word for word, but rather translated them into a "language of the parliament" [15,16].

Broadly speaking, this project aims to compile a corpus of political speeches and subject it to linguistic analyses, progressing from classical NLP metrics such as word frequency and textual complexity to LLM-assisted classification tasks (hate-speech detection and target identification amongst them). This is why I ultimately chose to gather a corpus of **external speeches** rather than internal ones. Large language models are trained with tokenizers, and the resulting token distribution is highly imbalanced: Chung and Kim's controlled study (2025) [17] scaled the vocabulary of a set of language models from 24,000 to 196,000 tokens while holding data, computation, and optimization unchanged. They discovered that the models are disproportionately optimized on the part of language that occurs most frequently, meaning they are far better at understanding words and patterns that appear more often. This suggests that internal speeches are less understandable to AI agents.

Moreover, Birkenmaier and Lechner (2025) [18] made a similar dataset choice: they measured politicians' public personality traits using computational text analysis, and their corpus comprises data from interviews, social media posts, and parliamentary speeches. Specifically: "interview statements provide one of the most natural and authentic sources of politicians' interactions and self-presentation to the public", making them one of the best data sources for political speeches [19,20].

Let us now shift the **focus to the analyses** themselves.
There's a widespread view that populism is on the rise, from the United States and Turkey to India and Hungary [21–24], a development that has generated considerable concern because, in its most radical

[6] Privacy-focused media downloader API for Linux VPS deployments: https://pypi.org/project/ytp-dl/

[7] For example, the Italian Senate's website contains a database with every (recent) seating in HTML format. An example of seating n.1, March 23 2018, can be seen at the following link: https://www.senato.it/show-doc?tipodoc=Sindisp&leg=18&id=1066811

and authoritarian form, populism threatens democracy [25], polarizes societies [26], and erodes trust in experts [27].

Moreover, the progressive simplification of political discourse [28] has attracted increasing attention, as it allegedly favours populist politicians, particularly those on the far right. Many studies tried to explore these claims, with mixed results: Decadri and Boussalis (2019) [29] used text analytic techniques to inspect parliamentary speeches given by the members of nine Italian parties. Their results suggest that populist ideology, electoral strategy, and party membership influence legislators' language complexity, and that language simplicity might be thought of as a feature of populist communication.

On the other hand, McDonnell and Ondelli in their 2020 study [30] investigate the linguistic simplicity of four right-wing populists compared to their principal opponents in the United States, France, United Kingdom, and Italy. Contrary to expectations, they find that Donald Trump was only slightly simpler than Hillary Clinton, while Nigel Farage in the UK and Marine Le Pen in France were more complex than their main rivals, and Italy's Matteo Salvini was simpler on some measures but not others, concluding that the relationship between populism and simplicity should not be taken for granted.

Similar results were found by Rebecca C. Kittel in 2025 [28] after analyzing German parliamentary debates from January 1991 to September 2021. Their findings show that right-wing populist actors use the most complex language, while left-wing populists' language complexity is average. At the same time, the study finds that overall complexity decreased significantly in the German parliament over time, confirming at least partially the claims about "progressive simplification".

Other studies shifted the focus from textual complexity to the emotiveness[8] of speech. Wang et al. (2026) [32] explore the sound of populism by integrating classic Linguistic Inquiry and Word Count with a fine-tuned RoBERTa model. Their findings reveal that populist rhetoric consistently features a direct, assertive "sound" that forges a connection with "the people" and constructs a charismatic leadership persona. Notably, right-wing populism and people-centrism exhibit a more emotionally charged discourse, resonating with themes of identity, grievance, and crisis, in contrast to the relatively restrained emotional tones of left-wing and anti-elitist expressions.

Bliuc et al.'s work (2026) [33] highlights how anger consistently emerges as the most powerful emotional correlate of populist attitudes, followed by fear, resentment, and nostalgia, which agrees with Laura Alonso-Muñoz and Casero-Ripollés's 2023 study [34]. Their findings show that fear, uncertainty and resentment are the emotions most frequently used by European populist parties and leaders[9].

Caiani and Di Cocco (2023) [35], on the other hand, try to draw the connection between emotion and populism using machine learning: focusing on Italy as a case study, they systematically investigate the intensity and trends of specific emotions in political discourses of all Italian political parties over the last 20 years. Their findings confirm that populist politicians tend to leverage more emotional appeals than non-populist parties. However, there is an increase in the use of these appeals overall, especially in terms of negative emotions. Most notably, different types of emotions are mobilized by different types of populisms: right-wing populists mainly use negative emotions, while left-wing populists employ positive emotional appeals.

[8] *Emotive*, adjective: 1. of or relating to the emotions; 2. appealing to or expressing emotion; 3. causing strong emotions often in support of or against something [31].

[9] Alonso-Muñoz and Casero-Ripollés for their 2023 study built a corpus of tweets from four European political parties: Podemos, 5 Star Movement, National Front, and the UKIP [34].

Turning now to the effects populism has on society, polarization may be the one most consistently observed [36,37]. A lot of research has been done on the risks posed by a polarized society, such as Benson's (2023) [38], which argues that polarization's epistemic harms are best located in its tendency to reduce the diversity of perspectives utilized in a democratic system and in how this weakens the system's ability to identify and address problems of public concern.
Rostbøll (2024) [39] argues that "it is widely agreed that the increased polarization many countries experience is bad for democracy", and recommends a systemic approach to assess the democratic implication of polarization, which analyzes both the effects of polarization at different sites and on democracy as a composite whole.
Schedler (2023) [40] frames political polarization as a form of public conflict that aims to destroy the basic democratic trust. Citizens living in a polarized society fear for the subversion of democracy instead of their economic well-being, physical safety, or ways of life. They then argue that the institutional implications of such fears are dramatic, spelling the end of democratic consolidation.
Other literature argues that affective polarization contributes to democratic erosion by increasing partisan loyalty and decreasing the importance citizens give to democratic procedures. More specifically, the strength of partisanship has also been found to be associated with "partisan double standard" [41] or "democratic hypocrisy" [42], that is, the willingness to overlook democratic violations by one's own party.

This Master Thesis will explore the three verticals just discussed (textual complexity, polarization, and emotiveness) from a computational point of view.
The dataset consists of over five thousand annotated speeches delivered by Italian Prime Ministers between 1945 and 2025. Section 3.3 outlines how the corpus was built, and section 3.3.3 describes in detail how it was annotated to address the topic of "emotiveness": every speech is classified by its degree of aggressiveness, hate speech or negativity, as well as whether it targets a specific group or not. Possible target groups are: political adversaries, gender minorities, ethnic minorities, and religious minorities.
Section 4.1 showcases all the analyses performed on the matter of textual complexity, section 4.2 does so on the matter of polarization, while section 4.3 explores the matter of emotiveness.
Lastly, in section 5 the results are discussed as well as possible future work.

1.1 Research Questions

Before delving into the research questions, it is important to specify the term *hostility*.
In the pursuit of measuring emotiveness, I measured three qualities of speech (hate speech, aggressiveness, and negativity), and I classified whether a speech is targeting a group or not (political adversaries, gender minorities, ethnic minorities, and religious minorities).
I define the term *hostility* as the compound metric that incorporates all three aforementioned qualities[10].

[10] To clarify: if a speech has high negativity and high aggressiveness, it is a speech with medium to high hostility.

1. **Textual Complexity:**
   1.1. Does it have a correlation with the quality of democracy?
   1.2. Does it evolve over time? How?
   1.3. Does it change amongst different political leanings?
   1.4. Does lexical complexity evolution exhibit some common patterns amongst different Prime Ministers?
2. **Polarization:**
   2.1. Can we identify symptoms of polarization?
      2.1.1. Per Prime Minister;
      2.1.2. Per political leaning;
      2.1.3. Per year period;
   2.2. Do the symptoms evolve over the years? Do the evolution patterns change depending on the political leaning?
   2.3. Do the symptoms have a correlation with the quality of democracy?
3. **Emotiveness:**
   3.1. Can we identify hate speech?
   3.2. Can we identify negativity?
   3.3. Can we identify aggressiveness?
   3.4. Do emotionally charged speeches evolve over the years?
   3.5. What emotions are used the most? By which leaning? In which periods?
   3.6. Does hostility have a correlation with the quality of democracy?
   3.7. What Prime Minister is more hostile?
   3.8. What political leaning is more hostile?
   3.9. What period is more hostile? Is hostility evolving? How?
   3.10. Target:
      3.10.1. Which group is more often targeted?
      3.10.2. Which political leaning, if any, targets groups more often?
      3.10.3. Which Prime Minister, if any, targets groups more often?
      3.10.4. Which time periods, if any, have more speeches that target a group?

2 State of the Art

2.1 Textual Complexity

Text complexity or readability refers to how difficult a text is to understand [43], influenced by linguistic factors such as word choice (e.g., "utilize" vs. "use"), sentence structure (complex vs. simple), and content type (academic vs. children's books) [44,45].
Common comprehension metrics are Flesch Reading Ease [46], which considers the average sentence length and syllables per word, and outputs a value between zero (hard to read) and one hundred (easy to read); Dale-Chall [44], which measures a raw score taking in consideration a list of 3000 words that groups of fourth-grade American students could reliably understand, considering any word not on that list to be difficult; and SMOG [47], which uses intuitive features like sentence length and word complexity.

Another way to measure the complexity is by measuring the **lexical diversity**, which is the ratio of different unique word types compared to the total number of words. In a sense, it measures the vocabulary richness of a sample.
The most common metric is the Type-Token Ratio, obtained by simply dividing word types over tokens. A TTR of 0.5 means that half of the words in the sample appear only once.
A more refined metric was introduced by McCarthy in 2005: it is the measure of textual lexical diversity (MTLD) [48]. It uses a sequential algorithm to count tokens until lexical repetition forces the ratio below a fixed threshold, reducing text-length bias, and it's widely applied in computational linguistics for corpus profiling, tracking writing development, and evaluating machine translation quality. MTLD has become a standard tool in applied linguistics and computational text analysis, and it is often used as a measure of lexical diversity, lexical complexity, or lexical sophistication [49].

Let's now have a look at the recent research.
Bestgen (2024) [50] published a methodological review on the subject of lexical diversity, (type–token ratio), MTTRRS (mean type–token ratio in random samples), MATTR (moving-average type–token ratio), MSTTR (mean segmental type–token ratio), MTTRSS (mean type–token ratio in sequential samples), and MTLD (measure of textual lexical diversity). Its findings show that for a local estimation, MATTR and MTLD are recommended. While MTLD appeared to be slightly more sensitive to length than MATTR, the differences were insufficient to declare one superior to the other.

Velasco and Roque (2025) [51] explored the role of text complexity in language model pretraining. In their study, they measured how complexity affects language modeling across model sizes, whether useful representations can be learned from simpler text alone, and how text complexity in pretraining influences

downstream language understanding. They used a subset of *FineWeb-Edu* [52], then simplified by prompting *Llama 3.1 8B* [53] at the paragraph level.
To get a rough idea of what the simplified texts look like, see the following example:

> ***Original:*** *Your comment really helped me feel better the most. I was sitting in my office, feeling so bad that I didn't say how inappropriate and out of line his comments were, and this helped.*

> ***Simplified:*** *Your comment really helped me feel better. I was feeling bad because I didn't speak up when someone made inappropriate comments.*

Since LLM-based scoring is computationally costly, they used Flesch Reading Ease to estimate readability, and Type-Token Ratio to measure lexical diversity.

Amadori et al. (2025) [54] examined the linguistic complexity of content produced by influential users on Twitter across three globally significant and contested topics: COVID-19, COP26, and the Russia–Ukraine war. Their findings show significant differences across account type, political leaning, content reliability, and sentiment. Additionally, profiles producing more negative and offensive content tend to use more complex language.
They used three metrics: Flesch readability score; Yule's K measure of lexical richness [55], which quantifies how repetitive or diverse the vocabulary is, and it is largely independent of the overall length of a text; and gzip-based compression complexity, which assesses text repetitiveness by measuring its compression ratio, an approach already explored in previous studies [56–58].

Lastly, Fredrick and Craven (2025) [59] compared AI-generated texts (via ChatGPT) and student-written essays in terms of lexical diversity, syntactic complexity, and readability, using metrics such as Type-Token Ratio and Flesch-Kincaid. Results indicate that while ChatGPT produces texts with greater lexical diversity and syntactic complexity, the outputs are overall less readable.
They chose a range of linguistic complexity metrics: for lexical diversity they used TTR and MTLD, for syntactic complexity MLT and DC/T, for readability Flesc-Kincaid and Gunning-Fog, and lastly they also measured lexical sophistication.

For the purposes of this project, I decided to limit the scope to one lexical complexity metric. MTLD was chosen due to its wide adoption in literature and its robustness to text length: collected speeches (section 3) vary in length between one another, going from a couple of lines to more than twenty thousand words.

2.2 Polarization

Political polarization is the divergence of political attitudes away from the center, towards ideological extremes [60–62]. Scholars distinguish between ideological polarization (differences in ideological positions) and affective polarization (a dislike and distrust of political out-groups) [63], and between elite polarization (polarization of the political elites, such as elected officials), and mass polarization (the polarization of the general public) [64–66].

Affective polarization refers to the phenomenon where individuals' feelings and emotions towards members of their own in-group[11] become more positive, while their feelings towards members of the opposing out-group become more negative. This can lead to increased hostility, aggressive attitudes, and unwillingness to compromise or work together with people with different political views [67], and in extreme cases can even lead to societal disintegration and ideological sorting [68,69].

Elite polarization refers to polarization between the governing parties and the opposition [61]. Polarized political parties are internally cohesive, unified, and ideologically distinct, and they are typically found in a parliamentary system of democratic governance [70]. While in bipartisan systems the direction of polarization is intuitive, studies show that in multiparty systems it is not the number of parties itself, but the way parties interacts with one another that influences the magnitude and nature of affective polarization [71].
On the other hand, **mass polarization** occurs when an electorate's attitudes towards political issues, policies, celebrated figures, or other citizens are neatly divided along party lines [72]. At the extreme, each camp questions the moral legitimacy of the other, viewing the opposing camp and its policies as an existential threat to their way of life or the nation as a whole [73,74].

While there are no straightforward and established metrics to measure polarization, it has been estimated via a variety of methods.
McMurtrie et al. proposed in 2024 the Affective Polarization Scale [75]. They subjected participants to a series of statements. Each participant was asked to rate each statement on a 7-point scale, from *strongly disagree* to *strongly agree*. Questions were in the form of:

> *I would be upset if my friend married a* ____

The researchers filled in the blanks with the out-group respective to the single participant (meaning, for a Democrat-leaning participant, the blank could be filled with *"Conservative"*). The resulting score is then averaged across all statements.

A different approach was taken by Gentzkow et al. (2019) [76]. Their mathematical model revolves around the formula for partisanship of speech for given characteristics x:

$$\pi_t(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{q}_t^R(\boldsymbol{x}) \cdot \boldsymbol{\rho}_t(\boldsymbol{x}) + \frac{1}{2}\boldsymbol{q}_t^D(\boldsymbol{x}) \cdot (1 - \boldsymbol{\rho}_t(\boldsymbol{x})), \qquad 1.$$

They measured differences in speech patterns between Republicans and Democrats, finding a dramatic increase in polarized speech patterns starting from 1994.

Nettasinghe et al. (2025) [77] used a regression-based approach. They estimated two parameters (in-group love α, and out-group hate β) by measuring how likely it is for an individual to switch their stance on an opinionated topic (e.g., pro or anti lockdown during COVID). They took pairs of consecutive observations of a user's stance and fitted a logistic regression on the outcome *"did the stance change?"*. They used as predictors how supportive of said change their in-group and out-group were, and the coefficients resulting from the fitting are α and β.
To build the corpus, they used a fine-tuned Llama model to classify the stances of millions of tweets.

[11] In social psychology and sociology, an in-group is a social group to which a person psychologically identifies as being a member. By contrast, an out-group is a social group with which an individual does not identify.

Karjus and Cuskley (2023) [1] quantified linguistic divergence between left- and right-leaning Twitter users in the United States. They collected 1.5 million tweets authored in 2021 and vectorized them with a doc2vec text embedding. Then, they plotted each tweet as a UMAP dimension reduction [78], and colored each of them by their estimated political leaning. In doing so, they were able to visualize clustering that can potentially indicate polarization (figure 2.1).

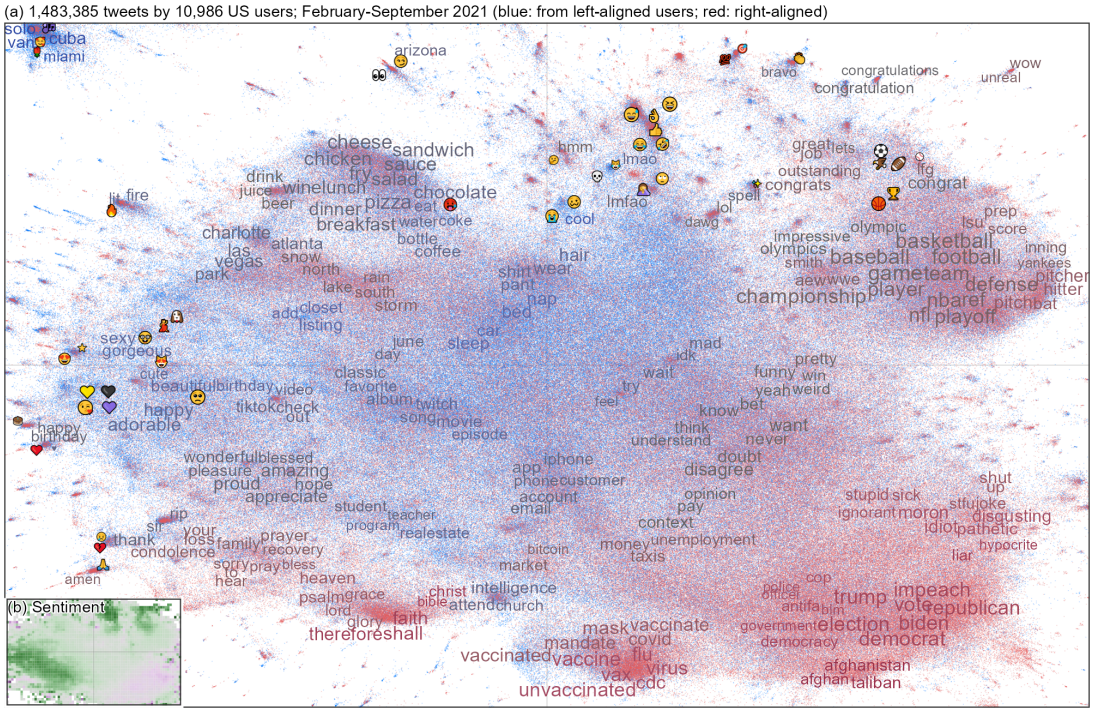


Figure 2.1: (a) 1.5 million tweets authored in the US in 2021, colored by estimated political alignment (blue is left-leaning, red is right-leaning). Tweets close together are semantically similar. Topical keywords have been plotted over dense clusters (colored similarly, by the share of red vs blue user tweets in the cluster). Some topics like food and birthdays are discussed regardless of political alignment. The blue areas stand out with everyday life topics (keywords like sleep, car, birthday). The top left blue corner are mostly bilingual tweets containing Spanish. Some political figures, religion and vaccination related topics appear more popular in the right-leaning subcorpus. The inset (b) is a heatmap of the same UMAP, colored by the average estimated sentiment of the tweets (purple negative through gray neutral to green positive). The political tweet cluster in the bottom right again stands out as notably more negative. This map illustrates how groups of people of opposing political alignment in the US, while sharing some topics of conversation, noticeably diverge in others. *Image and caption thanks to Karjus and Cuskley* [1]

In this project, I tried to identify and visualize polarization in two ways: first, by measuring the overlapping vocabulary among three political areas (left, right, and center) over time, and comparing it against those areas' exclusive vocabularies (section 4.2.2); and second, by projecting on a t-SNE graph all collected speeches vectorized with a doc2vec algorithm (section 4.2.3).

2.3 Sentiment Analysis

Sentiment analysis refers the the use of natural language processing, text analysis, and computational linguistics, to systematically identify, extract, quantify, and study the emotional qualities of a text. A classical example is evaluating the sentiment of a review (for example a movie review), a task that is often

aided by the presence of an explicit score in the review itself (for instance *"two stars out of five"*). With the rise of deep language models, harder data domains can be analyzed, such as news or opinion pieces where authors express their sentiment less explicitly [79].
A sentiment analysis task can be assess the verbal aggression in X's comments [80], measure the negative sentiment on social media during the COVID-19 outbreak [81], or detect hate speech in online debate [82].

The scope, on the other hand, can be mostly divided into three aspects: document, phrase, and aspect [83,84]. At document-level, the task is researching the emotion of the entire document, and each document is treated as an independent object. Mao et al. (2022) [85] tackled this by using an attention-based bi-directional long-short-term-memory network and a convolutional neural network.
Sentence-level aims to classify the sentiment polarity of a single sentence, categorizing each sentence into objective (a sentence that does not convey any opinion), or subjective (a sentence that presents the owner's thoughts and ideas) [86]. Chen et al. (2017) [87] used a sequence model to categorize sentences' sentiment polarity based on their target, while Su et al. (2023) [88] proposed a supervised sentence-level method based on gradual machine learning.
Lastly, aspect-level is a finer analysis where the aim is to model the relationship among the aspect term, aspect category, opinion term, and sentiment polarity [89]. For example, in the sentence "This restaurant's steak is delicious." the "steak" is an aspect term of the aspect category "food", "delicious" is the opinion term and the sentiment polarity of "steak" is positive.

When it comes to assess performances, sentiment analysis is framed as a classification task [90], so both standard classification metrics and sentiment-specific metrics are employed:

- **Accuracy:** number of correct predictions over total number of predictions. Misleading for imbalanced datasets;
- **Precision:** how many of the positive predictions are actually correct. Useful when the cost of false positives is high;
- **Recall (or sensitivity):** how many of the actual positive cases were correctly identified. Useful when the cost of false negatives is high;
- **F1:** harmonic mean of precision and recall;
- **Confusion Matrix:** used for binary classifiers, visualizes at a glance true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN);
- **Area Under the Curve (AUC):** compares binary classifiers by measuring the area under the ROC curve, which visualizes true positives rate and false positives rate;
- **Cohen's Kappa:** measure inter-rater reliability for qualitative or categorical data, and attempts to incorporate the possibility of the agreement occurring by chance.

In general, sentiment analysis is approached in four ways: lexicon-based approaches, traditional machine learning approaches, deep learning approaches and hybrid approaches [91–93].
Lexicon-based methods utilize a sentiment lexicon that assigns scores to tokens [94], meaning the token "bad" would have a negative score, while the token "good" would have a positive score. We can adopt dictionary adaptation techniques to mitigate locality problems, for example in the phrase "a long battery life", *"long"* has a positive connotation, while in "a long wait time", *"long"* connotation is negative.

Machine learning methods span from conventional classifiers, such as Naive Bayes [95] or support vector machines [96] to K-nearest neighbors [97]. More interesting are deep learning-based classifiers, for example we can use a convolutional neural network (CNN) as a feed-forward neural network with convolutional computation and pooling operation: Chen in 2015 [98] proposed a CNN-based sentiment analysis method built on word2vec for sentence-level sentiment categorization, outperforming rival approaches at the time. Xu et al. in 2016 [99] used a cached long short-term memory network, and LSTM networks often outperform both recurrent neural networks (RNN) and gated recurrent unit (GRU) when the amount of data becomes large [100].
To improve over pure machine learning-based approaches, Chang et al. (2020) [101] combined SVM and Relief algorithms, outperforming most models when 96 percent of the features are included, but the biggest improvement came after the introduction of the bi-directional transformer model BERT in 2018 [102]: Hoang et al. (2019) [103] showed the potential of mixing BERT's contextual word representations with fine-tuning and additional generated text, solving out-of-domain aspect-level sentiment analysis and outperforming previous results; Amira Samy Talaat (2023) [104] proposed four deep learning models based on a combination of BERT with Bidirectional Long ShortTerm Memory (BiLSTM) and Bidirectional Gated Recurrent Unit (BiGRU) algorithms for text sentiment classification with and without emoji, showing that BiGRU had the best results; Bello et al. (2023) [105] demonstrated that the combination of BERT with CNN, with RNN, and with BiLSTM performs well in terms of accuracy rate, precision rate, recall rate, and F1-score compared to pure BERT or BERT combined with Word2vec.

Lastly, most recent research focused on doing sentiment analysis with decoder-only models, most commonly known as large language models (LLMs). Sun et al. (2023) [106] proposed a multi-LLM negotiation framework for sentiment analysis, where two LLMs use rationale to persuade each other to reach consensus. The approach consistently yielded better performances than the baseline across all benchmarks; Ghatora et al. (2024) [107] applied machine learning-based classifiers (Random Forest, Naive Bayes, and Support Vector Machine) alongside the GPT-4 model to benchmark their effectiveness for sentiment analysis. Traditional models showed better results and efficiency in processing short, concise text, however GPT-4 showed better results with more detailed texts, showing that LLMs outperform traditional models in context-rich sentiment analysis; Jin et al. (2025) [108] proposed a self-adaptive framework to address common LLM hallucinations by bootstrapping instructions optimization. The framework integrates various in-context augmentation strategies, including emotion-oriented backgrounds, constraints, and analogical reasoning, and showed exceptional results, even surpassing human judgment in certain scenarios.

In this work, I use an LLM (specifically Gemma 4[12]) to classify Italian Prime Ministers' speeches from 1945 to 2025. The classification is multi-dimensional: three classes, each with three possible values, and a target class with four possible values[13]. The model's performance was assessed by measuring Cohen's Kappa on a pre-labeled validation set (all details in section 3.3.3).

[12] Gemma 4 models are designed to deliver frontier-level performance at each size. They are well-suited for reasoning, agentic workflows, coding, and multimodal understanding. I use the `gemma4:e4b` variant, via Ollama: https://ollama.com/library/gemma4:e4b

[13] The three classes are hate speech, negativity, and aggressiveness, and each can be classified, for each speech, as low, mid, or high. In addition, each speech can be classified with a target, either none (no target), political adversaries, gender minorities, ethnic minorities, or religious minorities.

3 The Corpus

The purpose of this chapter is to explain and outline the choices I made while building the dataset and to describe the resulting corpus. By following the steps in section 3.3, any external actor should be able to replicate the corpus as a whole.

3.1 Similar Corpora

The natural first step was to look at other corpora that aggregate similar data.

The **Europarl** dataset [109] aggregates proceedings of the European Parliament in twenty-one different European languages[14] with the goal of generating sentence-aligned text for statistical machine learning translation systems. The overall structure is akin to a collection of plaintext with minimal tagging.

ItaParlCorpus [110], on the other hand, is a comprehensive, annotated, and machine-readable database of Italy's parliamentary speeches spanning from 1948 to 2022. Each speech is identified by a precise date, the role of the speaker, their party name and family, and the legislature number. It is not, however, very rich in terms of linguistic processing tagging.

IMPAQTS [111] is a multimodal corpus of around 2.65 million tokens, including 1,500 speeches uttered by 150 prominent politicians, spanning from 1946 to 2023. For each speaker, the corpus contains 4 parliamentary speeches, 2 rallies, 1 party assembly, and 3 statements (in person or broadcasted). The goal is to annotate political rhetoric that hides a deeper meaning, for example by employing sarcasm. There are four main types of annotation: implication (conventional, generalized, etc.), presupposition (e.g., pragmatic), vagueness (syntactic, semantic, metaphoric), and topicalization (either syntactic or prosodic).

Parola di Leader, and specifically the corpus LP4 [112], is a corpus of more than five million occurrences taken from the stenographic archives of the Chamber of Deputies of Italy[15], between 1948 (first legislature of the Italian Republic) and 2011 (excluding the Monti Government, the 61st legislature).

They selected a list of thirty "highly influential politicians"[16] and then for each of them they selected some speeches. Overall, they collected 1877 distinct speeches.

[14] All 21 Europarl languages: Romanic (French, Italian, Spanish, Portuguese, Romanian), Germanic (English, Dutch, German, Danish, Swedish), Slavic (Bulgarian, Czech, Polish, Slovak, Slovene), Finno-Ugric (Finnish, Hungarian, Estonian), Baltic (Latvian, Lithuanian), and Greek.

[15] Chamber of Deputies official online public archive: https://legislaturepreceedenti.camera.it/.

[16] The full list of politicians follows: Giorgio Almirante, Giuliano Amato, Giulio Andreotti, Enrico Berlinguer, Silvio Berlusconi, Pier Luigi Bersani, Fausto Bertinotti, Rosy Bindi, Emma Bonino, Umberto Bossi, Pier Ferdinando Casini, Francesco Cossiga, Bettino Craxi, Massimo D'Alema, Alcide De Gasperi, Ciriaco De Mita, Antonio Di Pietro, Amintore Fanfani, Gianfranco Fini, Ugo La Malfa, Aldo Moro, Pietro Nenni, Achille Occhetto, Marco Pannella, Romano Prodi, Giuseppe Saragat, Giovanni Spadolini, Palmiro Togliatti, Valter Veltroni, Nichi Vendola.

The corpus could be considered a collection of thirty distinct corpora, one for each politician. Each of these has, among others, the following features: name of the politician, role of the politician (for example, Prime Minister), whether the politician is in the majority party or in the opposition, legislature number and sitting number of the day, date of the sitting, topic of the speech, name of the current Government (e.g., *Berlusconi I*), and lastly a reference to the file that holds the actual speech.

There is also a corpus for the vocabulary that holds, for each word, the total number of occurrences, a part-of-speech[17] tag, its lemma, a count for the number of occurrences for each politician, majority party and opposition, and lastly the imprinting tag (e.g., masculine singular).

Lastly, **ParlaMint** [114,115] is a compiled collection of comparable parliamentary corpora for a number of countries and languages. ParlaMint corpora are interoperable, i.e. encoded to a very constrained common ParlaMint schema, a specialization of the Parla-CLARIN[18] recommendations, which are a customization of the TEI Guidelines[19].

It is a collection of parliamentary sittings split into utterances. An example of two utterances could be *Politician: "I ask permission to speak."*, *President of the Senate: "Permission granted."*. For each utterance there are, among others, the following metadata: an ID for both the whole sitting and the utterance itself, a title of the sitting, a date of the sitting, the State Body where it took place (e.g., the Senate of the Republic), the name of the speaker and their affiliation, as well as the name of their coalition.

Speeches are collected before being split into their own datasets, and for each speech we have, among others, a unique ID and a date, the type of speech (e.g., *Interview* or *Rally*), the venue channel (e.g., *BBC News*), the unique ID of the speaker and their role (e.g., *host* or *main speaker*), the text of the speech itself and its type (e.g., *Question*, *Answer*, *Interruption*) and whether there have been any *Incidents* and their types.

Incidents are breaks in the flow of the conversation, and they can be of different types[20], for example *"leaving"* or *"break"*. If an incident is of the kinesic type, it can be further subdivided into different kinesic types, such as *"applause"* or *"laughter"*[21]. If the incident is of the vocal type, it can be further subdivided into different vocal types, such as *"question"* or *"shouting"*[22].

Speakers and Parties also have their own datasets, with unique IDs, names, affiliations, roles and the like.

Overall, the taxonomy of ParlaMint is strongly codified, which makes the dataset machine-readable and easier to interpret. All data is also saved in XML format.

[17] In grammar, a part of speech or part-of-speech (abbreviated as POS or PoS) is a category of words (or, more generally, of lexical items) that have similar grammatical properties [113].

[18] Parla-CLARIN, a TEI schema for corpora of parliamentary proceedings: https://clarin-eric.github.io/parla-clarin/.

[19] P5 TEI Guidelines: https://tei-c.org/guidelines/p5/.

[20] Full list of incident types: "action", "incident", "leaving", "entering", "break", "pause", "sound", "editorial".

[21] Full list of kinesic types: "kinesic", "applause", "ringing", "signal", "playback", "gesture", "smiling", "laughter", "snapping", "noise".

[22] Full list of vocal types: "greeting", "question", "clarification", "speaking", "interruption", "exclamat", "laughter", "shouting", "murmuring", "noise", "signal".

3.2 Corpus Structure

The corpus comprises speeches from twenty four out of the thirty one Italian Prime Ministers of the Italian Republic, as shown in figure 3.2. Prime Ministers from the Kingdom of Italy (1861-1946) are excluded.

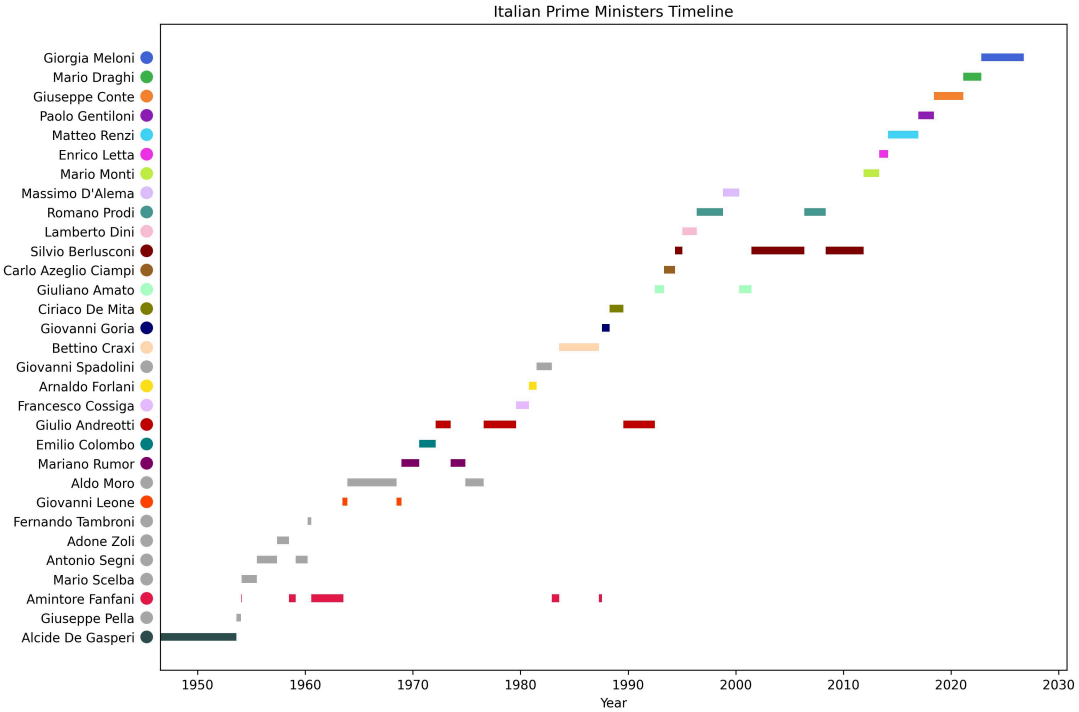


Figure 3.2: Italian Prime Ministers from 1946 to 2025 and their time in office. In grey Prime Ministers not included in the corpus. Colors will remain consistent throughout this work.

Overall, collected data amounts to 6, 038 speeches, for a total of 15, 185, 640 words. Table 3.1 surmise the overall corpus, while 3.2 shows metrics for each Prime Minister. One important notice: speeches were collected for each Prime Minister as a person, meaning that the corpus contains all the speeches delivered by each Prime Minister regardless of whether they were in office or not at the moment of the speech. For example, Giorgia Meloni's speeches date as far back as the sixteen of October 2000, even though she became Prime Minister twenty-two years later on the twenty-first of October 2022. The rationale is that the corpus is meant to track political rhetoric as a whole: politicians capable of becoming Prime Ministers were chosen as representatives of the political spectrum and ideas they embody. This also facilitated data collection, affording a denser and more informative dataset.

It is worth mentioning that reported tokens amount is obtained by using Spacy's tokenizer, so it will be slightly different in other use cases throughout the project. For example, corpus annotation is done with Gemma 4 via Ollama, which uses the model's own tokenizer (section 3.3.3).

Metric	Value
# Speeches	6,038
# Words (Tokens)	15,185,640 (16,878,514)
Median $\frac{\text{Words(Tokens)}}{\text{Speech}}$	2,132 (2,356)

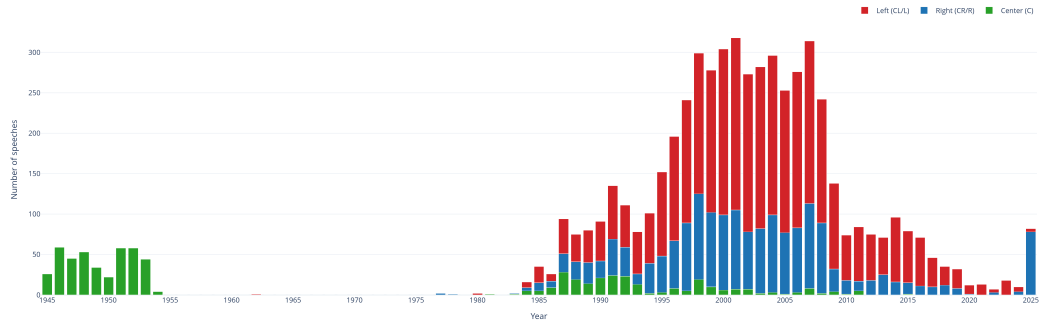
Table 3.1: Overall corpus size and pooled median speech length. First row shows how many total speeches were collected, second row shows how many words (and tokens) were collected, and third row shows the overall median speech length, in words (and tokens). Tokenization done via `spacy.load("it_core_news_sm")`.

PM	Time Period	# Speeches	# Words (Tokens)	Median $\frac{\text{Words(Tokens)}}{\text{Speech}}$
De Gasperi	1901-09-21 – 1954-07-24	473	534,228 (609,142)	701 (803)
Fanfani	1962-03-22 – 1993-07-25	21	26,520 (29,806)	774 (948)
Leone	1987-04-02 – 2007-09-09	8	7,712 (8,921)	886 (1,013)
Rumor	1986-09-03 – 1989-02-20	5	9,405 (10,234)	1,880 (2,100)
Colombo	1984-05-23 – 1984-05-23	1	3,031 (3,393)	3,031 (3,393)
Andreotti	1978-04-04 – 2009-05-11	400	1,014,190 (1,117,148)	1,926 (2,122)
Cossiga	1977-04-21 – 2009-06-11	225	539,510 (608,165)	2,014 (2,292)
Forlani	1985-12-01 – 2015-02-05	75	137,655 (153,626)	1,718 (1,779)
Spadolini	1981-12-14 – 1994-04-22	105	208,077 (229,816)	1,698 (1,853)
Craxi	1980-06-06 – 2005-09-17	114	228,924 (254,538)	1,786 (1,973)
Goria	1986-09-04 – 1993-05-22	42	85,183 (94,885)	1,930 (2,163)
De Mita	1985-02-15 – 2017-09-28	208	634,323 (706,142)	3,044 (3,328)
Amato	1984-03-28 – 2024-11-20	901	2,576,265 (2,799,551)	2,671 (2,877)
Ciampi	1990-07-04 – 2010-02-23	73	135,902 (152,299)	1,545 (1,701)
Berlusconi	1984-01-09 – 2019-12-12	535	1,490,217 (1,644,026)	2,188 (2,394)

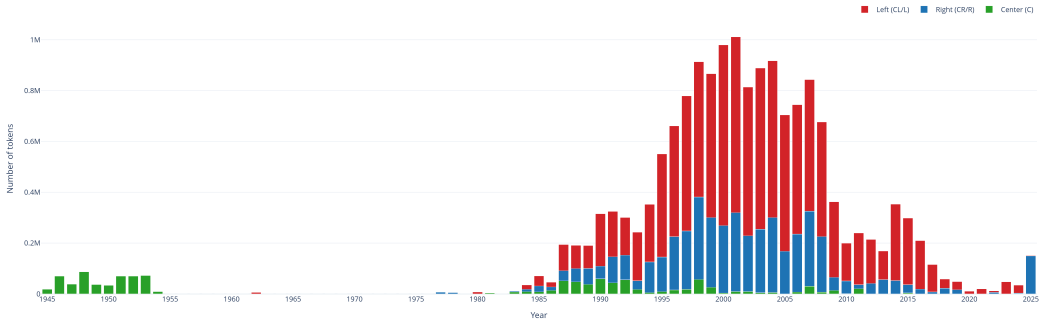
Dini	1989-02-21 – 2023-06-01	310	415,560 (469,214)	913 (1,018)
Prodi	1985-11-30 – 2024-11-09	515	1,508,664 (1,699,657)	2,622 (2,911)
D'Alema	1987-03-13 – 2024-04-12	829	2,888,875 (3,245,195)	3,158 (3,523)
Monti	1991-06-12 – 2017-03-14	240	605,443 (673,174)	2,096 (2,312)
Letta	1992-09-06 – 2021-11-23	509	1,177,761 (1,301,164)	2,176 (2,411)
Renzi	1997-06-13 – 2024-09-19	143	415,955 (467,270)	1,683 (1,789)
Gentiloni	2023-06-28 – 2023-06-28	1	566 (637)	566 (637)
Conte	2002-08-04 – 2026-02-24	30	50,177 (57,140)	471 (545)
Draghi	1996-06-07 – 2015-12-14	26	87,947 (99,206)	3,476 (3,897)
Meloni	2000-10-16 – 2026-05-21	249	403,550 (444,165)	1,186 (1,327)

Table 3.2: Corpus overview per prime minister. In order: Prime Minister's names, time period from which their speeches are pooled, how many speeches were pooled, how many words (and tokens) were pooled in total, and median speech length, in words (and tokens). Tokenization done via `spacy.load("it_core_news_sm")`.

Figure 3 visualizes all the collected speeches (and tokens) and their distribution over time. We can immediately see an imbalance both in therms of time (most data is concentrated between 1995 and 2010) and in terms of political leaning (left-leaning speeches are overrepresented). There is also a big gap between 1955 and 1985 where there is almost no data at all, and between 2019 and 2024 we see a strong reduction in data density, which shoots back up in 2025 (all Giorgia Meloni's speeches). Specifically, 3,936 speeches (65.2%) are concentrated between 1995 and 2010, and 5,363 (88.8%) between 1985 and 2020.



(a) Dateset: words. The graph show strong data imbalances: years between 1995 and 2010 contain most of the data, and left-leaning Prime Ministers are overrepresented.



(b) Dataset: tokens. Tokenization done via `spacy.load("it_core_news_sm")`.

Figure 3: Figure 3a show the number of words collected for the dataset over the span of eighty years. In green are words pertaining to center-leaning Prime Ministers, in red are words pertaining to left-leaning Prime Ministers, and in blue are words pertaining to right-leaning Prime Ministers. Figure 3b, on the other hand, shows the same data but in terms of tokens instead of whole words.

The structure of the dataset is as follows:

```
└─ crossdem/
 └─ datasets/
    └─ prime_ministers/
       ├─ pm_1/
       |   └─ csv_out/
       |       ├─ speech_1.csv
       |       ├─ speech_2.csv
       |       └─ ...
       ├─ pm_2/
       |   └─ csv_out/
       |       ├─ speech_1.csv
       |       ├─ speech_2.csv
       |       └─ ...
       ├─ ...
       └─ pm_N/
```

```
└─ csv_out/
   └─ ...
```

Each speech (with few exceptions) is structured with the following fields:

- `politician`: the name of the Prime Minister who delivered the speech, in lower case without spaces or punctuation (e.g., "dalema" and not "D'Alema");
- `historical_date`: the date when the speech was delivered, in the format *YYYY-MM-DD*;
- `location`: the location where the speech was delivered;
- `title`: the title of the video from which the speech was retrieved;
- `url`: the URL of the page from which the speech was retrieved;
- `audio_file`: the audio file of the speech in the local machine;
- `text`: the transcribed text of the speech;
- `hate_speech`: hate speech level, can be either `low`, `mid`, or `high`;
- `negativity`: negativity level, can be either `low`, `mid`, or `high`;
- `aggressiveness`: aggressiveness level, can be either `low`, `mid`, or `high`;
- `target`: target of the speech, can be either `none` if there is no target, `pol_adv` for political adversaries, `minor_etn` for ethnic minorities, `minor_gnd` for gender minorities, or `minor_rel` for religious minorities;

De Gasperi's corpus is structured differently, as it is not scraped but rather taken from a pre-existing dataset instead (section 3.3.1). Each speech has the following fields: `politician`, `historical_date`, `location`, `keywords` (extracted keywords about the speech itself),`text`, `hate_speech`, `negativity`, `aggressiveness`, `target`.

Some of Meloni's speeches also have a slightly different structure, as they were scraped from YouTube instead of Radio Radicale. They are easy to recognize in the dataset because all files that contain transcriptions of speeches scraped from Radio Radicale are in the form `123456_meloni_s2t.csv`, with the first characters being all numbers, while speeches scraped from YouTube are in the form `_DA9zjY_meloni_speech2text.csv`, with the first characters being a mix of letters, numbers, and symbols. Each of these speeches has the following fields: `politician`, `historical_date`, `location`, `tags` (tags of the YouTube video),`description` (description of the YouTube video),`title`, `url`, `audio_file`, `text`, `hate_speech`, `negativity`, `aggressiveness`, `target`.

3.3 Building the Corpus

This section will explain how the corpus was built, starting from the collection of the speeches, to their transcription, and finally their annotation. For clarification on the corpus' structure, refer to section 3.2.

3.3.1 Alcide De Gasperi's Corpus

I would like to thank Sara Tonelli for providing me with the De Gasperi's Corpus [116], a collection of Alcide De Gasperi's public documents with gold and silver annotation.

The corpus is a collection of 2,762 documents issued between 1901 and 1954, formatted into XML files which include metadata that covers not only the title, the date and the place of publication, but also key-concepts automatically extracted from each text (with the corresponding relevance score) and

genre labels manually assigned by domain experts. Furthermore, the release includes silver annotation for lemma, part of speech, person names and place names witPh associated coordinates in a CoNLL-like format.

An example of the XML formatting can be seen at listing 3.1. Thanks to the `\<genres\>` tag, I was able to extract 474 public speeches, each of which with a location, a precise date and a list of keywords.

```xml
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<document id="I.doc_7">
  <publication_date>1901-11-26</publication_date>
  <publication_place>
      <place_name>Trento</place_name>
      <place_latitude>46.0664228</place_latitude>
      <place_longitude>11.1257601</place_longitude>
  </publication_place>
  <keywords>
      <keyword score="39.16">first keyword</keyword>
      ...
      <keyword score="5.59">last keyword</keyword>
  </keywords>
  <genres>
      <genre>Speech Type</genre>
  </genres>
  <text>The speech itself if stored between these brackets</text>
</document>
```

Code 3.1: An example of how a single speech in De Gasperi's Corpus is formatted in XML

3.3.2 Scraping

In this section I will show how I scraped two websites to retrieve the speeches of the Italian Prime Ministers. The sites are YouTube (https://www.youtube.com/) and Radio Radicale https://www.radioradicale.it/.

3.3.2.1 Scraping Radio Radicale

The whole pipeline (scraping and transcribing) can be executed by running the script `~/crossdem/source/radrad_scraper.py`. Make sure to have the dependencies listed in `~/crossdem/requirements.txt`. To choose which Prime Minister to scrape, simply edit the `DATA` list at line 22 with the appropriate name and second half of the URL. For example, to scrape Amintore Fanfani, whose URL is https://www.radioradicale.it/soggetti/545/amintore-fanfani, add the following tuple in the `DATA` list: `("fanfani", "/soggetti/545/amintore-fanfani")`.

The pipeline works as follows: the script iterates through all URLs in `DATA`, and for each Prime Minister runs the main function `main(politician, SUBJECT_URL)`.
First, all URLs for all speeches of the given Prime Minister are collected by calling the function `get_all_audio_urls`, which returns a list of strings, each of which a URL for a speech.
Then, each speech gets processed. The details and metadata are collected with function `extract_speech_details`, while the audio is downloaded with function `download_audio_subprocess`. Since most audio files contain more than one speaker, they have to be trimmed to the timestamps of the interventions of the target Prime Minister. This is done with the function `trim_to_speaker`.
Then, the audio is transcribed with the function `speech_to_text` and lastly all downloaded audios are trimmed down to 1 second to save space. If, for any reason, the pipeline halts, the failed speech is logged in `~/crossdem/logs/discarded.log` and the pipeline continues with the next speech.
Each video is fully processed (download audio, trim, transcribe, and save in a csv file) before moving on to the next one. This prevents losing progress in case of errors, since the code can be re-run at any time

and resumes from where it left off.

A summary of the `run` pipeline can be seen in listing 3.2. The source code for the script is available in the repository at the link: https://github.com/mhetacc/crossdem/blob/main/source/radrad_scraper.py.

```python
def main (politician, SUBJECT_URL):
    urls = get_all_audio_urls(SUBJECT_URL=SUBJECT_URL)
    for url in urls:
        # Skip any file already processed
        processed_ids = get_processed_ids(AUDIO_DIR)
        scheda_id = extract_id_from_url(url)
        if scheda_id in processed_ids:
            print(f"Skipping {scheda_id}, already processed.")
            continue

        try:
            speech_details = extract_speech_details(url, speaker=politician)
            timestamps = speech_details["interventions"]
            if any(i["start_time"] is None for i in timestamps):
                raise ValueError("No timestamps") #skip

            # Download metadata, trim to speaker timestamps, and transcribe
            audio_metadata = download_audio_subprocess(
                              url,
                              AUDIO_DIR,
                              politician)
            audio_path = trim_to_speaker(
                          audio_metadata["filename"],
                          timestamps,
                          AUDIO_DIR)
            speech_to_text(
              audio_metadata,
              speech_details,
              audio_path,
              politician, OUT_DIR, AUDIO_DIR, CSV_DIR, url)

            trim_to_1s(AUDIO_DIR)
        except Exception as e:
            log_discard(politician, url, e)   # Log which speech failed and why
            continue                          # Move on to the next speech

if __name__ == "__main__":
    for politician, SUBJECT_URL in DATA:
        main(politician, f"{BASE_URL}/{SUBJECT_URL}")
```

Code 3.2: Example of the *"run"* pipeline to scrape and transcribe all Prime Ministers from Radio Radicale.

Let's now explain the pipeline's main phases in greater detail.

3.3.2.1.1 Speeches URL Extraction

Since most Prime Ministers have hundreds of speeches, selecting each one by hand (as I did for Giorgia Meloni's speeches on YouTube, section 3.3.2.2) would be asinine. Instead, I wrote a function that, given the URL of a Prime Minister's page on Radio Radicale, scrapes all the URLs of all their public speeches (parliamentary speeches are excluded).

In Radio Radicale's website, the speeches of each Prime Minister can be filtered by categories (for example, "All" or "Interviews"). The categories to scrape are defined in the `CATEGORIES` dictionary, which maps each category name to its filter value. The *Istituzioni* category is left out, as it contains the parliamentary speeches.

The function `_scrape_category` (listing 3.3) scrapes a single category. It requests the Prime Minister page with the category filter and the page number (many Prime Ministers have multiple pages of

speeches) as query parameters, and parses the HTML with the Python library *BeautifulSoup*[23]. The resulting page contains, among the usual elements such as header, footer, and menus, a list of links that point to speeches[24]. The code iterates through all list elements `<li>`, extracting the absolute ULRs for the ones pointing to a speech. This process is repeated for each successive page. A pause of 0.5 seconds between requests avoids overloading the server. A the end of this process, a list containing all URLs for a specific category (of a specific Prime Minister) is returned.

```python
def _scrape_category(session, cat_value, SUBJECT_URL):
    urls = []
    page = 0

    while True:
        params = {"field_registrazione_raggruppamenti_radio": cat_value,
                  "page": page}
        resp = session.get(SUBJECT_URL, params=params,
                           headers=HEADERS, timeout=20)
        resp.raise_for_status()
        soup = BeautifulSoup(resp.text, "html.parser")

        for li in soup.select("ol.lista_list li.views-row"):
            if not li.select_one("div.tipo_media.audio"):
                continue
            a = li.select_one("div.ls_text h3 a")
            if a and a.get("href"):
                urls.append(urljoin(BASE_URL, a["href"]))

        if not soup.select_one("li.pager__item--next a"):
            break

        page += 1
        time.sleep(0.5)

    return urls
```

Code 3.3: Scraping the audio URLs of a single category

The main function, `get_all_audio_urls` (listing 3.4), calls `_scrape_category` on every category and merges the results. The same recording can appear under more than one category, and multiple times in the same category. Speeches URL are in the from *".../scheda/55884/...?i=00000"*, and by stripping everything after the key *"?"* duplicates can be removed, since they point to the same *"scheda"*, meaning to the same speech. The function thus returns a list of unique URls, which point to all the speeches of the target Prime Minister.

[23] Beautiful Soup is a library that makes it easy to scrape information from web pages. It sits atop an HTML or XML parser, providing Pythonic idioms for iterating, searching, and modifying the parse tree. Source: https://pypi.org/project/beautifulsoup4/

[24] An example of the HTML can be seen in appendix A.21

```python
def get_all_audio_urls(categories=None, verbose=True, SUBJECT_URL="."):
    if categories is None:
        categories = CATEGORIES

    session = requests.Session()
    all_urls = []
    seen_paths = set()

    for label, value in categories.items():
        cat_urls = _scrape_category(session, value, SUBJECT_URL)
        new = []
        for u in cat_urls:
            path = u.split("?")[0]
            if path not in seen_paths:
                seen_paths.add(path)
                new.append(u)
        all_urls.extend(new)

    return all_urls
```

Code 3.4: Get all URLs of all speeches of a single Prime Minister, without duplicates.

3.3.2.1.2 Speech Download

The audio of each speech is downloaded with `yt-dlp`, a command line and Python tool for downloading media from web pages[25]. The function `download_audio_subprocess` (3.6) takes the URL of a speech page, an output directory and the name of the Prime Minister, and saves the audio as `<id>_<pm>.mp3`.

The URLs collected in the previous step have to be sanitized, going from *".../scheda/55884/..."* to *".../scheda/55884"*. The function `sanitize_url` (3.5) reduces them to the form *"https://www.radioradicale.it/scheda/"*, and `extract_id_from_url` returns the numeric ID which is then used in the file name, so that each file can be traced back to its recording page.

```python
def sanitize_url(page_url: str) -> str:
    base = page_url.split("?")[0]
    parts = base.split("/")
    return "/".join(parts[:5])  # https: + '' + domain + scheda + ID

def extract_id_from_url(url):
    parts = url.split("/scheda/")
    if len(parts) > 1:
        return parts[1].split("/")[0]
    return None
```

Code 3.5: Helper functions to normalise a recording URL and extract its ID

`yt-dlp` is called through `subprocess`. Spawning a process each time is not a big concern, since between downloading the speech (often one hour long if not more) and transcribing it, this operation is never done more than five to ten times per hour. Only the audio is downloaded (`-x`) and then converted to MP3 at the highest quality (`--audio-quality 0`). Recordings split into multiple parts are treated by `yt-dlp` as a playlist, so each part is saved with its playlist index in the file name, and `--concat-playlist always` merges the parts into a single file, which in the end overwrites all the previous parts so that only one file per speech remains.

The metadata of the recording is fetched with a second call to `yt-dlp` (`--dump-json`), which does not download anything. The output is parsed as JSON, and an empty dictionary is used if the call or the parsing fails. The file name, the recording ID and the URL are added to the dictionary, which is returned to be used by other functions in the script.

[25] `yt-dlp` is a feature-rich command-line audio/video downloader with support for thousands of sites. Source: https://github.com/yt-dlp/yt-dlp.

```python
def download_audio_subprocess(url, out_dir, politician="politician"):
    url = sanitize_url(url)
    id = extract_id_from_url(url)
    stem = f"{id}_{politician}"
    final_file = f"{out_dir}/{stem}.mp3"

    result = subprocess.run([
        "yt-dlp",
        "-x",
        "--audio-format", "mp3",
        "--audio-quality", "0",
        "--concat-playlist", "always",
        "-o", f"{out_dir}/{stem}_part%(playlist_index)s.%(ext)s",
        url
    ], text=True)
    if result.returncode != 0:
        raise RuntimeError(f"yt-dlp exited {result.returncode}")

    # concatenation overwrites the first part
    concat_result = f"{out_dir}/{stem}_part0.mp3"
    if not os.path.exists(concat_result):
        candidates = glob.glob(f"{out_dir}/{stem}*.mp3")
        if not candidates:
            raise FileNotFoundError(f"No mp3 found for stem {stem}")
        concat_result = candidates[0]
    os.rename(concat_result, final_file)

    meta_result = subprocess.run([
        "yt-dlp", "--dump-json", "--no-playlist", "--flat-playlist", url
    ], capture_output=True, text=True)

    info = {}
    if meta_result.returncode == 0 and meta_result.stdout.strip():
        lines = [l for l in meta_result.stdout.splitlines() if l.strip()]
        try:
            info = json.loads(lines[-1])
        except json.JSONDecodeError:
            pass

    info["filename"] = os.path.basename(final_file)
    info["file_id"] = id
    info.setdefault("url", url)
    return info
```

Code 3.6: Downloading, merging and renaming the audio of a recording

3.3.2.1.3 Extract Speech Metadata and Timestamps

Each recording page on Radio Radicale lists the interventions of all the speakers at an event, so we need to trim the downloaded audio to only the parts (can be more than one) where the target Prime Minister speaks. For example, in a 1-hour long debate, the target Prime Minister could intervene between minute 14 and minute 20, and between minute 48 and minute 53.

The function `extract_speech_details` (listing 3.11) takes the URL of a recording page and the name of the Prime Minister, and returns the date, the location and the timestamps of all their interventions. The page is fetched by `fetch_page`, which sends a GET request with browser-like headers and returns the parsed HTML as a *BeautifulSoup* object.

Historical Date. Different dates appear in the pages in several formats, and the helpers in listing 3.7 handle four of them: with the full Italian month name (*22 marzo 1962*), with the abbreviated month (*22 mar 1962*), dotted (*22.03.1962*) and ISO (*1962-03-22*). The function `find_date` tries them in this order and returns the first match. Each helper has the same structure as `_try_long`, so only the latter is shown.

```
MONTHS_IT = {"gennaio": 1, "febbraio": 2, ..., "dicembre": 12}
MONTHS_IT_SHORT = {"gen": 1, "feb": 2, ..., "dic": 12}

RE_DATE_LONG = re.compile(
    r"\b(\d{1,2})\s+(" + "|".join(MONTHS_IT) + r")\s+(\d{4})\b", re.I)
RE_DATE_SHORT = ...   # same, with MONTHS_IT_SHORT
RE_DATE_DOTTED = re.compile(r"\b(\d{1,2})\.(\d{2})\.(\d{4})\b")
RE_DATE_ISO = re.compile(r"(\d{4})-(\d{2})-(\d{2})")

def _try_long(text):
    m = RE_DATE_LONG.search(text)
    return date(int(m[3]), MONTHS_IT[m[2].lower()], int(m[1])) if m else None

def find_date(text):
    return (_try_long(text) or _try_short(text)
            or _try_dotted(text) or _try_iso(text))
```

Code 3.7: Parsing dates in the formats found on the pages

The date of the event is extracted by `extract_page_date` (listing 3.8). The sources are checked from the most to the least reliable: the metadata tags, the page title, a date with abbreviated month in the page text, and a date with full month name in the page text. The function returns the date together with a label indicating which source it came from.

```
def extract_page_date(soup):
    for attr, val in [("name", "dcterms.date"),
                      ("property", "article:published_time")]:
        tag = soup.find("meta", {attr: val})
        if tag and tag.get("content"):
            d = _try_iso(tag["content"])
            if d: return d, "meta_tag"

    title = soup.find("title")
    if title:
        d = _try_dotted(title.get_text())
        if d: return d, "page_title"

    page_text = soup.get_text(" ", strip=True)
    d = _try_short(page_text)
    if d: return d, "page_stamp"
    d = _try_long(page_text)
    if d: return d, "page_sommario"

    return None, "not_found"
```

Code 3.8: Extracting the date of the event from the page

Sometimes the date of the speech is explicitly mentioned (for example *"L'on. Fanfani a una conferenza stampa del 22 marzo 1962..."*), and it can be different to any other date fetched with `extract_page_date`. Since such dates are more precise, I extract them with `extract_speaker_date` (listing 3.9). The speaker-level date has the priority, and is thus used instead of the page-level date (if present).

```python
def extract_speaker_date(soup, speaker):
    items = [li for li in soup.select("li.intervento") if li.find("h2")]

    for idx, li in enumerate(items):
        if speaker.lower() not in li.find("h2").get_text().lower():
            continue

        subtext = li.find("div", class_="int_subtext")
        if subtext:
            d = find_date(subtext.get_text())
            if d: return d, "speaker_subtext"

        if idx > 0:
            prev = items[idx - 1].find("div", class_="int_subtext")
            if prev and speaker.lower() in prev.get_text().lower():
                d = find_date(prev.get_text())
                if d: return d, "preceding_subtext"

        d = find_date(li.get_text(" ", strip=True))
        if d: return d, "speaker_block"
        return None, "speaker_block_no_date"

    return None, "speaker_not_found"
```

Code 3.9: Extracting the date of the #pm's intervention

Location. The location is read from the `primo_suffisso` element of the page by `extract_location`. A regular expression matches the uppercase name between two hyphens that precedes the time of the event (e.g. *- ROMA - 17:00*). If no location is found, or if it is the generic value `RADIO`, the location is set to `UNKNOWN`.

Timestamps. Each video in Radio Radicale has a column associated with it that contains a list of all timestamps for each intervention of each speaker. An example HTML can be seen at appendix A.22. Each intervention is a list item that contains, among others, the class *"durata"* which stores the starting timestamp of an intervention and how long the intervention lasts (for example `<div class="durata">1:07 Durata: 16 min</div>`). The function `parse_timestamps` (listing 3.10) extracts the timestamps. The start times are written in one of two ways: as time elapsed since the start of the recording (e.g., *0:30* meaning half an hour since the start), or as the time of the day (e.g., *17:30*). In the second case, the start time is converted into the elapsed time format, meaning if starting time is *17:00*, and *intervention A* starts at *17:30*, then *intervention A* starting time is converted to *0:30*.
To differentiate between the two cases, the script looks at the start time of the first intervention. If its value is *0:00*, the format is considered elapsed time.

```python
RE_DURATA = re.compile(r'(\d+:\d+)\s+Durata:\s+(\d+)\s+min', re.I)
RE_INT_ID = re.compile(r'^int(\d+)$')
RE_D_SEC  = re.compile(r'^d(\d+)$')

def parse_timestamps(li, is_clock_time=False, event_start=None):
    result = {"int_id": None, "start_time": None,
              "duration_min": None, "duration_seconds": None}

    for cls in li.get("class", []):
        if m := RE_INT_ID.match(cls):
            result["int_id"] = int(m.group(1))
        if m := RE_D_SEC.match(cls):
            result["duration_seconds"] = int(m.group(1))

    durata_div = li.find("div", class_="durata")
    if durata_div and (m := RE_DURATA.search(durata_div.get_text())):
        raw_time = m.group(1)
        result["duration_min"] = int(m.group(2))

        if is_clock_time and event_start:
            h0, m0 = (int(x) for x in event_start.split(":"))
            h1, m1 = (int(x) for x in raw_time.split(":"))
            offset = (h1 * 60 + m1) - (h0 * 60 + m0)
            result["start_time"] = f"{offset // 60:02d}:{offset % 60:02d}"
        else:
            result["start_time"] = raw_time

    return result
```

Code 3.10: Parsing the identifier, start time and duration of an intervention

Main function. `extract_speech_details` (listing 3.11) combines the previous steps. It returns a dictionary with the page URL, the date and its source, the location and the list of the Prime Minister's interventions. All the interventions of the Prime Minister are collected, not only the first, and the match on the speaker name is case-insensitive. If the page cannot be fetched, the error is stored in the dictionary. If the Prime Minister is not found among the interventions, the error is stored too. The date of the speech is the one found by `extract_speaker_date`, with the date of the event as fallback. When neither is found, the source is set to `llm_needed`, which flags the recording for separate handling. Thankfully, this was never needed while collecting the data.

```python
def extract_speech_details(url, speaker, timeout=20):
    result = dict(
        url=url, speaker_query=speaker, speaker_found=False,
        speech_date=None, date_source=None, page_event_date=None,
        interventions=[], error=None,
    )
    try:
        soup = fetch_page(url, timeout=timeout)
    except Exception as e:
        result["error"] = str(e)
        return result

    page_date, page_src = extract_page_date(soup)
    if page_date:
        result["page_event_date"] = page_date.isoformat()

    location = extract_location(soup)
    result["location"] = location if (location is not None and location != "RADIO")
else "UNKNOWN"

    event_start = None
    first_li = soup.select_one("li.intervento")
    if first_li and (d := first_li.find("div", class_="durata")):
        if m := RE_DURATA.search(d.get_text()):
            event_start = m.group(1)
    is_clock_time = event_start != "0:00"

    for li in soup.select("li.intervento"):
        h2 = li.find("h2")
        if h2 and speaker.lower() in h2.get_text().lower():
            result["speaker_found"] = True
            ts = parse_timestamps(li, is_clock_time, event_start)
            if ts["start_time"] or ts["duration_seconds"] is not None:
                result["interventions"].append(ts)

    if not result["speaker_found"]:
        result["error"] = f"'{speaker}' not found in interventi"
        result["date_source"] = "llm_needed"
        return result

    if not result["interventions"]:
        result["interventions"] = "no timestamps"

    spk_date, spk_src = extract_speaker_date(soup, speaker)
    if spk_date:
        result["speech_date"] = spk_date.isoformat()
        result["date_source"] = spk_src
    elif page_date:
        result["speech_date"] = page_date.isoformat()
        result["date_source"] = f"page_event ({page_src})"
    else:
        result["date_source"] = "llm_needed"

    return result
```

Code 3.11: Extracting date, location and timestamps of a #pm's speech

3.3.2.1.4 Trim Audio to Timestamps

Almost all videos in Radio Radicale's website are integral recordings of events, such as conferences or interviews, and they often include more than one speaker. Only the target Prime Minister's interventions are needed, so the audio file is trimmed to those segments. The function `trim_to_speaker` (listing 3.12) takes the name of the audio file, the list of interventions returned by `extract_speech_details` and the audio directory. It uses `ffmpeg`[26] through `subprocess` (once again, for the reasons mentioned above the overhead of spawning new processes can be ignored).

[26] FFmpeg is a complete, cross-platform solution to record, convert and stream audio and video. In this project I used the command line tool installed via Fedora's package manager. Source: https://ffmpeg.org/.

The start time of each intervention is converted from the `h:mm` format into seconds by `hhmm_to_seconds`. For each intervention, `ffmpeg` cuts the segment that starts at that time and lasts `duration_seconds`, and saves it to a temporary file. The audio stream is copied (`-acodec copy`) instead of being re-encoded, which is fast and does not degrade the audio quality. The segments are then listed in a text file and joined with the `concat` command into a single file, in the order of the interventions. The temporary files are deleted and the result replaces the original file, so in the end one trimmed audio file per event is kept.

```python
def trim_to_speaker(audio_filename, interventions, AUDIO_DIR):
    input_path = Path(AUDIO_DIR) / audio_filename

    def hhmm_to_seconds(t):
        h, m = t.split(":")
        return int(h) * 3600 + int(m) * 60

    # extract each segment to a temporary file
    tmp_files = []
    for i, iv in enumerate(interventions):
        tmp = input_path.with_name(f"_tmp_{i}_{input_path.name}")
        cmd = [
            "ffmpeg", "-y",
            "-ss", str(hhmm_to_seconds(iv["start_time"])),
            "-i", str(input_path),
            "-t", str(iv["duration_seconds"]),
            "-acodec", "copy",
            str(tmp),
        ]
        subprocess.run(cmd, stdout=subprocess.DEVNULL,
                       stderr=subprocess.DEVNULL)
        tmp_files.append(tmp)

    # concatenate the segments
    concat_list = input_path.with_name("_concat_list.txt")
    concat_list.write_text("\n".join(f"file '{f.name}'" for f in tmp_files))

    tmp_out = input_path.with_name(f"_out_{input_path.name}")
    cmd = [
        "ffmpeg", "-y",
        "-f", "concat", "-safe", "0",
        "-i", str(concat_list),
        "-acodec", "copy",
        str(tmp_out),
    ]
    subprocess.run(cmd, stdout=subprocess.DEVNULL,
                   stderr=subprocess.DEVNULL)

    # remove temporary files and overwrite the original
    for f in tmp_files:
        f.unlink(missing_ok=True)
    concat_list.unlink(missing_ok=True)
    tmp_out.replace(input_path)

    return input_path
```

Code 3.12: Trimming a recording to the segments of the speaker

3.3.2.1.5 Speech Transcription

The trimmed audio is transcribed with OpenAI's Whisper library, which uses an encoder-decoder Transformer model. The model size I used is the *"medium"*[27], which at 5 GB fits within the total 6 GB of VRAM available in my RTX 4050 GPU. The model was instructed to predict Italian.

The difference in precision between the model sizes *tiny* (which requires less than 1 GB of VRAM) and *medium* is quite high, as shown in the following transcriptions of the same audio file:

[27] Whisper model sizes available are tiny, base, medium and large.

> ***Model tiny:*** *"iamo con se è beruto fuori da questa rio neone sotto il profiro tecnico per quanto riguarda la franha e come il governo un tino era ad essere vicino alla popolazione di Nishenia."*

> ***Model medium:*** *"Diciamo cosa è venuto fuori da questa riunione sotto il profilo tecnico per quanto riguarda la frana e come il governo continuerà ad essere vicino alla popolazione diniscemeca."*

The transcription obtained with the model *medium* is very close to the original:

> ***Original speech:*** *"Diciamo cosa è venuto fuori da questa riunione sotto il profilo tecnico per quanto riguarda la frana e come il governo continuerà ad essere vicino alla popolazione di Niscemi[28]."*

The function `speech_to_text` (3.13) takes as parameters the metadata returned by `download_audio_subprocess`, the details returned by `extract_speech_details`, the name of the Prime Minister and the directories to use. It transcribes the audio and saves the transcript, together with the metadata of the speech, in a CSV file.

Whisper is called through `subprocess` with the command line interface. The language and the model are set by the variables `lang_code` and `model_name`, and the transcription runs on the GPU (`--device cuda`). The call uses `check=True`, so that an error in Whisper raises an exception (which then causes the script to skip to the next speech).

The transcript and the metadata are then written to a CSV file named `<id>_<pm>_s2t.csv`. Each row contains the name of the Prime Minister, the date of the speech, the location, the title of the recording, the URL of the recording page, the path of the audio file and the whole speech transcribed in text (in the field `text`). The line breaks in the title and in the text are replaced with spaces, so that each speech occupies a single line. The date is converted to a `date` object, and left empty if it was not found. The path of the audio file is stored relative to the project directory.

[28] Niscemi is a small city and comune in the free municipal consortium of Caltanissetta, Sicily, Italy.

```python
def speech_to_text(audio_metadata, speech_details, audio_path,
                   politician, OUT_DIR, AUDIO_DIR, CSV_DIR, url):
    historical_date = speech_details["speech_date"]
    location = speech_details["location"]
    title = audio_metadata["fulltitle"]
    file_id = audio_metadata["file_id"]
    filename = audio_metadata["filename"]

    output_dir = str(Path(OUT_DIR) / f"output-{model_name}")
    audio_file = str(Path(AUDIO_DIR) / filename)
    csv_output = str(Path(CSV_DIR) / f"{file_id}_{politician}_s2t.csv")

    # transcription
    if not os.path.exists(audio_file):
        raise FileNotFoundError(f"Audio file '{audio_file}' not found.")

    subprocess.run([
        "whisper",
        "--language",        lang_code,
        "--word_timestamps", "True",
        "--model",           model_name,
        "--output_dir",      output_dir,
        "--device",          "cuda",
        audio_file
    ], check=True)

    audio_path = Path(audio_file)
    transcript_path = Path(output_dir) / f"{audio_path.stem}.txt"
    if not transcript_path.exists():
        raise FileNotFoundError(f"Transcript not found at '{transcript_path}'.")

    text = transcript_path.read_text(encoding="utf-8").strip()

    # CSV row
    audio_path_str = str(audio_path)
    row = {
        "politician":      politician,
        "historical_date": (datetime.strptime(historical_date, "%Y-%m-%d").date()
                            if historical_date else ""),
        "location":        location,
        "title":           title.replace("\n", " ").replace("\r", " ").strip(),
        "url":             url,
        "audio_file":      audio_path_str[audio_path_str.index("crossdem"):],
        "text":            text.replace("\n", " ").replace("\r", " ").strip(),
    }

    write_header = not os.path.exists(csv_output)
    with open(csv_output, "a", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=row.keys())
        if write_header:
            writer.writeheader()
        writer.writerow(row)
```

Code 3.13: Transcribing a speech and saving it with its metadata in a CSV file

3.3.2.2 Scraping YouTube

Almost all the corpus is composed of speeches scraped from Radio Radicale, so to diversify it a bit I decided to scrape some of Giorgia Meloni's speeches from YouTube. Fortunately, there is an unofficial YouTube channel (that I reached from the Prime Minister's official website) which aggregates more than four thousand videos of her public appearances. The channel is called "Giorgia Meloni News"[29]. In total, I manually selected and scraped seventy-two YouTube videos.

Given a YouTube URL, I use once again `yt-dlp` to retrieve the video's metadata and download its audio content in MP3 format, as shown in listing 3.14. I had to manually pass the cookies taken from

[29] Giorgia Meloni News: https://www.youtube.com/@GiorgiaMeloniTv

my web-browser, and I used some extra commands to prevent YouTube from blocking requests due to suspicious activity.

```
result = subprocess.run([
    "yt-dlp",
    "--print-json",
    "-x",                                   # download audio only
    "--audio-format", "mp3",
    "--cookies", "yt_cookies.txt",          # manually extracted cookies
    "--sleep-requests", "2",                # Sleep 2s between requests
    "--sleep-interval", "5",                # Sleep 5s between downloads
    "--max-sleep-interval", "15",           # Randomize up to 15s
    "--limit-rate", "5M",                   # Throttle to 5MB/s (mimics streaming)
    "-o", audio_file.replace('.mp3', ''),
    video_url
], capture_output=True, text=True, check=True)
```

Code 3.14: Downloading audio and metadata from a YouTube video

Then, the MP3 file just retrieved is passed to *Whisper* library for transcription, as shown in listing 3.15. I used the model size *"medium"* as before.

```
subprocess.run([
    "whisper",
    "--language",         lang_code,      # "Italian" chosen
    "--word_timestamps", "True",
    "--model",            model_name,     # model "medium" chosen
    "--output_dir",       output_dir,
    "--device",           "cuda",         # ensures the model gets loaded in GPU
    audio_file
], check=True)
```

Code 3.15: Transcribe the audio file into text using Whisper

The last step of the pipeline saves the transcription into a csv file, along with the extracted metadata. Each file has the following fields: `politician` ("meloni" in this case), `historical_date` (the upload date of the video), `location` and `tags` (extracted from the metadata if available, empty strings otherwise), `description` and `title` of the video, `url` which stores the permalink of the video itself, and lastly `text` which holds the whole transcription.

Each video is processed immediately after being retrieved, and until it has been saved in a csv file the program does not try to fetch the next one. This is done for two reasons: first, to prevent `yt-dlp` from sending requests too close to each other, which could result in YouTube denying them due to suspicious activity. Second, even if the pipeline halts for any reason, the videos processed up to that point will not be lost.

3.3.2.3 Scraping Sanity Check

Since the whole pipeline is fully automatic, it is necessary to verify the correctness of the resulting corpus. By "correct" I mean to say that the transcribed texts of a Prime Minister are in fact transcriptions of their speeches and not of someone else's (in section 3.3.2 we mentioned how the audio files often contain more than one speaker). This problem is commonly known as authorship attribution.

Of course, spot-checks were performed during scraping, but manually verifying every speech would defeat the purpose of automating the process in the first place. Among many possible methods, I decided to use an N-gram centroid-based one [117–119], since it is relatively straightforward and does not require a lot of resources.

The idea is as follows: if I embed all speeches into vectors, I can compute the centroid[30] of all speeches of a specific Prime Minister (their vectorial center). Then, for each speech, I can calculate the cosine similarity[31] between the speech embedding and the centroid. By doing so, I can compute a degree of similarity between the speeches of a Prime Minister in the vector space, assessing whether a speech adheres to their style or not (hence, whether the speech was authored by that specific Prime Minister). Each speech is graded with a degree of self-similarity (from zero to one), which represents how much that speech resembles the Prime Minister's corpus. The mean similarity for a Prime Minister corpus is calculated by averaging all self-similarity values. I flag the bottom 10% of speeches as "below threshold". Results are shown in table 3.3. De Gasperi was included as a baseline, since their corpus was not scraped.

We can see that overall the corpus quality is solid, with the vast majority of Prime Ministers' corpora above baseline. Some speeches in the corpus with very low self-similarity values (under 0.1) contain either a series of *"Ciao ciao ciao"* (likely background noise wrongly identified by Whisper), or a series of *"Sottotitoli a cura di QTSS"*. These speeches are so few compared to the whole corpus size that I decided to maintain the corpus as-is, without any arbitrary low-similarity cutoff. This was done to prevent the risk of manually introducing bias by cutting off "good" outliers, such as one-off topics or short speeches.

PM	Correct (%)	# Below Treshold / # Speeches	Mean Similarity
De Gasperi	86.7	63/473	0.67
Fanfani	95.2	1/21	0.70
Leone	83.3	1/6	0.63
Rumor	100.0	0/4	0.89
Andreotti	93.6	21/327	0.80
Cossiga	88.2	21/178	0.73
Forlani	92.3	5/65	0.75
Spadolini	91.8	8/97	0.75
Craxi	82.5	17/97	0.69
Goria	81.1	7/37	0.76
De Mita	96.2	6/157	0.83
Amato	96.0	29/722	0.82
Ciampi	90.4	7/73	0.73
Berlusconi	92.9	38/535	0.80
Dini	92.9	22/310	0.71
Prodi	92.6	38/515	0.80
D'Alema	96.1	32/829	0.84
Monti	97.9	5/240	0.83
Letta	96.3	19/509	0.80
Renzi	95.1	7/143	0.79

[30] In mathematics and physics, the centroid, also known as geometric center or center of figure, of a plane figure or solid figure is the mean position of all the points in the figure. The same definition extends to any object in n-dimensional Euclidean space [120].

[31] Cosine similarity is the cosine of the angle between two vectors, which in turn measures how similar the two vectors are.

Conte	80.0	6/30	0.61
Draghi	92.3	2/26	0.85
Meloni	92.5	15/199	0.73

Table 3.3: Per-PM classification results, ordered by their first term in office. In order: Prime Minister's name, percentage of speeches above threshold, number of speeches below the similarity threshold. Mean similarity represents how much the speeches resemble each other (lower values mean more variety in style). Colombo and Gentiloni were skipped (only one usable speech each, minimum three required). In total, 567 flagged speeches were exported to `outlier_speeches.csv`.

3.3.3 Corpus Annotation

Each speech of the corpus was annotated with an LLM run locally via Ollama[32] for its degree of hate speech, aggressiveness, and negativity (either *low*, *mid*, or *high*), as well as whether it targets a specific group or not (possible target groups are political adversaries, gender minorities, ethnic minorities, and religious minorities).

3.3.3.1 Model Choice

To choose the best model, I first compiled a validation set containing 207 speeches[33]. Since most Prime Ministers' speeches have very low hostility values[34], and almost none are targeting any minority group, a candidate model could perform very well by always predicting low hostility and no target for each speech. To mitigate this, I generated 100 highly hostile synthetic speeches with the frontier model Gemini 3.6 Flash via its online chat. I prompted the model to have at least "mid" values for hate speech, aggressiveness, and negativity, and to have each speech target one of the four possible targets. The full prompt can be seen in appendix A.23.
The remaining 107 speeches were taken from the dataset, annotated with Claude Sonnet 4 and Gemini 3.6 Flash via their online chat interfaces, and checked manually. The resulting validation set has 207 gold-labeled speeches.

I compared eight different models, and I chose the best one based on the Cohen's Kappa, meaning by their degree of agreement with the gold labels. I chose this metric because it is more robust than simple percent agreement calculation, as it incorporates the possibility of the agreement occurring by chance. Since the ordinal labels (hate speech, negativity, and aggressiveness) have strong differences between their levels (a speech with *"low"* hate speech is very different from a speech with *"mid"* or *"high"* levels of hate speech), I used the quadratic-weighted κ for them, which penalizes large disagreements disproportionately. For target, whose categories have no order nor distance, I kept the plain unweighted κ. The models with their respective Kappa values are shown in table 3.4.
Table 3.5 shows the best models for each metric (Cohen's kappa, accuracy, macro precision, macro recall,

[32] Ollama is an open-source platform for running and managing large language models on local GPU infrastructure. It provides a command-line interface, model-management tools, and integrations for using open-weight models with coding assistants and other applications. Website: https://ollama.com/.
[33] Validation set is found at: *crossdem/datasets/prime_ministers_validation*
[34] I previously defined the term "hostility" as the compound metric of hate speech, aggressiveness, and negativity.

and macro F1)[35]. We can see that `gemma4:e4b` performs the best in most situations, and even when it does not (table 3.6) it is outperformed by models of similar size by around 5% (on five metrics out of twenty), and by bigger models by around 13% (on two metrics out of twenty). To see all values of all metrics for all models refer to the table in appendix A.11.
It is interesting to note how Qwen 2.5, despite being almost two years older than both Qwen 3.5 and Gemma 4 (released respectively in February and in April 2026), outperforms them both for *negativity* in three metrics: Cohen's Kappa, accuracy and macro F1.

Four models fully fit in the VRAM of my GPU: `qwen2.5:7b-instruct-q4_K_M`, `mistral:7b-instruct-v0.3-q4_K_M`, `qwen3.5:4b`, and `gemma4:e4b`. The other models get split between GPU and CPU, making them unsuitable for this project since it slows down token generation considerably: with 6,038 speeches to classify I chose to use only models that can fully run in GPU.

Let's now briefly talk about *thinking models*. DeepSeek-AI in 2025 [121] trained the LLM DeepSeek-R1 through reinforcement learning to generate an explicit chain of thought before committing to a final answer. This way the model simulates a reasoning process, ultimately achieving better outputs.

The only model I could benchmark with the thinking functionality enabled is `gemma4:e4b`. Mistral 0.3 and Qwen 2.5 do not have the functionality in the first place, and models too big to fit in the VRAM would have been slowed down even further. `qwen3.5:4b`, on the other hand, displayed a faulty behaviour: with "thinking" enabled the model exhausted its entire token budget, never converging to an answer.

Ultimately, due to its recency and its great performance both in terms of classification quality and generation speed, I decided to use the model `gemma4:e4b` with thinking disabled to classify *negativity* and *aggressiveness*, and with thinking enabled to classify *hate speech* and *target*.

Model	Thinking	Hate speech	Negativity	Aggressiveness	Target
`qwen2.5:7b-instruct-q4_K_M`	-	0.173	**0.297**	0.255	0.705
`mistral:7b-instruct-v0.3-q4_K_M`	-	0.128	0.061	0.051	0.699
`qwen3.5:4b-q8_0`	off	0.091	0.222	0.194	0.783
`qwen3.5:4b`	off	0.268	0.208	0.363	0.760
`qwen3.5:9b-q4_K_M`	off	0.037	0.246	−0.009	0.721
`gemma4:e4b`	off	0.168	**0.293**	**0.476**	0.674
`gemma4:12b`	off	0.228	0.230	0.303	0.722
`gemma4:e4b`	on	**0.538**	0.139	0.246	**0.869**

Table 3.4: Agreement between candidate models' predictions and gold labels on the 207 validation speeches, measured by Cohen's κ. Quadratic-weighted κ is used for the ordinal labels (hate speech, negativity, aggressiveness) and unweighted κ for target. Values in bold mark the best results per label. Models are identified by their Ollama tags. Thinking mode does not apply to Qwen 2.5 and Mistral (-).

[35] Macro-averaged metrics are computed by calculating the metric separately for each class and then taking the unweighted mean across classes, so that every class contributes equally regardless of its support; this is preferable to micro or weighted averages when the classes are imbalanced, as in the crossdem validation set, where majority classes (e.g., *low hate speech*) would otherwise dominate the score and mask poor performance on minority classes (e.g., *high hate speech*).

Label	κ	Accuracy	Macro Precision	Macro Recall	Macro F1
Hate speech	0.538 gemma4:e4b (on)	0.826 gemma4:e4b (on)	0.819 gemma4:e4b (on)	0.777 gemma4:e4b (on)	0.773 gemma4:e4b (on)
Negativity	0.297 qwen2.5:7b-instruct-q4_K_M	0.570 qwen2.5:7b-instruct-q4_K_M	0.620 qwen3.5:4b-q8_0	0.625 gemma4:12b	0.566 qwen2.5:7b-instruct-q4_K_M
Aggressiveness	0.476 gemma4:e4b (off)	0.691 qwen3.5:4b, gemma4:e4b (off)	0.736 qwen3.5:4b	0.736 qwen3.5:4b	0.690 gemma4:e4b (off)
Target	0.869 gemma4:e4b (on)	0.903 gemma4:e4b (on)	0.942 gemma4:e4b (on)	0.927 gemma4:e4b (on)	0.930 gemma4:e4b (on)

Table 3.5: Best result per label and metric across all models and thinking settings on the 207 validation speeches. Each cell reports the best value and the model that achieved it. If a model was tested with thinking enabled and with thinking disabled, the setting is given in brackets. Cohen's κ is quadratic-weighted for hate speech, negativity and aggressiveness, and unweighted for target.

Label	Metric	gemma4:e4b	Best Result	Difference	Best Model	Does It Fit?
Negativity	κ	0.293	0.297	1.37%	qwen2.5:7b-instruct-q4_K_M	Yes
Negativity	Accuracy	0.536	0.570	6.34%	qwen2.5:7b-instruct-q4_K_M	Yes
Negativity	Macro precision	0.534	0.620	16.10%	qwen3.5:4b-q8_0	No
Negativity	Macro recall	0.564	0.625	10.82%	gemma4:12b	No
Negativity	Macro F1	0.532	0.566	6.39%	qwen2.5:7b-instruct-q4_K_M	Yes
Aggressiveness	Macro precision	0.701	0.736	4.99%	qwen3.5:4b	Yes
Aggressiveness	Macro recall	0.726	0.736	1.38%	qwen3.5:4b	Yes

Table 3.6: Model gemma4:e4b compared on all the metrics where it does not perform the best. The difference is around around 5% with models that fit in VRAM, and around 13% with models too big to fit. On every single other metric, gemma4:e4b beats all other candidate models.

3.3.3.2 Annotation Pipeline

To run the pipeline and annotate the whole corpus, simply run the script `~/crossdem/source/classifier.py`. Make sure to have the dependencies listed in `~/crossdem/requirements.txt`, and the Ollama model `gemma4:e4b` (link: https://ollama.com/library/gemma4:e4b).

Each speech is labelled on four dimensions: hate speech, negativity, aggressiveness, and target. The first three can be either *low*, *mid* or *high*, while the target can be either *none*, *pol_adv* (political adversaries), *minor_etn* (ethnic minorities), *minor_gnd* (gender minorities), or *minor_rel* (religious minorities).

The four dimensions are split between two groups, each with its own system prompt (full prompts can be seen in appendix A.24): one prompt for hate speech and target (fed to the model with thinking enabled), and one prompt for negativity and aggressiveness (fed to the model with thinking disabled). The prompts are in Italian, each containing the definition of the two dimensions they consider, as well as three examples for each level of the dimension: hate speech, negativity, and aggressiveness each have three examples for *low*, three examples for *mid*, and three examples for *high*, and target has three examples for each target group. The model is instructed to answer only with a JSON object with the provided formatting. The speech is passed as the user message.

The function `classify_one` (3.17) feeds a speech to the model and returns the labels. The output is constrained by a JSON schema (3.16) that restricts each field to its allowed values. The temperature is kept to zero, and the context window is 35,000 tokens, which is big enough to contain both the longest speech in the corpus as well as the tokens needed by the *"thinking"* phase at the same time. I chose this size after some manual testing: the default context window of 4,096 tokens requires 4,262 MiB of VRAM, and the model do not converge to an answer since it runs out of tokens during the thinking phase. The prompt is 1,449 tokens long, and the longest speech in the corpus is 20,196 tokens long (tokenization done with Spacy). A context window that includes both, plus a 20% buffer, would need to be 24,226 tokens wide. With a context window size of 27,000 tokens, the model requires 4,556 MiB of VRAM and does not converge to an answer, so I increased the context window to 30,000 tokens, which requires 4,596 MiB of VRAM and affords enough room for the model to finally produce an output. Since the increase in VRAM is negligible, I decided to use a context window of 35,000 tokens, which requires 4,662 MiB of VRAM and, of course, affords the model enough room to converge to an answer.

The number of generated tokens is unlimited. The response is parsed as JSON and its values are checked against the allowed ones. If the call or the check fails, the Ollama call is retried up to three times. The wrappers `classify_think` and `classify_nothink` call the classification function with the appropriate parameters.

```python
LEVEL_VALUES = ["low", "mid", "high"]
TARGET_VALUES = ["none", "pol_adv", "minor_etn", "minor_gnd", "minor_rel"]

THINK_COLS = ["hate_speech", "target"]            # think enabled
NOTHINK_COLS = ["negativity", "aggressiveness"]   # think disabled
LABEL_COLS = ["hate_speech", "negativity", "aggressiveness", "target"]

OLLAMA_MODEL = "gemma4:e4b"
RETRIES = 3
RETRY_SLEEP_S = 2

client = Client()

JSON_SCHEMA_THINK = {
    "type": "object",
    "properties": {
        "hate_speech": {"type": "string", "enum": LEVEL_VALUES},
        "target": {"type": "string", "enum": TARGET_VALUES},
    },
    "required": THINK_COLS,
}
# JSON_SCHEMA_NOTHINK is the same, with negativity and aggressiveness
```

Code 3.16: Label values, model settings and output schema

```python
def classify_one(text, model, system_prompt, schema, valid_cols, think):
    for attempt in range(1, RETRIES + 1):
        try:
            resp = client.chat(
                model=model,
                messages=[
                    {"role": "system", "content": system_prompt},
                    {"role": "user", "content": text},
                ],
                format=schema,  # forces the response to a JSON schema
                think=think,    # true if enabled
                options={"temperature": 0, "num_ctx": 35000,
                         "num_predict": -1},
            )
            parsed = json.loads(resp["message"]["content"])
            for col, allowed in valid_cols.items():
                assert parsed[col] in allowed
            return parsed
        except Exception as e:
            print(f"attempt {attempt}/{RETRIES} failed: {e}")
            time.sleep(RETRY_SLEEP_S)
    return None

def classify_think(text):
    return classify_one(
        text, OLLAMA_MODEL, SYSTEM_PROMPT_THINK, JSON_SCHEMA_THINK,
        {"hate_speech": LEVEL_VALUES, "target": TARGET_VALUES},
        think=True,
    )
```

Code 3.17: Classifying a single speech.

The classification is run in two passes over the whole corpus, one per prompt, by `run_pass` (3.18) which processes the CSV files of the speeches of each Prime Minister, read with `csv.DictReader`. For each row (all files have only one row), the function `needs_fn` checks whether any label is missing. If so, and if the row has text, the speech is classified and the row is updated with the labels. The file is rewritten only after all its rows have been processed, and only if at least one row has changed.

Each pass uses a single reasoning mode for the whole corpus, instead of switching modes once per speech[36]. A speech with all four labels is skipped by both passes, and a speech for which all the retries

[36] This is not necessary if you don't switch models every time (e.g., between `gemma4:e4b` and `qwen3.5:4b`), which I originally intended to do. Model switching has a lot of overhead, requiring (on my machine) around twenty seconds.

fail is left unlabelled. The script can therefore be interrupted and run again, and it only fills in what is missing.

In the end, all speeches were classified one the first try.

```python
def _cols_missing(row, cols):
    return any(str(row.get(c) or "").strip() == "" for c in cols)

def needs_think_classification(row):
    return _cols_missing(row, THINK_COLS)

def needs_nothink_classification(row):
    return _cols_missing(row, NOTHINK_COLS)

def run_pass(files, needs_fn, classify_fn):
    for path in files:
        rows, fieldnames = read_speech_csv(path)
        file_changed = False

        for row in rows:
            if not needs_fn(row):
                continue
            text = (row.get("text") or "").strip()
            if not text:
                continue
            result = classify_fn(text)
            if result is not None:
                row.update(result)
                file_changed = True

        if file_changed:
            write_speech_csv(path, rows, fieldnames)

def main():
    files = find_speech_files()
    run_pass(files, needs_think_classification, classify_think)
    run_pass(files, needs_nothink_classification, classify_nothink)
```

Code 3.18: Labelling the corpus in two passes.

3.3.4 Varieties of Democracy

This section will briefly present the Varieties of Democracy Dataset.

3.3.4.1 Democracy Index

There are plenty of sources for measuring democracy levels. Our World in Data mainly uses six: Varieties of Democracy (V-Dem), the Lexical Index of Electoral Democracy (LIED) by Skaaning et al. (2015), Freedom House's (FH) Freedom in the World index, the Bertelsmann Transformation Index (BTI) by the Bertelsmann Foundation, the Economist Intelligence Unit's (EIU) Democracy Index, and Polity by the Center for Systemic Peace [122].

Out of the six mentioned datasets, only V-Dem and LIED have data spanning between 1946 and 2025. However, V-Dem offers more granularity and is generally considered a more complete and robust dataset. It is the best choice if we are interested in both large and small differences in varieties of democracy far into the past, or if we want to use country experts to measure characteristics of political systems that are difficult to observe. These experts are anonymous and are primarily academics or members of the media and civil society. They are also often nationals or residents of the country they assess; therefore, they know its political system well and can evaluate aspects that are difficult to observe. V-Dem's own team of researchers supplements these expert evaluations [123].

3.3.4.2 V-Dem Dataset

All data is available for download at https://v-dem.net/data/the-v-dem-dataset/ or can be accessed directly as an R package.

V-Dem distinguishes between *indicators*—lower-level data that is often not normalized—and *indices*, which aggregate indicators and are normalized on a scale between zero and one.

There are four datasets: *Coder-Level*, with raw data and 273 indicators intended for the experts; *CD Country-Date*, with day-to-day granularity, 531 indicators, and 95 aggregated indices; *CY Country-Year*, with year-level granularity, 5 indices, 93 sub-indices, and 179 indicators; and lastly, *CY Full + Others*, with year-level granularity, 531 indicators, 251 indices, and 62 external indicators.

I must use the *CD Country-Date* dataset (version 16, released in March 2026) since, to keep track of Italian governments, year-level granularity is not enough. This is evident from 3.4: half of Italy's governments lasted less than a year.

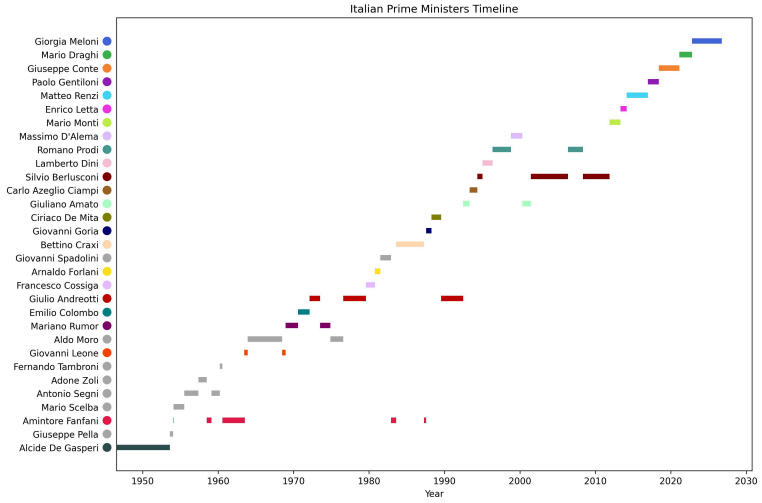


Figure 3.4: Italian Prime Ministers for each Italian Republic, from the first (1946) to the latest (2025).

The dataset covers Italy (ID 82) between 1861 and 2025, France (ID 76) between 1789 and 2025, and the U.S. (ID 20) between 1789 and 2025.

Useful identifiers include: `country_id`, unique for each country; `country_text_id`, ISO 3166-1 alpha-2 country codes (e.g., *IT*, *FR*); `historical_date`, in *YYYY-MM-DD* format, used for election-date-specific variables (all other variables use January 1st); and `gapstart` and `gapend`, representing time periods where no data is available.

Democracy Indices are the highest-level indices, each normalized to a scale between zero and one unless otherwise specified: `v2x_polyarchy`, the *electoral* democracy index (aggregates `v2x_elecoff`, `v2xel_frefair`, `v2x_frassoc_thick`, `v2x_suffr`, `v2x_freexp_altinf`); `v2x_libdem`, the *liberal* democracy index (aggregates `v2x_polyarchy`, `v2x_liberal`); `v2x_partipdem`, the *participatory* democracy index (aggregates `v2x_polyarchy`, `v2x_partip`); `v2x_delibdem`, the *deliberative* democracy index focusing on process, respectful dialogue, and the common good (aggregates `v2x_polyarchy`, `v2xdl_delib`); and `v2x_egaldem`, the *egalitarian* democracy index considering social status, inequalities, and resource distribution (aggregates `v2x_polyarchy`, `v2x_egal`).

Components of Democracy Indices consist of second-level indices and subcomponents: `v2x_freexp_altinf`, freedom of expression and alternative information (aggregates *v2mecenefm*,

v2meharjrn, *v2meslfcen*, *v2xcl_disc*, *v2clacfree*, *v2mebias*, *v2mecrit*, *v2merange*); `v2x_frassoc_thick`, freedom of association for parties and civil society organizations (aggregates *v2psparban*, *v2psbars*, *v2psoppaut*, *v2elmulpar*, *v2cseeorgs*, *v2csreprss*, `v2x_elecreg`); `v2x_suffr`, the percentage of adult suffrage (aggregates *v2elsuffrage*); `v2xel_frefair`, the quality of free and fair elections (aggregates *v2elembaut*, *v2elembcap*, *v2elrgstry*, *v2elvotbuy*, *v2elirreg*, *v2elintim*, *v2elpeace*, *v2elfrfair*, `v2x_elecreg`); `v2xcl_rol`, rule of law, transparency, and fair enforcement (aggregates *v2clrspct*, *v2cltrnslw*, *v2cltort*, *v2clkill*, *v2clrelig*, *v2clfmove*, `v2xcl_dmove`, `v2xcl_slave`, `v2xcl_acjst`, `v2xcl_prpty`); `v2x_jucon`, executive law compliance and judiciary independence (aggregates *v2exrescon*, *v2jucomp*, *v2juhccomp*, *v2juhcind*, *v2juncind*); `v2xlg_legcon`, executive external scrutiny (aggregates *v2lgqstexp*, *v2lgotovst*, *v2lginvstp*, *v2lgoppart*); `v2x_partip`, individual participation (aggregates `v2x_cspart`, `v2xdd_dd`, `v2xel_locelec`, `v2xel_regelec`); `v2x_cspart`, participation in associations like labor unions or NGOs (aggregates *v2pscnslnl*, *v2cscnsult*, *v2csprtcpt*, *v2csgender*); `v2xdd_dd`, the direct vote index for referendums and ballots (aggregates *v2ddlexci* through *v2ddthreci*); `v2xel_regelec` and `v2xel_locelec`, regional and local government elections; `v2xeg_eqprotec`, equal rights across social groups; `v2xeg_eqaccess`, equality of accessing power; and `v2xeg_eqdr`, equality of resource distribution (aggregates *v2dlencmps*, *v2dlunivl*, *v2peedueq*, *v2pehealth*).

4 Results

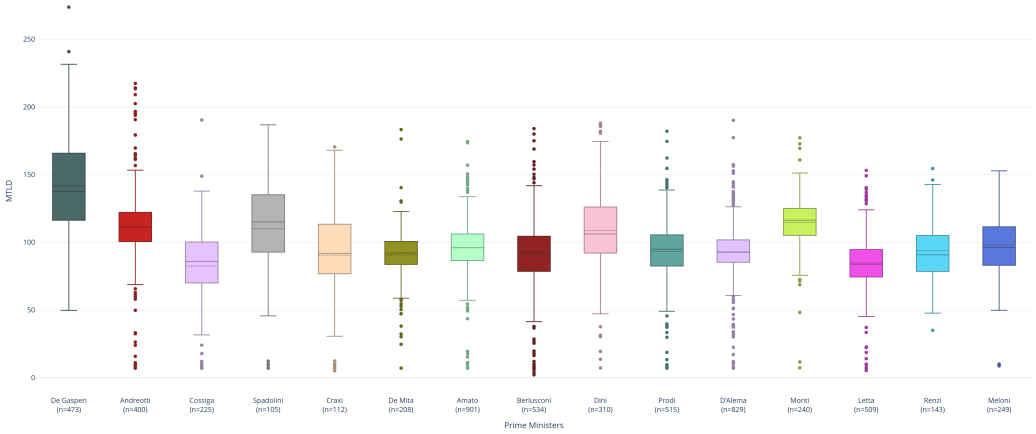
4.1 Textual Complexity

4.1.1 Measure of Textual Lexical Diversity

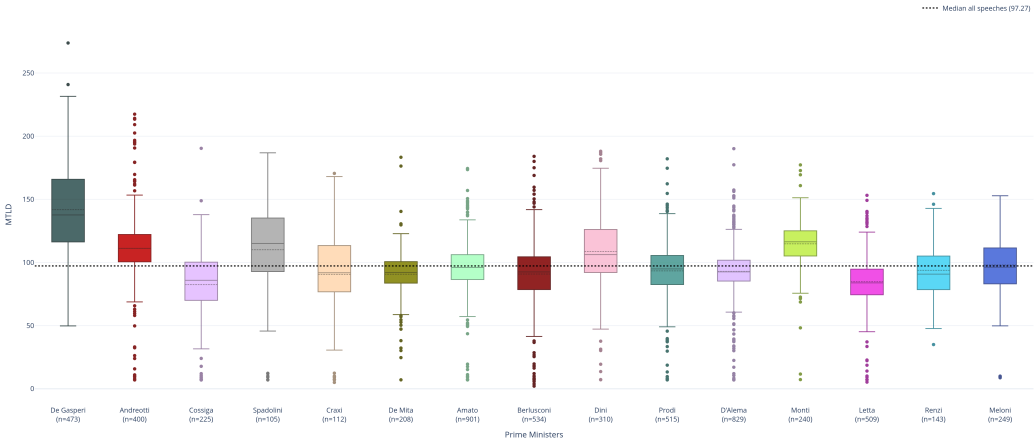
Measure of Textual Lexical Diversity (MTLD) is a metric used to assess the diversity of vocabulary in a text. It calculates the average length of sequences of words that maintain a certain level of lexical diversity, providing insights into the richness and variety of language used in the text. yada yada

4.1.2 Prime Minister's MTLDs

4.1.2.1 Boxplot of MTLD Values for Each Italian Prime Minster



(a) Boxplot of MTLD values for each Italian Prime Minster, without overall median.

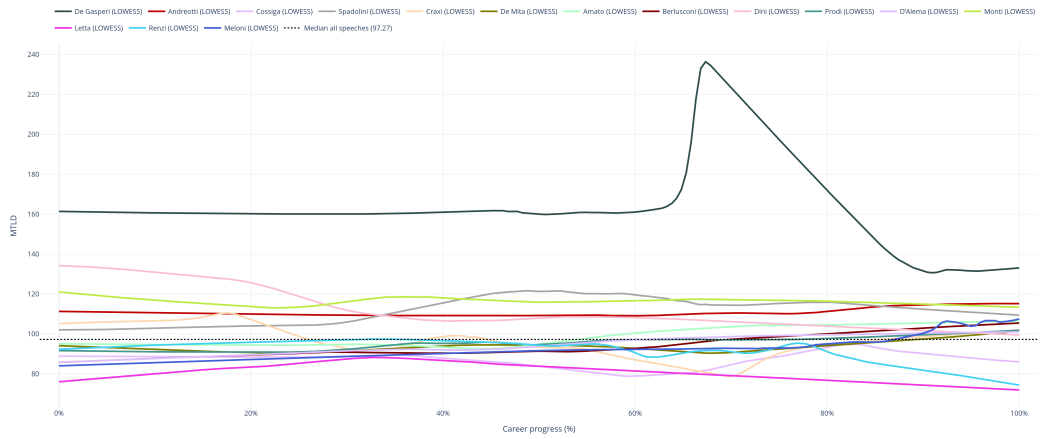


(b) Boxplot of MTLD values for each Italian Prime Minster, with overall median.

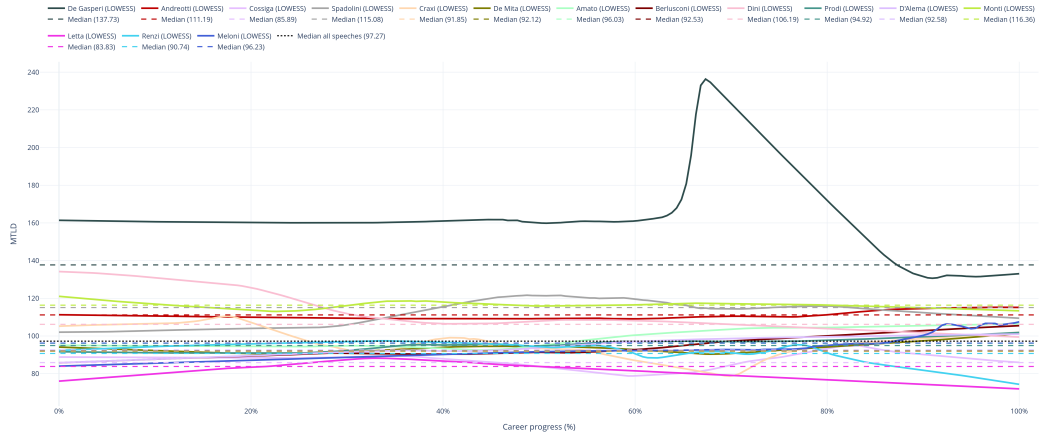
Figure 5: 5a and 5b show MTLD values for each Italian Prime Minster, ordered by their first time in office. This allows for not only a direct comparison between Prime Ministers, but also a comparison of the evolution of MTLD values over time. Prime Minister with fewer than 100 speeches are omitted.

4.1.2.2 Evolution Over Time of MTLD Values for Each Italian Prime Minster

- what is LOWESS



(a) LOWESS curve of MTLD values for each Italian Prime Minster, without overall medians.

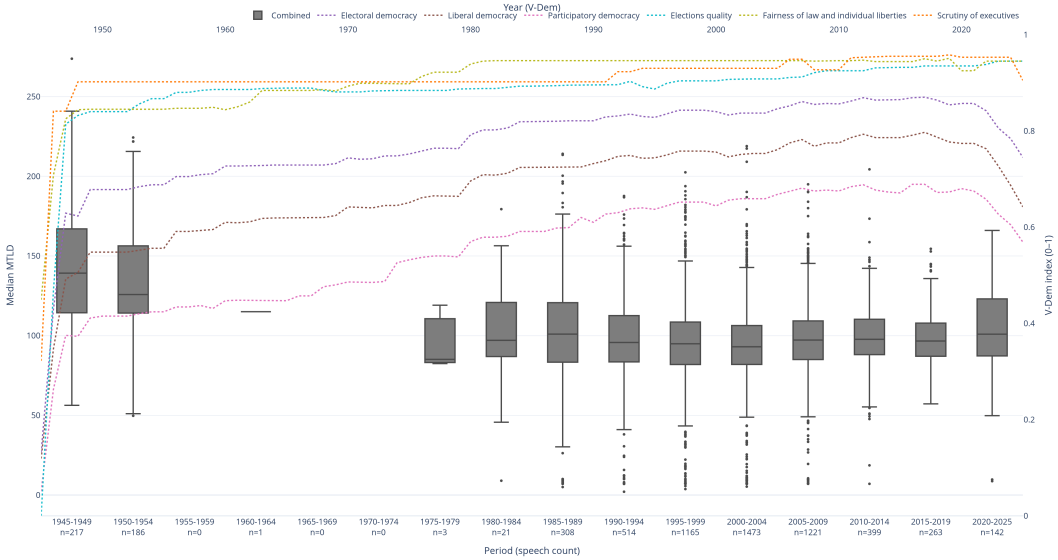


(b) LOWESS curve of MTLD values for each Italian Prime Minster, with overall medians.

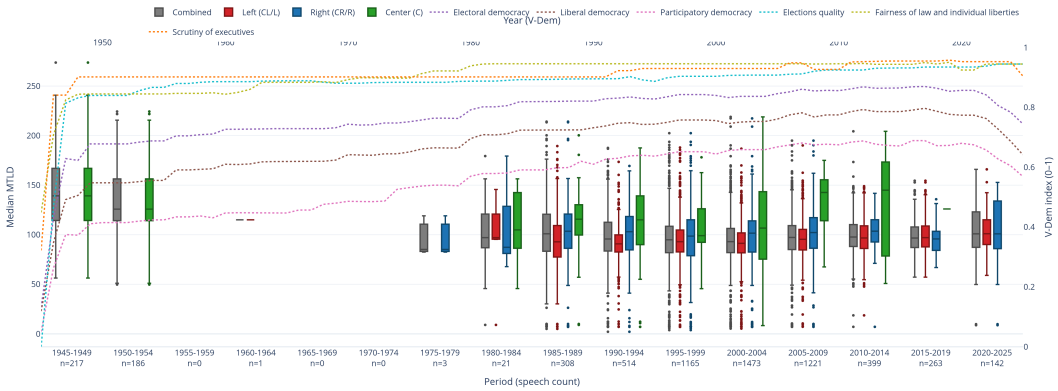
Figure 6: 6a and 6b show MTLD values for each Italian Prime Minster over their careers span. The LOWESS curve allows was chosen to aid in the interpretation of the values, and the overall medians allow for a comparison between Prime Ministers. Prime Minister with fewer than 100 speeches are omitted.

4.1.3 VDEM

4.1.3.1 5 YEARS



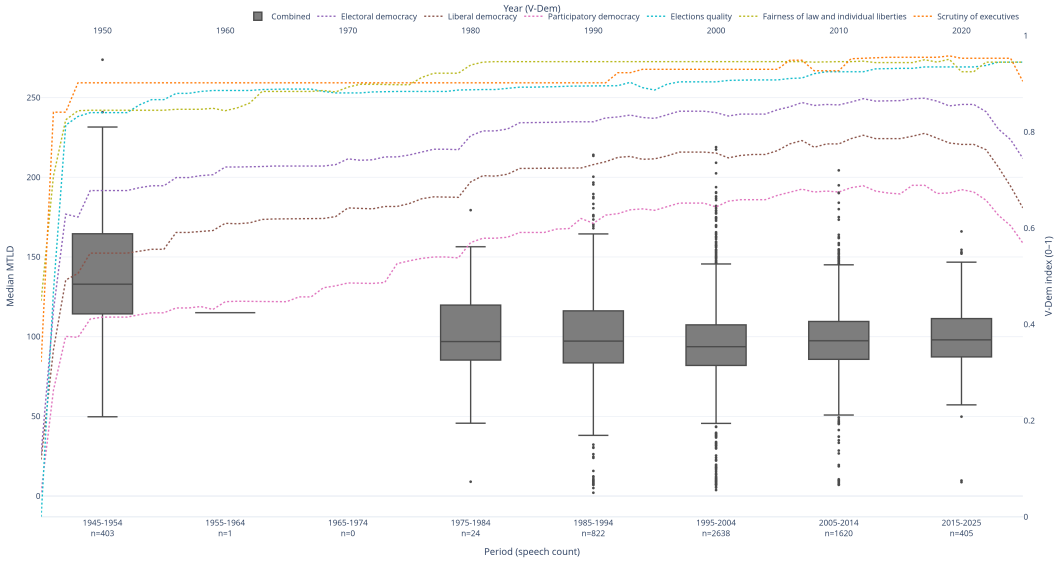
(a) MTLD values in a Boxplot by 5-year periods, with VDEM indices. Each grey box combines all speeches for all political leanings in that 5-year period.



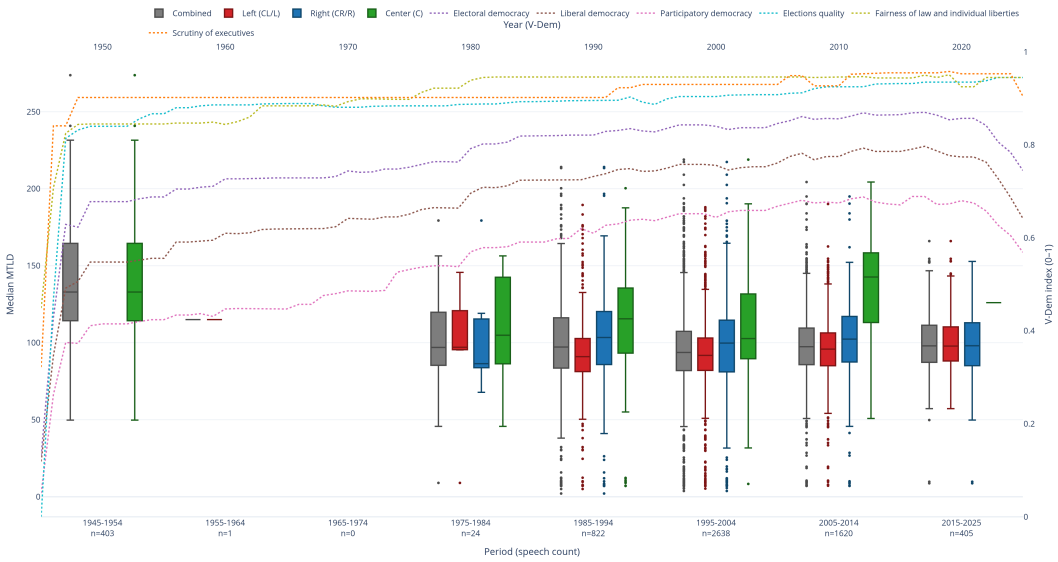
(b) MTLD values in a Boxplot by 5-year periods, with VDEM indices. As above, each grey box combines all speeches for all political leanings in that 5-year period, while the colored boxes represent respectively center-leaning (in green), left-leaning (in red), and right-leaning (in blue) MTLD values in that 5-year preiod.

Figure 7: MTLD values in a Boxplot by 5-year periods. There seem to be a reduction in MTLD variance starting from 1989 all the way to 2019, whith an abrupt inversion in the 2020-2025 period. If this is a correlation with the inflection of some VDEM indices, it is not clear.

4.1.3.2 10 YEARS



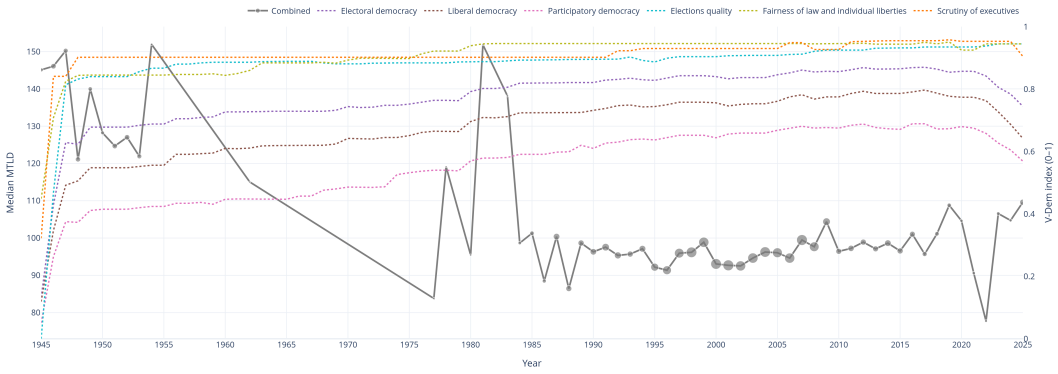
(a) MTLD values in a Boxplot by 10-year periods, with VDEM indices. Each grey box combines all speeches for all political leanings in that 10-year period.



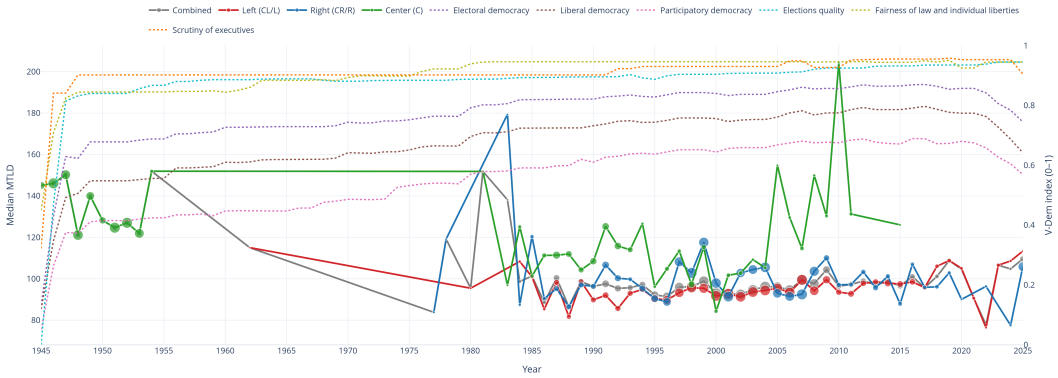
(b) MTLD values in a Boxplot by 10-year periods, with VDEM indices. As above, each grey box combines all speeches for all political leanings in that 10-year period, while the colored boxes represent respectively center-leaning (in green), left-leaning (in red), and right-leaning (in blue) MTLD values in that 10-year preiod.

Figure 8: MTLD values in a Boxplot by 10-year periods. There seem to be a reduction in MTLD variance starting from 1989 all the way to 2025. If this is a correlation with the inflection of some VDEM indices, it is not clear.

4.1.3.3 All



(a) MTLD values plotted for each year, aggregating all speeches from all political leanings. Each dot of the graph (for each year) is scaled based on the amount of speeches available for that specific year. The MTLD line chart is plotted against VDEM indices.



(b) MTLD values plotted for each year, aggregating speeches from three political leanings, respectively colored as green (center), red (left), and blue (right). Each dot of the graph (for each year) is scaled based on the amount of speeches available for that specific year. The MTLD line chart is plotted against VDEM indices.

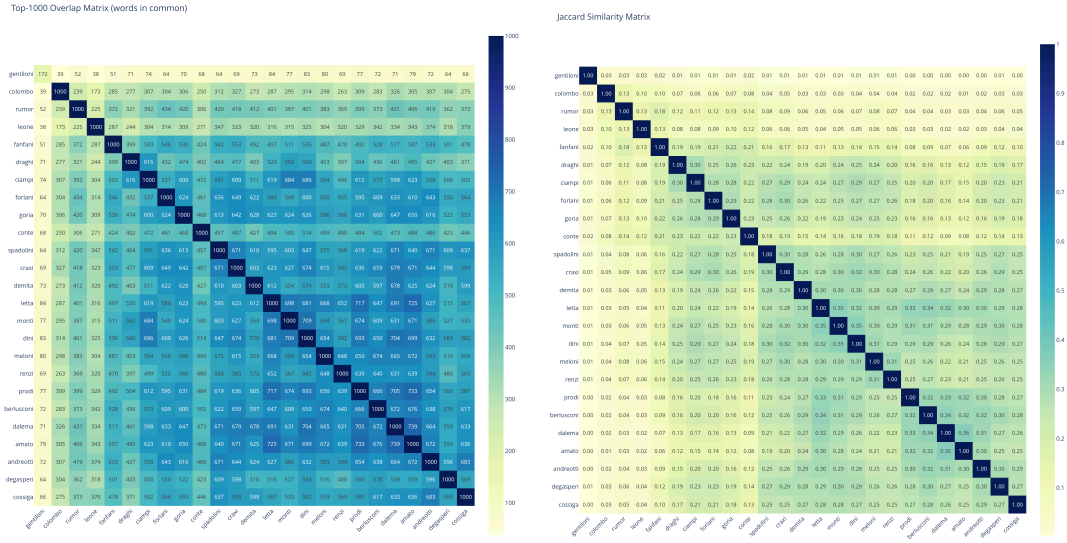
Figure 9: MTLD values aggregated for each year, plotted against VDEM indices. There does not seem to be a correlation between a decrease in complexity and a decrease in democratic indices. Instead, the graph seems to suggest the opposite.

4.1.4 Conclusions

There does not seem to be a clear correlation between MTLD values and democratic indices, with the exception for the variance which ...

4.2 Polarization

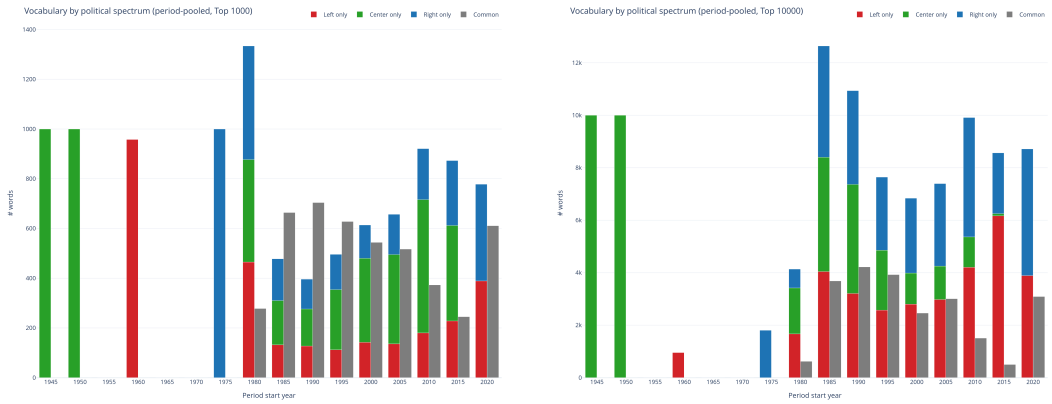
4.2.1 Prime Minister's Lexical Similarity



(a) Words similarity. (b) Jaccard similarity.

Figure 10: Heatmaps showing lexical similarity between Prime Ministers' speeches. 10a was made by pooling the 1000 most used words by each Prime Minister, then calculating how many words are shared between each of them. 10b, on the other hand, is made by calculating the Jaccard index between each pair of Prime Ministers, by pooling *all* words from all speeches used by each Prime Minister.

4.2.2 Political Leanings Lexicons Over Time



(a) Top 1,000 pooling.

(b) Top 10,000 pooling

Figure 11: All speeches are pooled in 5-year time frames. Then, for each 5-year time frame, the top-N most frequently used words are extracted for each political leaning[37] (one thousands words for graph at 11a, and ten thousands words for graph at 11b). This results in three distinct sets[38] of words for left, right, and center leaning speeches respectively.

The graph shows, for each 5-year time frame, words in common among all leanings (in grey), and words unique for each leaning (in red, blue, and green respectively). For the years with most data (from 1985 to 2015), 11a shows a progressive increase in unique vocabularies compared to a decrease in shared ones, which can be interpreted as symptoms of increasing polarization.

[37] A speech is considered "right-leaning" if it pertains a right-leaning Prime Minister.

[38] When speeches for each leaning are present. Otherwise only two, one, or no set at all can be extracted from that 5-year time frame.

4.2.3 t-SNE

4.2.3.1 Word2Vec: Word Embeddings

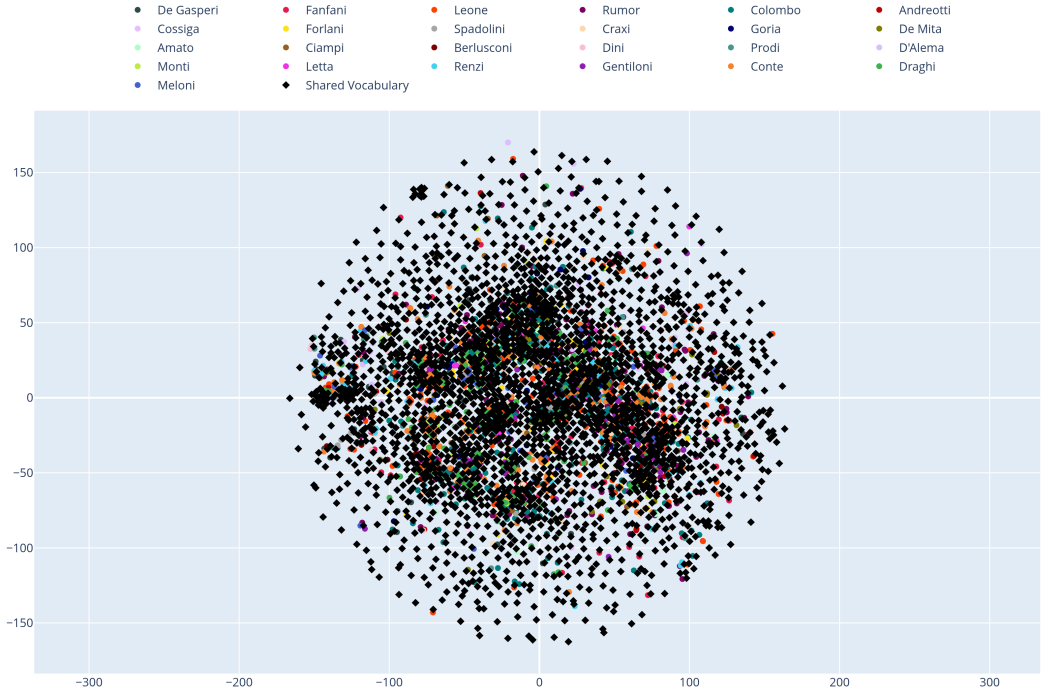
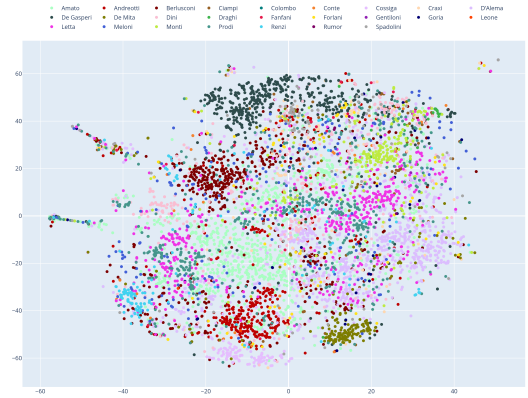


Figure 4.12: Top-1000 words[39] from each Prime Minster. The embedding vectors are pretrained Italian word vectors (CBOW, 300 dimensions) taken from *fastText*. Words shared by all Prime Ministers are displayed as black rhombuses. The graph show a lot of clustering, suggesting a certain similarity among Prime Ministers. That being said, the graph is also too hard to read to gain any conclusive evidence.

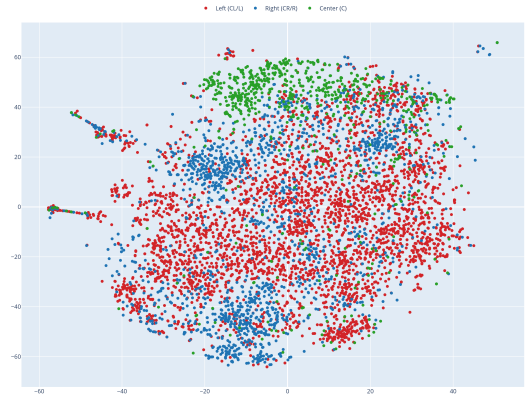
,

[39] Top-1000 words means pooling the one thousand most used words of a specific Prime Minister. Stopwords are removed and the lexicon is lemmatized.

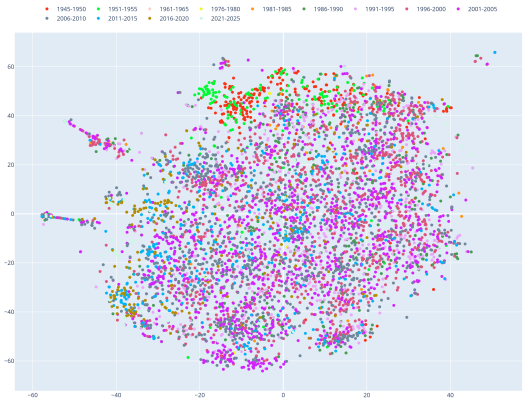
4.2.3.2 Doc2Vec: Speech Embeddings



(a) Doc2Vec: Prime Ministers



(b) Doc2Vec: political leaning



(c) Doc2Vec: year periods

Figure 13: Using `TfidfVectorizer` from `sklearn` library, I computed one embedding for each speech[40]. The embeddings can then be projected in two dimensions. 13a shows the projected speech vectors grouped by Prime Minsters. We can see clustering, suggesting that most prime ministers have a lexical style and wording that are unique to them. 13b groups the vectors by political leaning, showing that different leanings occupy different and distinct regions in the vector space. Both graphs strongly hints to a polarized political landscape.

On the other hand, 13c shows two main clustering regions: on the top, speeches from 1945 to 1955 gather, while speeches from 1991 to 2025 are more uniformly scattered across the whole vector space. This suggests that earlier speeches were significantly different compared to modern ones, while speeches from the nineties onward are quite similar.

TODO: compare with VDEM polarization indices (if exists).

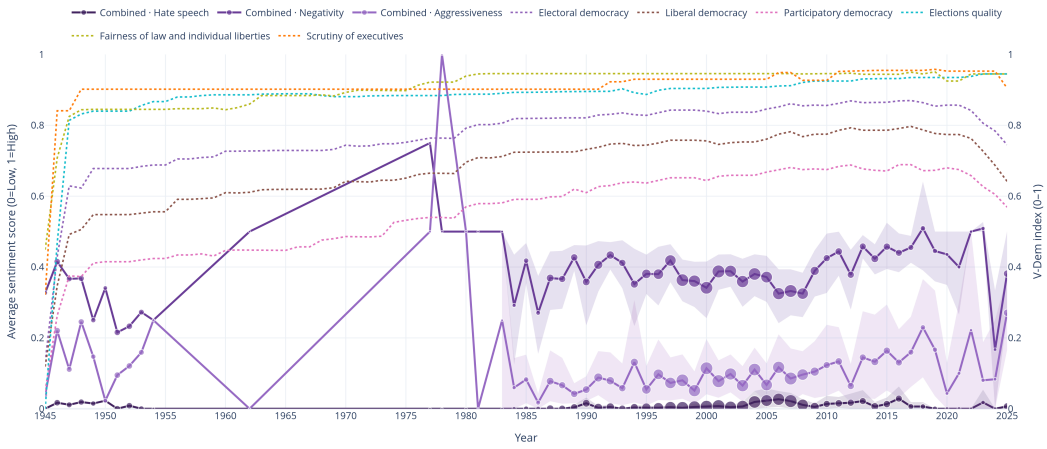
4.2.4 Conclusions

We see quite a bit of polarization, but ...

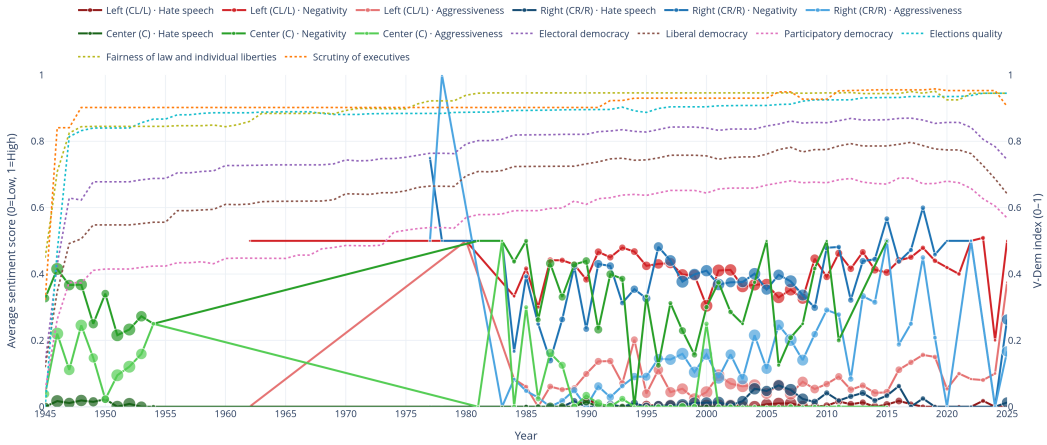
[40] Each speech counts as a document.

4.3 Sentiment Analysis

4.3.1 Democratic Indices Against Sentiment Scoring



(a) Sentiment values plotted for each year, aggregating all speeches from all political leanings. Each dot of the graph (for each year) is scaled based on the amount of speeches available for that specific year. The confidence intervals are shown. The sentiment scores chart is plotted against VDEM indices.

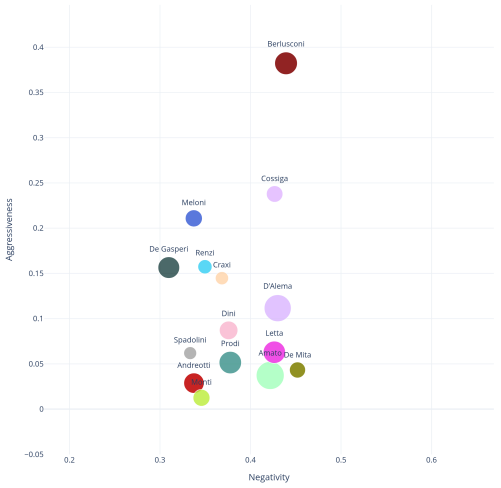


(b) Sentiment values plotted for each year, aggregating speeches from three political leanings, respectively colored as green (center), red (left), and blue (right). Each dot of the graph (for each year) is scaled based on the amount of speeches available for that specific year. The The sentiment scores line chart is plotted against VDEM indices.

Figure 14: Sentiment analysis results for *hate speech*, *aggressiveness*, and *negativity* plotted against VDEM indices. Its hart to tell wether there is any correlation between them.

4.3.2 Toxicity (TODO: change name)

First thing first: hate speech is so low for everyone that we can ignore it



(a) Sentiment: Prime Ministers ($|\text{speeches}| \geq 100$).

(b) Sentiment: leanings.

(c) Sentiment: years zoomed out.

(d) Sentiment: years zoomed in.

Figure 15: Sentiment analysis results for *negativity* and *aggressiveness*. Values go from zero to one, and the classification task considers three possibile values for each speech: *low* (0), *mid* (0.5), and *high* (1).

Size of bubbles depends on the amount of speeches. 15a project results for single Prime Ministers: Berlusconi stands out as the most aggressive by far. 15b project results aggregated by leanings. The right appears to be more aggressive, while the left more negative. The center seems to be more positive than both. The last two figures show the sentiment scores aggregated by years (with 5-year pool range). While 15c shows an outlier for the period 1976 — 1980, 15d shows that overall the sentiment scores are quite close to each other, without great variance especially for periods with a lot of speeches.

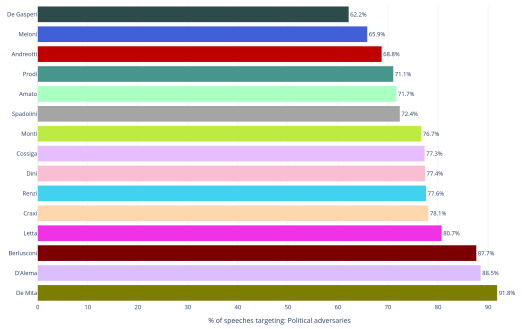
4.3.3 Targets

Five possible targets:

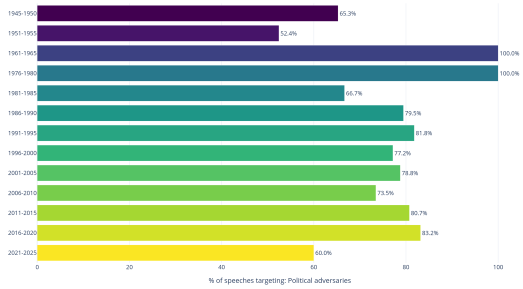
- None;
- Political adversaries;
- Gender minorities;

- Religious minorities;
- Ethnic minorities.

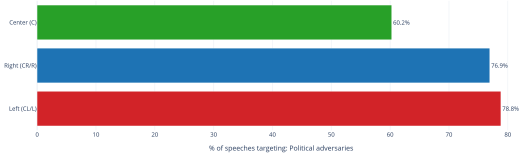
The targets are classified independently to the sentiment scores. For example, a speech could have *low* aggressiveness and still be targeting political adversaries.



(a) Target: political adversaries. Grouped by: PMs.



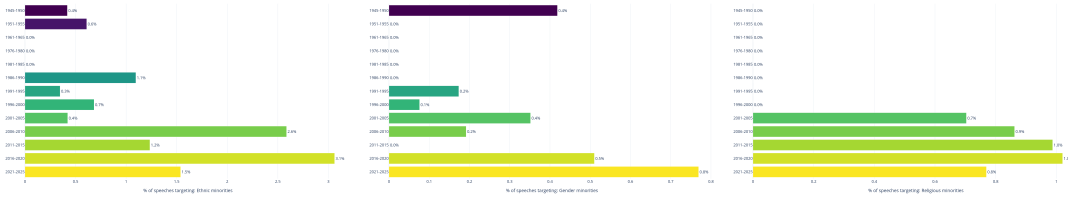
(b) Target: political adversaries. Grouped by: years.



(c) Target: political adversaries. Grouped by: leanings.

Figure 16: All graphs show the percentage of speeches that target political adversaries. 16a shows which Prime Minister speaks more often about the opposition (prime ministers with less than 100 speeches are omitted). 16b shows in which 5-year periods there are more speeches targeting political adversaries (ordered by year). Lastly, 16c shows which of the three political fields is more likely to speak about the opposing front. The results show how, across the board, more than one speech out of two targets the opposition. This is true regardless of time period or affiliation.

Thankfully, Italian Prime Minister apparently do not target minorities often.



(a) Target: ethnic minorities. (b) Target: gender minorities. (c) Target: religious minorities.

Figure 17: Overall, an extremely low percentage of speeches target minority groups. Therefore, I will ignore this analysis. (TODO explain better.. within error yada yada).

4.3.4 Conclusions

Italian prime minister seem to be overall a bit aggressive and negative, and they speak more often then not about the opposition. Almost no traces of hate speech and attacks against minority groups are found in the data.

This does not seem to change over time, failing to explain the VDEM indices drop in 2021.

5 Discussion & Future Works

- Analyses results
  - so can we explain VDEM drops and stuff?
- How to improve the work
  - more scraping (increase dataset)
  - better transcription (even tho they are pretty good)
  - bigger LLMs for the sentiment analysis
  - better doc2vec? gensim still not compatible with current Python version
- How to expand the work
  - include leader of opposition
  - include trend setter of the time
  - include other countries (eg France or USA)

A Appendix

1.1 Tables

1.1.1 Not Colored

Level	Field	Description	Example
1	`tenant_id`	Logical isolation between organizations	`agroTech01`
2	`site_type`	Type of agricultural facility	`greenhouse`, `open field`
3	`site_id`	Physical site identifier	`GH-001`
4	`section_id`	Section within the site	`section-2`
5	`zone_id`	Operational zone	`zone-A`
6	`entity_type`	Device category	`sensor`, `actuator`
7	`entity_subclass`	Specific device type	`temp`, `ph`, `water pump`
8	`entity_id`	Unique device identifier	`mqtt-device-03`
9	`message_type`	Message classification	`data`, `cmd`, `status`, `alert`
10	`target_id`	Target device for commands (optional)	`cooler-001`

Table A.7: 10-level topic hierarchy structure

1.1.2 Colored

Level	Field	Description	Example
1	tenant_id	Logical isolation between organizations	agroTech01
2	site_type	Type of agricultural facility	greenhouse, open field
3	site_id	Physical site identifier	GH-001
4	section_id	Section within the site	section-2
5	zone_id	Operational zone	zone-A
6	entity_type	Device category	sensor, actuator
7	entity_subclass	Specific device type	temp, ph, water pump
8	entity_id	Unique device identifier	mqtt-device-03
9	message_type	Message classification	data, cmd, status, alert
10	target_id	Target device for commands (optional)	cooler-001

Table A.8: 10-level topic hierarchy structure

1.1.3 Half Colored Half Not

Level	Field	Description	Example
1	tenant_id	Logical isolation between organizations	agroTech01
2	site_type	Type of agricultural facility	greenhouse, open field
3	site_id	Physical site identifier	GH-001
4	section_id	Section within the site	section-2
5	zone_id	Operational zone	zone-A
6	entity_type	Device category	sensor, actuator
7	entity_subclass	Specific device type	temp, ph, water pump
8	entity_id	Unique device identifier	mqtt-device-03
9	message_type	Message classification	data, cmd, status, alert
10	target_id	Target device for commands (optional)	cooler-001

Table A.9: 10-level topic hierarchy structure

1.1.4 Stacked

Table iot_records_tenant_01

Time	Device ID	Tenant ID	Type	Data (JSONB)
2025-11-10 08:03:12 UTC	temp_001	tenant_01	data	{“value”: 22.4, “unit”: “C”}
2025-11-10 08:05:44 UTC	fan_003	tenant_01	status	{“running”: true, “speed”: “medium”}
2025-11-10 08:07:01 UTC	temp_001	tenant_01	alert	{“code”: “OVER_THRESHOLD”, “value”: 29.1}
2025-11-10 08:09:33 UTC	fan_003	tenant_01	cmd	{“action”: “set_speed”, “target”: “high”}

Table iot_records_tenant_02

Time	Device ID	Tenant ID	Type	Data (JSONB)
2025-11-10 08:10:22 UTC	temp_007	tenant_02	data	{“value”: 18.9, “unit”: “C”}
2025-11-10 08:10:25 UTC	air_008	tenant_02	data	{“value”: 412, “unit”: “ppm”}
2025-11-10 08:12:01 UTC	temp_007	tenant_02	status	{“online”: true, “battery”: 87}

Table A.10: Sample rows from `iot_records_tenant_01` (top) and `iot_records_tenant_02` (bottom).

1.2 Figures

`#align()` without any arguments defaults to *center*.

1.2.1 Single Figure

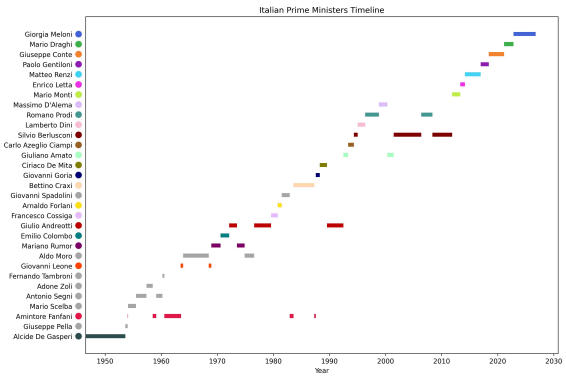
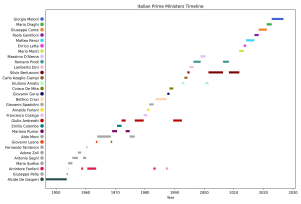


Figure A.18: Architecture of IoT (A: 3-layers) (B: 5-layers)

1.2.2 Double Figures



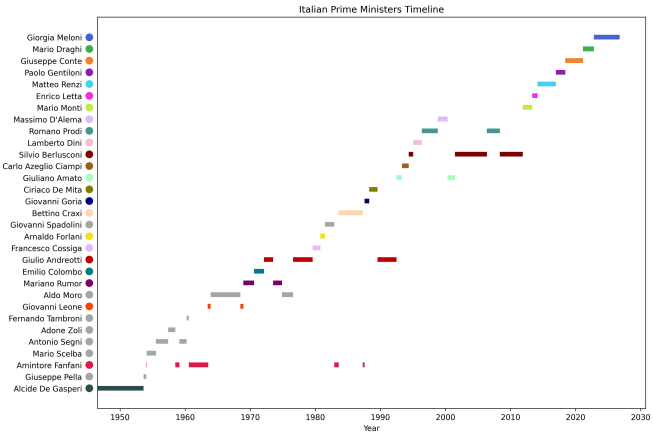


Figure A.19: Comparison between (a) Request-Response and (b) Publish-Subscribe paradigms

1.3 Code Listings

```
{
  "entity": { "type": "tenant", "id": "agroTech01" },
  "relation": "admin",
  "subject": { "type": "user", "id": "john_doe" }
}
```

Code A.19: Example of a Permify tuple

```
def extract_page_date(soup: BeautifulSoup) -> tuple[date | None, str]:
    """Page-level event date (meta > title > stamp > sommario prose)."""
    for attr, val in [("name", "dcterms.date"), ("property", "article:published_time")]:
        tag = soup.find("meta", {attr: val})
        if tag and tag.get("content"):
            d = _try_iso(tag["content"])
            if d: return d, "meta_tag"

    title = soup.find("title")
    if title:
        d = _try_dotted(title.get_text())
        if d: return d, "page_title"

    page_text = soup.get_text(" ", strip=True)
    d = _try_short(page_text)
    if d: return d, "page_stamp"

    d = _try_long(page_text)
    if d: return d, "page_sommario"

    return None, "not_found"
```

Code A.20: Example of a Permify tuple

```
   <li class="views-row views-row-1 clearfix">
         <div class="tipo_media audio">audio</div>

         <div class="ls_text">
         <span class="date-display-single" property="dc:date" datatype="xsd:dateTime"
content="1993-07-25T00:00:00+02:00">25.07.1993</span>
         <h3><a href="/scheda/55884/per-litalia-una-nuova-presenza-popolare?i=2447650">"Per
l'Italia una nuova presenza popolare"</a></h3>


         </div>
         <div class="ls_meta">
         <p class="dove"><span class="date-display-single" property="dc:date"
datatype="xsd:dateTime" content="1993-07-25T02:39:01+02:00">2:39</span></p>
```

```
        <p class="durata">Durata: 0:05:00</p>
        </div>  </li>
<li class="views-row views-row-2 clearfix">
        <div class="tipo_media audio">audio</div>

        <div class="ls_text">
        <span class="date-display-single" property="dc:date" datatype="xsd:dateTime"
content="1992-04-15T00:00:00+02:00">15.04.1992</span>
        <h3><a href="/scheda/46078/consiglio-nazionale-della-dc?i=2529451">Consiglio
Nazionale della DC</a></h3>


        </div>
        <div class="ls_meta">
        <p class="dove"><span class="date-display-single" property="dc:date"
datatype="xsd:dateTime" content="1992-04-15T02:29:01+02:00">2:29</span></p>
        <p class="durata">Durata: 0:04:00</p>
        </div>  </li>
<li class="views-row views-row-3 clearfix">
        <div class="tipo_media audio">audio</div>

        <div class="ls_text">
        <span class="date-display-single" property="dc:date" datatype="xsd:dateTime"
content="1991-11-30T00:00:00+01:00">30.11.1991</span>
        <h3><a href="/scheda/43012/conferenza-nazionale-della-democrazia-cristiana?
i=2554258">Conferenza Nazionale della Democrazia Cristiana</a></h3>


        </div>
        <div class="ls_meta">
        <p class="dove"><span class="date-display-single" property="dc:date"
datatype="xsd:dateTime" content="1991-11-30T07:15:02+01:00">7:15</span></p>
        <p class="durata">Durata: 0:11:00</p>
        </div>  </li>
<li class="views-row views-row-4 clearfix">
        <div class="tipo_media audio">audio</div>

        <div class="ls_text">
        <span class="date-display-single" property="dc:date" datatype="xsd:dateTime"
content="1991-09-28T00:00:00+02:00">28.09.1991</span>
        <h3><a href="/scheda/41372/riforme-istituzionali-e-sistema-dei-partiti?
i=2567492">"Riforme istituzionali e sistema dei partiti"</a></h3>


        </div>
        <div class="ls_meta">
        <p class="dove"><span class="date-display-single" property="dc:date"
datatype="xsd:dateTime" content="1991-09-28T02:32:01+02:00">2:32</span></p>
        <p class="durata">Durata: 0:10:00</p>
        </div>  </li>
<li class="views-row views-row-5 clearfix">
        <div class="tipo_media audio">audio</div>

        <div class="ls_text">
        <span class="date-display-single" property="dc:date" datatype="xsd:dateTime"
content="1989-11-18T00:00:00+01:00">18.11.1989</span>
        <h3><a href="/scheda/33836/consiglio-nazionale-dc?i=2631437">Consiglio nazionale Dc</
a></h3>


        </div>
        <div class="ls_meta">
        <p class="dove"><span class="date-display-single" property="dc:date"
datatype="xsd:dateTime" content="1989-11-18T03:49:02+01:00">3:49</span></p>
        <p class="durata">Durata: 0:13:00</p>
        </div>  </li>
<li class="views-row views-row-6 clearfix">
        <div class="tipo_media audio">audio</div>

        <div class="ls_text">
        <span class="date-display-single" property="dc:date" datatype="xsd:dateTime"
content="1989-09-06T00:00:00+02:00">6.09.1989</span>
        <h3><a href="/scheda/58780/lo-schok-da-perestrojka?i=2419587">"Lo schok da
perestrojka"</a></h3>

```

```
73
74          </div>
75          <div class="ls_meta">
76          <p class="durata">Durata: 0:00:01</p>
77          </div>  </li>
78  <li class="views-row views-row-7 clearfix">
79          <div class="tipo_media audio">audio</div>
80
81          <div class="ls_text">
82          <span class="date-display-single" property="dc:date" datatype="xsd:dateTime"
    content="1989-09-06T00:00:00+02:00">6.09.1989</span>
83          <h3><a href="/scheda/58780/lo-schok-da-perestrojka?i=2419589">"Lo schok da
    perestrojka"</a></h3>
84
85
86          </div>
87          <div class="ls_meta">
88          <p class="durata">Durata: 0:00:01</p>
89          </div>  </li>
90  <li class="views-row views-row-8 clearfix">
91          <div class="tipo_media audio">audio</div>
92
93          <div class="ls_text">
94          <span class="date-display-single" property="dc:date" datatype="xsd:dateTime"
    content="1989-09-06T00:00:00+02:00">6.09.1989</span>
95          <h3><a href="/scheda/58780/lo-schok-da-perestrojka?i=2419592">"Lo schok da
    perestrojka"</a></h3>
96
97    ..................
98
99          </div>
100         <div class="ls_meta">
101         <p class="durata">Durata: 3:00:00</p>
102         </div>  </li>
```

Code A.21: Example of the HTML in Radio Radicale that contains a list of links pointing to the speeches of a Prime Minister.

```
1             <div class="l-region l-region--seconda-colonna">
     <div id="block-quicktabs-blocco-interventi" class="block block--quicktabs block--
2   quicktabs-blocco-interventi block-first block-1 block-last">
3     <div class="block__content">
                  <div id="quicktabs-blocco_interventi" class="quicktabs-wrapper quicktabs-
    style-nostyle jquery-once-1-processed"><ul class="quicktabs-tabs quicktabs-style-nostyle"><li
    class="active"><a href="/scheda/41372/riforme-istituzionali-e-sistema-dei-partiti?
    i=2567492&amp;qt-blocco_interventi=0#qt-blocco_interventi" id="quicktabs-tab-
4   blocco_interventi-0" class="quicktabs-tab quicktabs-tab-block quicktabs-tab-block-views-
    delta-87664a805c1e84a93a9afe91f2d174cd active quicktabs-loaded jquery-once-2-
    processed">Interventi<span id="active-quicktabs-tab" class="element-invisible">(scheda
    attiva)</span></a></li>
    <li><a href="/scheda/41372/riforme-istituzionali-e-sistema-dei-partiti?i=2567492&amp;qt-
5   blocco_interventi=1#qt-blocco_interventi" id="quicktabs-tab-blocco_interventi-1"
    class="quicktabs-tab quicktabs-tab-block quicktabs-tab-block-rrblocks-delta-box-trascrizione
    active jquery-once-2-processed">Trascrizione automatica</a></li>
    </ul><div id="quicktabs-container-blocco_interventi" class="quicktabs_main quicktabs-style-
6   nostyle"><div id="quicktabs-tabpage-blocco_interventi-0" class="quicktabs-tabpage "><div
    id="block-views-87664a805c1e84a93a9afe91f2d174cd" class="block block--views block--
    views-87664a805c1e84a93a9afe91f2d174cd block-first block-1">
7     <div class="block__content">
                  <div class="view view-interventi-2016 view-id-interventi_2016 view-display-id-
8   blocco_interventi view-dom-id-cdc25d3e2de2a31806b2fe74f563f3ef">
9
10
11
12        <div class="view-content int_menu">
13        <ul>  <li class="int2567502 intervento d2220">
14
15
16            <div class="int_name_section">
17            <h2>Flaminio Piccoli</h2><p>DC</p>
18            </div>
19            <div class="int_subtext">
20            </div>
```

```
            <div class="durata"> 0:00 Durata: 37 min</div>
                <div class="actions" style="display: none;">
                <a href="/soggetti/247/flaminio-piccoli" title="altri interventi dell'oratore"><i class="fa fa-external-link" aria-hidden="true"></i>altri interventi</a>
            <a class="condividi condividi-intervento" href="#" data-param-i="2567502"><i class="fa fa-share-alt"></i>condividi intervento</a>
        </div>

    </li>
      <li class="int2567501 intervento d1800">


            <div class="int_name_section">
            <h2>Carlo Bernini</h2><p>MINISTRO</p>
            </div>
            <div class="int_subtext">
            </div>
            <div class="durata"> 0:37 Durata: 30 min</div>
                <div class="actions" style="display: none;">
                <a href="/soggetti/19978/carlo-bernini" title="altri interventi dell'oratore"><i class="fa fa-external-link" aria-hidden="true"></i>altri interventi</a>
            <a class="condividi condividi-intervento" href="#" data-param-i="2567501"><i class="fa fa-share-alt"></i>condividi intervento</a>
        </div>

    </li>
      <li class="int2567500 intervento d960 over">


            <div class="int_name_section">
            <h2>Lucio Toth</h2><p>DC</p>
            </div>
            <div class="int_subtext">
            </div>
            <div class="durata"> 1:07 Durata: 16 min</div>
                <div class="actions" style="display: block;">
                <a href="/soggetti/16113/lucio-toth" title="altri interventi dell'oratore"><i class="fa fa-external-link" aria-hidden="true"></i>altri interventi</a>
            <a class="condividi condividi-intervento" href="#" data-param-i="2567500"><i class="fa fa-share-alt"></i>condividi intervento</a>
        </div>

    ..................

    </li>
      <li class="int2567473 intervento d1800 ">


            <div class="int_name_section">
            <h2>Rosa Iervolino Russo</h2><p>MINISTRO</p>
            </div>
            <div class="int_subtext">
            </div>
            <div class="durata"> 6:56 Durata: 30 min</div>
                <div class="actions" style="display:none;">
                <a href="/soggetti/25029/rosa-iervolino-russo" title="altri interventi dell'oratore"><i class="fa fa-external-link" aria-hidden="true"></i>altri interventi</a>
            <a class="condividi condividi-intervento" href="#" data-param-i="2567473"><i class="fa fa-share-alt"></i>condividi intervento</a>
        </div>

    </li>
    </ul>    </div>
```

Code A.22: Example of the HTML in Radio Radicale that contains a list of interventions in a conference.

Model	Thinking	Label	κ	Accuracy	Macro Precision	Macro Recall	Macro F1
qwen2.5:7b-instruct-q4_K_M	-	Hate speech	0.173	0.696	0.634	0.622	0.584
qwen2.5:7b-instruct-q4_K_M	-	Negativity	0.297	0.570	0.579	0.612	0.566

qwen2.5:7b-instruct-q4_K_M	-	Aggressiveness	0.255	0.623	0.682	0.676	0.594
qwen2.5:7b-instruct-q4_K_M	-	Target	0.705	0.773	0.807	0.865	0.824
mistral:7b-instruct-v0.3-q4_K_M	-	Hate speech	0.128	0.729	0.650	0.682	0.616
mistral:7b-instruct-v0.3-q4_K_M	-	Negativity	0.061	0.382	0.326	0.421	0.348
mistral:7b-instruct-v0.3-q4_K_M	-	Aggressiveness	0.051	0.546	0.538	0.613	0.499
mistral:7b-instruct-v0.3-q4_K_M	-	Target	0.699	0.768	0.868	0.873	0.848
qwen3.5:4b-q8_0	off	Hate speech	0.091	0.560	0.463	0.497	0.475
qwen3.5:4b-q8_0	off	Negativity	0.222	0.502	0.620	0.494	0.451
qwen3.5:4b-q8_0	off	Aggressiveness	0.194	0.614	0.689	0.666	0.599
qwen3.5:4b-q8_0	off	Target	0.783	0.836	0.890	0.906	0.891
qwen3.5:4b	off	Hate speech	0.268	0.628	0.586	0.585	0.581
qwen3.5:4b	off	Negativity	0.208	0.459	0.424	0.446	0.397
qwen3.5:4b	off	Aggressiveness	0.363	0.691	0.736	0.736	0.683
qwen3.5:4b	off	Target	0.760	0.816	0.886	0.898	0.879
qwen3.5:9b-q4_K_M	off	Hate speech	0.037	0.517	0.501	0.486	0.476
qwen3.5:9b-q4_K_M	off	Negativity	0.246	0.522	0.599	0.518	0.483
qwen3.5:9b-q4_K_M	off	Aggressiveness	−0.009	0.541	0.535	0.560	0.538
qwen3.5:9b-q4_K_M	off	Target	0.721	0.787	0.872	0.878	0.864
gemma4:e4b	off	Hate speech	0.168	0.739	0.683	0.662	0.666
gemma4:e4b	**off**	**Negativity**	**0.293**	**0.536**	**0.534**	**0.564**	**0.532**
gemma4:e4b	**off**	**Aggressiveness**	**0.476**	**0.691**	**0.701**	**0.726**	**0.690**
gemma4:e4b	off	Target	0.674	0.758	0.833	0.837	0.835
gemma4:e4b	**on**	**Hate speech**	**0.538**	**0.826**	**0.819**	**0.777**	**0.773**
gemma4:e4b	on	Negativity	0.139	0.546	0.604	0.577	0.561
gemma4:e4b	on	Aggressiveness	0.246	0.604	0.689	0.644	0.614
gemma4:e4b	**on**	**Target**	**0.869**	**0.903**	**0.942**	**0.927**	**0.930**
gemma4:12b*	off	Hate speech	0.228	0.743	0.691	0.656	0.664
gemma4:12b*	off	Negativity	0.230	0.563	0.583	0.625	0.552
gemma4:12b*	off	Aggressiveness	0.303	0.655	0.702	0.707	0.634
gemma4:12b*	off	Target	0.722	0.786	0.865	0.881	0.854

Table A.11: Full validation results on the 207 speeches for all models, identified by their Ollama tags, with thinking mode off and on (- where thinking mode does not apply). In order: model, thinking mode, label, Cohen's κ (quadratic-weighted for hate speech, negativity and aggressiveness; unweighted for target), accuracy, and macro-averaged precision (P), recall (R) and F1. Rows in bold mark the configuration adopted for each label (gemma4:e4b). *Evaluated on 206/207 speeches, as one failed to be classified.

```
look at this speech from giorgia meloni:
#-------begin of csv----------#

politician,historical_date,location,tags,description,title,url,audio_file,text,
hate_speech,negativity,aggressiveness,
target,hate_speech,negativity,aggressiveness,target
```

meloni,2026-05-04,EREVAN,[],"Vertice della Comunità Politica Europea 4 Maggio 2026 Il Presidente del Consiglio, Giorgia Meloni, ha partecipato oggi a Yerevan, in Armenia, all'8° Vertice della Comunità Politica Europea (CPE), dove ha tenuto un intervento al panel di discussione su “Maintaining European unity and coherence in times of polycrisis”, moderato dal Presidente cipriota Christodoulides. Prima di partire alla volta di Baku, il Presidente ha tenuto un punto stampa. #yerevan #cpe #armenia #giorgiameloni #governomeloni","Vertice della Comunità Politica Europea, l'intervento del Presidente Giorgia Meloni",https://youtu.be/6gJWd0_AlTI?si=QEgiswzWpwDsMzNn,crossdem/datasets/meloni/audio_out/sMzNn_audio.mp3,"E sono completamente d'accordo con questo approccio, che l'importanza strategica del Mediterraneo e del miglior Mio Oste, promuovendo un approccio di 360 gradi tra l'estero e le flacche del sud, ma anche a diversi tipo di problemi comuni. Uno dei problemi comuni che abbiamo è la migrazione. In un ambiente molto complesso e internazionale oggi, come i paesi europei, come l'Italia, potrebbero migliorare i prezzetti del miglioramento? Abbiamo la crisi nel Mio Oste, quindi la migrazione è qui. Inoltre, la migrazione, la stabilità e la sicurezza in Europa sono in contatto. Grazie molto, Nikos. Buongiorno, amici, e grazie molto al Prime Minister Vashinian per questa fantastica organizzazione. Siamo oggi parlando di una crisi polica, ma cosa significa una crisi polica? Non è semplicemente che stiamo affrontando molte crisi insieme, è tutto. Il problema è che tutte queste crisi sono rilasciate ad uno all'altro e aiutate l'altro. E penso che sia molto importante capire questo, anche per affrontare il fatto che non possiamo semplicemente affrontare una o due facette di questa crisi, ma dobbiamo affrontare tutte le facette. E io utilizzerò esattamente, per cui Nikos chiedeva, la migrazione, l'esempio della migrazione. L'esempio della migrazione è anche linkato all'energia, perché dobbiamo anche affrontare il fatto che molte flotte vengono dalle regioni instable che sono la chiave dei nostri prodotti di energia. Ultimamente, tutto questo effettua la qualità delle nostre democrazie. Perché quando i cittadini pensano che le difficoltà importanti non sono governate, si perdono la fiducia nelle istituzioni. E quando la fiducia è erogata, le persone si diventano più vulnerabili alla manipolazione, inoltre all'uso maliccioso dell'intelligenza artificiale. Quindi non è migrazione. È migrazione, è economia, è democrazia, è competitività, è energia, è sicurezza. E se prendo uno degli altri problemi, posso linkarli tutti in modo stesso. Quindi prima di tutto non possiamo affrontare solo un pezzo di questa crisi polica. Secondo, la migrazione è parte integrale della crisi polica. Terzo, nessuno può affrontarlo solo. Quindi la cooperazione su questo è una precondizione. Abbiamo fatto dei passaggi molto importanti negli ultimi anni, al livello dell'Unione Europea, dove ora abbiamo un nuovo Pacto di Assilio, abbiamo una nuova definizione dei terzi paesi sicuri, stiamo andando verso una nuova regolazione sui risultati che ci aiuterà anche su, diciamo, soluzioni innovative, come quelle che l'Italia sta cercando di provare. Abbiamo fatto passaggi in avanti al livello dell'Unione Europea, dove stiamo lavorando per dare una interpretazione più moderna delle convenzioni che abbiamo, ma le abbiamo scritte molti anni fa. E al livello dell'EPC, con Cure, abbiamo promulgato un'altra declarazione, con la proposta che abbiamo fatto con Cure, la posizione è abbastanza chiarissima, dobbiamo combattere contro i trafici, dobbiamo evitare che una nuova crisi, come quella che abbiamo in 2015, passi di nuovo, e dobbiamo cooperare con i paesi di origine e transito. E questo mi porti alla prima e alla seconda parte della domanda che ha fatto Nikos. Il problema è se abbiamo questa crisi poli, possiamo affrontare la crisi in modo che sia aderente? La mia idea è che il livello dell'Unione Europea, per esempio, in anni recenti, abbiamo mostrato la nostra capacità di rispondere alle emergenze. Abbiamo fatto questo con la crisi della pandemia, abbiamo fatto questo con l'Ukraine, ma penso anche che ora dobbiamo stoppare, passare dalla nostra capacità di reagire, alla nostra capacità di anticipare. Questo significa che dobbiamo concentrarci molto più sulla nostra strategia nel termine lungo e queste discussioni sono molto utili per questo. La nostra strategia, in mia opinione, deve essere più forte, non solo sui paesi similari, ovviamente è molto importante, ma anche nella nostra città geografica. Ovviamente ciò che stiamo facendo ora con i paesi similari, con l'EPC, è molto importante e dobbiamo migliorare e sono molto felice che Mark sia qui. Ma penso che dovremmo anche concentrarci, per esempio dal punto di vista dell'Italia, quindi penso che dovremmo concentrarci molto più, per esempio, sulla nostra città mediterranea. Penso anche che dovremmo lavorare su un forum come l'EPC, che è fondamentale, ovviamente per noi in questi tempi, concentrati nella città geografica del mediterraneo. Nel passato con l'Urzula, con gli altri Stati membri, con l'Unione Europea, abbiamo provato a trattare la migrazione, siamo stati forzati a pagare molto più attenzione ai paesi attraverso il Mediterraneo. Ora, in mia opinione, il rischio è di ampliare questa approach, di portare insieme energia di rivolto di sicurezza e sviluppo nella nostra cooperazione con i paesi nello spazio. Per questo è la migliore forma in cui dobbiamo

```
rispondere a una crisi che non è solo una crisi. Ci sono molte crisi insieme, ma
infortunatamente sono parte della stessa crisi e dobbiamo trattare tutto.
Grazie.",low,medium,low,none

#---------end of csv-----------------#

the above speech was classified with this prompt:
"""
 i will paste in this chat speeches from italian prime minister formatted as csv. you have to
classify the "text" coloumn.


for each speech, classify them.

"hate_speech", "negativity", and "aggressiveness" are classified with

three levels, respectively "low", "mid" and "high". each speech has to

be classfied with one of the three values for each .


"target", on the other hand, should be classified independently from the

previous three. what i want is just to know whether the speech have one

of the following five targets: "none" if there is no target, "pol_adv"

if it speaks about political adversaries, "minor_etn" if it speaks about

ethnical minorities (eg balck people), "minor_gnd" if it speaks about

gender minorities (e.g., women or LGBT community), and "minor_rel" if it

speaks about religious miniorities (e.g., muslims).


give me the results in the form: politician, date ,low,low,low,pol_adv
"""


#--------------------#

unfortunately, most of my speeches have very low diversity in their classification, meaning
that data is heavily skewred towards low hate speech, low aggressiveness, low negativity and
target = none.

make me a syntetic csv dataset with 100 speeches, where: politician="synthetic"; random
historical date; location = "UNKNOWN"; no title, no url, no audio file coloumns.

all speeches on the other hand must have hate speech, aggressiveness, and negativity at least
medium for all three of them. try to make it so that the speeches have different amount of
those so that there is as much diverisity as possibile.

Try to make it so that all speeches have one of the possible four targets: "pol_adv" if it
speaks about political adversaries, "minor_etn" if it speaks about ethnical minorities (eg
balck people), "minor_gnd" if it speaks about gender minorities (e.g., women or LGBT
community), and "minor_rel" if it speaks about religious miniorities (e.g., muslims).

when you make a speech, classify it too.

to reiterate the dataset coloums, they are the following:
[politician,historical_date,location,text,hate_speech,negativity,
aggressiveness,target]
```

Code A.23: Prompt to generate 100 synthetic speeches with high hostility and diverse targets. The speeches were generated with Gemini 3.6 Flash.

```python
RUBRIC_HEADER = """Sei un annotatore esperto di scienze politiche e linguistica
computazionale.
Il tuo compito è classificare un discorso pronunciato da un Presidente del Consiglio
italiano."""

# think=True group: hate_speech + target
SYSTEM_PROMPT_THINK = RUBRIC_HEADER + """
```

```
6
7   ---
8   ### DIMENSIONI E CATEGORIE
9
10  1) hate_speech: presenza di incitamento all'odio verso individui o gruppi protetti.
11     - low: assente o non significativa.
12     - mid: presente in forma velata, allusiva, tramite "dog whistles" o pregiudizi impliciti.
13     - high: presente in forma esplicita, deumanizzante o aperta ostilità sistematica.
14
15  2) target: il bersaglio principale di eventuale ostilità/critica nel discorso (assegna il
    target
16     anche se il discorso è nel complesso pacato, identificando verso chi è orientato).
17     - none: nessun bersaglio specifico identificabile.
18     - pol_adv: avversari politici, opposizioni, partiti, burocrazia europea o altre
    istituzioni.
19     - minor_etn: minoranze etniche, persone straniere, migranti di specifiche etnie.
20     - minor_gnd: minoranze di genere, donne, comunità LGBTQ+.
21     - minor_rel: minoranze religiose (es. musulmani, ebrei).
22
23  ---
24  ### ESEMPI GUIDA PER LA CALIBRAZIONE
25
26  #### Esempi per `hate_speech`:
27  - **low**:
28    - "Dobbiamo investire nella scuola pubblica e garantire il salario minimo per sostenere le
    famiglie in difficoltà."
29    - "La transizione ecologica richiede una visione industriale chiara che tuteli
    l'occupazione e le piccole imprese."
30    - "Rafforzeremo il presidio del territorio e la cooperazione tra le forze dell'ordine e le
    comunità locali."
31  - **mid**:
32    - "Non possiamo permettere che i valori della nostra civiltà vengano diluiti da chi arriva
    da fuori senza alcuna intenzione di assimilarsi."
33    - "Certe culture tradizionaliste restano intrinsecamente incompatibili con la tutela dei
    diritti fondamentali e dello Stato di diritto."
34    - "I quartieri storici stanno perdendo la propria identità a causa di una presenza
    straniera ormai dominante che rifiuta le nostre regole."
35  - **high**:
36    - "L'immigrazione è un'invasione pianificata che porterà degrado e criminalità nelle nostre
    città. Bisogna eliminarla sistematicamente strada per strada, casa per casa."
37    - "Questi gruppi parassitari infestano la nostra società e vanno estirpati prima che
    distruggano definitivamente la nostra razza e la nostra storia."
38    - "Alcune minoranze sono portatrici biologiche di violenza e inciviltà: vanno cacciate con
    ogni mezzo dal nostro suolo patrio."
39
40  #### Esempi per `target`:
41  - **none**:
42    - "Oggi approviamo la riforma del codice della strada per ridurre gli incidenti e tutelare
    i giovani."
43    - "Il piano di digitalizzazione della pubblica amministrazione consentirà di ridurre
    radicalmente i tempi di attesa."
44    - "I finanziamenti per la prevenzione del dissesto idrogeologico copriranno tutte le
    regioni a rischio."
45  - **pol_adv**:
46    - "La precedente maggioranza ha lasciato buchi di bilancio incalcolabili per fare
    propaganda elettorale."
47    - "L'Unione Europea pretende di imporre direttive ideologiche che penalizzano le nostre
    filiere produttive nazionali."
48    - "La magistratura politicamente orientata continua a travalicare i propri confini
    costituzionali per ostacolare l'esecutivo."
49  - **minor_etn**:
50    - "È necessario bloccare le partenze e combattere le reti di clandestinità che
    destabilizzano la sicurezza nazionale."
51    - "Certi flussi migratori incontrollati provenienti dall'Africa subsahariana importano
    modelli criminali inaccettabili."
52    - "Non tollereremo zone di franchigia gestite da bande etniche straniere nelle periferie
    dei nostri capoluoghi."
53  - **minor_gnd**:
54    - "Ci opporremo a chi vuole scardinare la famiglia naturale imponendo teorie ideologiche
    nelle scuole."
55    - "La propaganda sull'identità di genere mira a cancellare la figura della madre e il ruolo
    biologico della donna."
```

```
  - "I diritti delle donne vengono calpestati dall'ossessione per il politicamente corretto e la fluidità di genere."
- **minor_rel**:
  - "Alcune comunità religiose pretendono di applicare le proprie leggi teocratiche sul nostro territorio nazionale."
  - "Il proliferare di centri di preghiera abusivi legati all'Islam radicale rappresenta una minaccia diretta ai nostri valori laici."
  - "Non faremo concessioni a chi usa il proprio culto per giustificare la sottomissione femminile e l'odio verso l'Occidente."

---
### FORMATO DI OUTPUT

Rispondi ESCLUSIVAMENTE con un oggetto JSON con queste due chiavi (hate_speech, target), senza testo aggiuntivo, markdown o spiegazioni."""

# think=False group: negativity + aggressiveness
SYSTEM_PROMPT_NOTHINK = RUBRIC_HEADER + """

---
### DIMENSIONI E CATEGORIE

1) negativity: tono complessivo negativo del discorso (critica, pessimismo, minacce percepite, allarmismo).
   - low: tono prevalentemente neutro, costruttivo o positivo.
   - mid: presenza moderata di toni critici o allarmi su specifici problemi nazionali.
   - high: tono fortemente cupo, allarmista, focalizzato su crisi imminenti o declino.

2) aggressiveness: aggressività verbale, toni polemici o ostili verso interlocutori/avversari.
   - low: tono pacato, formale, istituzionale.
   - mid: presenza moderata di polemica, velate stoccate o sarcasmo istituzionale.
   - high: tono marcatamente polemico, attacchi diretti e personali, linguaggio di scontro.

---
### ESEMPI GUIDA PER LA CALIBRAZIONE

#### Esempi per `negativity`:
- **low**:
  - "I dati sull'occupazione sono incoraggianti e la crescita economica dimostra la resilienza del nostro tessuto produttivo."
  - "La riforma della giustizia procede spedita, restituendo efficienza e tempi certi a cittadini e imprese."
  - "Il nostro posizionamento internazionale si rafforza grazie a nuove partnership strategiche sull'energia."
- **mid**:
  - "I sistemi sanitario e scolastico presentano problematiche infrastrutturali apparentemente insanabili."
  - "L'instabilità geopolitica globale proietta ombre preoccupanti sull'approvvigionamento delle nostre materie prime."
  - "La burocrazia soffocante e i tempi della giustizia civile continuano a frenare gli investimenti esteri nel Paese."
- **high**:
  - "Il paese è sull'orlo del baratro finanziario; ereditiamo un disastro sistemico che rischia di spazzare via i risparmi degli italiani."
  - "Siamo di fronte a un declino demografico e sociale irreversibile che sta portando la Nazione alla completa estinzione."
  - "Il collasso della sicurezza urbana ha trasformato le nostre metropoli in zone di guerra allo sbaraglio."

#### Esempi per `aggressiveness`:
- **low**:
  - "Accogliamo le osservazioni delle opposizioni nel merito, tuttavia il Governo continuerà con la linea dettata dagli elettori."
  - "Il confronto parlamentare è la sede naturale per affinare i testi di legge nell'interesse generale."
  - "Valuteremo con attenzione tutti gli emendamenti proposti dalle forze di minoranza durante il percorso in commissione."
- **mid**:
  - "Riconosco il diritto dell'opposizione di protestare, anche se la loro memoria storica appare alquanto corta e di comodo."
  - "Ci danno lezioni di rigore di bilancio gli stessi banchieri e tecnocrati che hanno affossato i conti pubblici nel decennio scorso."
```

```
  - "Spiace constatare come parte dei media preferisca la polemica strumentale all'analisi obiettiva dei fatti."
- **high**:
  - "Dall'opposizione arrivano solo menzogne sfrontate e sciacallaggio politico da parte di chi ha svenduto la nazione per anni."
  - "Siete dei traditori del popolo italiano, dei cospiratori servili che prendono ordini da potenze e burocrazie straniere."
  - "La vostra ipocrisia fa schifo: avete le mani sporche di sangue per le politiche criminali che avete approvato!"

---
### FORMATO DI OUTPUT

Rispondi ESCLUSIVAMENTE con un oggetto JSON con queste due chiavi (negativity, aggressiveness),
senza testo aggiuntivo, markdown o spiegazioni."""


JSON_SCHEMA_THINK = {
    "type": "object",
    "properties": {
        "hate_speech": {"type": "string", "enum": LEVEL_VALUES},
        "target": {"type": "string", "enum": TARGET_VALUES},
    },
    "required": THINK_COLS,
}

JSON_SCHEMA_NOTHINK = {
    "type": "object",
    "properties": {
        "negativity": {"type": "string", "enum": LEVEL_VALUES},
        "aggressiveness": {"type": "string", "enum": LEVEL_VALUES},
    },
    "required": NOTHINK_COLS,
}
```

Code A.24: Prompts to be fed into Gemma 4 to classify the corpus and their respective JSON output schemas. One prompt to classify negativity and aggressiveness with thinking disabled , and one prompt to classify hate speech and target with thinking enabled.

References

[1] A. Karjus and C. Cuskley, "Evolving Linguistic Divergence on Polarizing Social Media." Accessed: Sep. 24, 2026. [Online]. Available: http://arxiv.org/abs/2309.01659

[2] B. I. Page and R. Y. Shapiro, "Presidents as Opinion Leaders: Some New Evidence," *Policy Studies Journal*, vol. 12, no. 4, pp. 649–662, Jun. 1984, doi: 10.1111/j.1541-0072.1984.tb00480.x.

[3] A. Beard, *The Language of Politics*. Routledge, 2000. [Online]. Available: https://newuniversityinexileconsortium.org/wp-content/uploads/2022/02/the-language-of-politics.pdf

[4] J. Charteris-Black, "Persuasion, Speech Making and Rhetoric," in *Politicians and Rhetoric: The Persuasive Power of Metaphor*, J. Charteris-Black, Ed., London: Palgrave Macmillan UK, 2011, pp. 1–27. doi: 10.1057/9780230319899_1.

[5] K. Clayton, N. T. Davis, B. Nyhan, E. Porter, T. J. Ryan, and T. J. Wood, "Elite Rhetoric Can Undermine Democratic Norms," *Proceedings of the National Academy of Sciences*, vol. 118, no. 23, p. e2024125118, Jun. 2021, doi: 10.1073/pnas.2024125118.

[6] A. Madhani and J. Colvin, "A Farewell to @realDonaldTrump, Gone after 57,000 Tweets." Accessed: Apr. 21, 2026. [Online]. Available: https://apnews.com/article/twitter-donald-trump-ban-cea450b1f12f4ceb8984972a120018d5

[7] A. Vinci, "Meloni leader anche sui social, 3,3 milioni di nuovi follower nel 2025: staccato Salvini. Schlein prima per engagement." Accessed: Apr. 21, 2026. [Online]. Available: https://www.corriere.it/tecnologia/25_dicembre_17/meloni-leader-anche-sui-social-3-3-milioni-di-nuovi-follower-nel-2025-staccato-salvini-schlein-prima-per-engagement-431d8cda-a6e3-4d04-880b-922639075xlk.shtml

[8] Eurobarometer, "Media & News Survey 2023 - November 2023 - - Eurobarometer Survey," 2023. doi: 10.2861/11595.

[9] B. Maher, "Twice as Many Brits Got 2024 Election News from TV as from Social Media, Ofcom Finds." Accessed: Jan. 29, 2026. [Online]. Available: https://pressgazette.co.uk/news/ofcom-2024-election-news-sources-television-social-media-working-class/

[10] P. Lorenz-Spreen, L. Oswald, S. Lewandowsky, and R. Hertwig, "A Systematic Review of Worldwide Causal and Correlational Evidence on Digital Media and Democracy," *Nature Human Behaviour*, vol. 7, no. 1, pp. 74–101, Jan. 2023, doi: 10.1038/s41562-022-01460-1.

[11] E. Amsalem and A. Zoizner, "Do People Learn about Politics on Social Media? A Meta-Analysis of 76 Studies," *Journal of Communication*, vol. 73, no. 1, pp. 3–13, Feb. 2023, doi: 10.1093/joc/jqac034.

[12] L. Belcastro, F. Branda, R. Cantini, F. Marozzo, D. Talia, and P. Trunfio, "Analyzing Voter Behavior on Social Media during the 2020 US Presidential Election Campaign," *Social Network Analysis and Mining*, vol. 12, no. 1, p. 83, Jul. 2022, doi: 10.1007/s13278-022-00913-9.

[13] P. Rita, N. António, and A. P. Afonso, "Social Media Discourse and Voting Decisions Influence: Sentiment Analysis in Tweets during an Electoral Period," *Social Network Analysis and Mining*, vol. 13, no. 1, p. 46, Mar. 2023, doi: 10.1007/s13278-023-01048-1.

[14] B. C. Silva and S.-O. Proksch, "Politicians Unleashed? Political Communication on Twitter and in Parliament in Western Europe," *Political Science Research and Methods*, vol. 10, no. 4, pp. 776–792, Oct. 2022, doi: 10.1017/psrm.2021.36.

[15] G. Holtus and M. Cortelazzo, "Dal parlato al (tra)scritto: i resoconti stenografici dei discorsi parlamentari," in *Gesprochenes Italienisch in Geschichte und Gegenwart*, G. Narr, 1985, pp. 86–118. [Online]. Available: https://www.uni-saarland.de/fileadmin/upload/fachrichtung/romanistik/W._Schweickard/Publikationen/Gesprochenes_Italienisch_in_Geschichte_und_Gegenwart.pdf

[16] A. Mohrhoff, "Dalla Lingua Del Parlamento Alla Lingua Del Parlamentare," in *Aurelia Mohrhoff, Dalla Lingua Del Parlamento Alla Lingua Del Parlamentare*, Roma: Camera dei Deputati, 1987, pp. 145–156. Accessed: Jan. 29, 2026. [Online]. Available: https://bpr.camera.it/bpr/allegati/show/3815_828_t

[17] W. Chung and J. Kim, "Exploiting Vocabulary Frequency Imbalance in Language Model Pre-Training." Accessed: Feb. 19, 2026. [Online]. Available: http://arxiv.org/abs/2508.15390

[18] L. Birkenmaier and C. Lechner, "Measuring Politicians' Public Personality Traits Using Computational Text Analysis: A Multimethod Feasibility Study for Agency and Communion," *Political Analysis*, vol. 34, no. 3, pp. 373–391, Jul. 2026, doi: 10.1017/pan.2025.10025.

[19] M. Schoonvelde, G. Schumacher, and B. N. Bakker, "Friends With Text as Data Benefits: Assessing and Extending the Use of Automated Text Analysis in Political Science and Political Psychology," *Journal of Social and Political Psychology*, vol. 7, no. 1, pp. 124–143, Feb. 2019, doi: 10.5964/jspp.v7i1.964.

[20] P. Bull and M. Waddle, *The Psychology of Political Communication: Politicians Under the Microscope*. London: Routledge, 2023. doi: 10.4324/9781003326533.

[21] J. Lange, J. Oliphant, and J. Lange, "Republicans Have Taken Sharp Populist Turn in the Trump Era: Reuters/Ipsos," *Reuters*, Mar. 2024, Accessed: Sep. 11, 2026. [Online]. Available: https://www.reuters.com/world/us/us-republicans-have-taken-sharp-populist-turn-trump-era-reutersipsos-data-shows-2024-03-21/

[22] É. Massicard, "Populism in Turkey: Towards the Demise of Democracy?." Accessed: Sep. 11, 2026. [Online]. Available: https://webservero7.reims.sciences-po.fr/research/cogito/home/populism-in-turkey-towards-the-demise-of-democracy/?lang=en

[23] L. Tillin, "The Political Economy of Populism in India | Feature from King's College London." Accessed: Sep. 11, 2026. [Online]. Available: https://www.kcl.ac.uk/the-political-economy-of-populism-in-india

[24] G. Gyori, "Populism in Europe: Hungary." Accessed: Sep. 11, 2026. [Online]. Available: https://www.opensocietyfoundations.org/publications/populism-europe-hungary

[25] C. R. Kaltwasser, "The Ambivalence of Populism: Threat and Corrective for Democracy," *Democratization*, vol. 19, no. 2, pp. 184–208, Apr. 2012, doi: 10.1080/13510347.2011.572619.

[26] G. Scheiring, "I Watched Hungary's Democracy Dissolve into Authoritarianism as a Member of Parliament – and I See Troubling Parallels in Trumpism and Its Appeal to Workers." Accessed: Sep. 11, 2026. [Online]. Available: https://theconversation.com/i-watched-hungarys-democracy-dissolve-into-authoritarianism-as-a-member-of-parliament-and-i-see-troubling-parallels-in-trumpism-and-its-appeal-to-workers-224930

[27] E. C. Nisbet, "The Tragedy of Turkish Democracy in Five Acts." Accessed: Sep. 11, 2026. [Online]. Available: https://theconversation.com/the-tragedy-of-turkish-democracy-in-five-acts-62678

[28] R. C. Kittel, "Simply Speaking? Language Complexity among (Non-)Populist Actors in Parliamentary Debates," *Government and Opposition*, vol. 60, no. 4, pp. 1336–1366, Oct. 2025, doi: 10.1017/gov.2025.4.

[29] S. Decadri and C. Boussalis, "Populism, Party Membership, and Language Complexity in the Italian Chamber of Deputies," *Journal of Elections, Public Opinion and Parties*, vol. 30, no. 4, pp. 484–503, Oct. 2020, doi: 10.1080/17457289.2019.1593182.

[30] D. McDonnell and S. Ondelli, "The Language of Right-Wing Populist Leaders: Not So Simple," *Perspectives on Politics*, vol. 20, no. 3, pp. 828–841, Sep. 2022, doi: 10.1017/S1537592720002418.

[31] Merriam-Webster, "Definition of EMOTIVE." Accessed: Sep. 15, 2026. [Online]. Available: https://www.merriam-webster.com/dictionary/emotive

[32] Y. Wang, R. Yu, Z. Wang, and J. He, "The Sound of Populism: Distinct Linguistic Features Across Populist Variants," *Humanities and Social Sciences Communications*, vol. 13, no. 1, p. 1001, Apr. 2026, doi: 10.1057/s41599-026-06799-8.

[33] A.-M. Bliuc, I. Yilmaz, R. Norrie, S. De Groot Heupner, N. Morieson, and M. J. Belanger, "The Emotional Logic of Populist Communication: A Systematic Review," *Information, Communication & Society*, vol. 0, no. 0, pp. 1–26, Mar. 2026, doi: 10.1080/1369118X.2026.2639567.

[34] L. Alonso-Muñoz and A. Casero-Ripollés, "The Appeal to Emotions in the Discourse of Populist Political Actors from Spain, Italy, France and the United Kingdom on Twitter," *Frontiers in Communication*, vol. 8, May 2023, doi: 10.3389/fcomm.2023.1159847.

[35] M. Caiani and J. D. Cocco, "Populism and Emotions: A Comparative Study Using Machine Learning," *Italian Political Science Review / Rivista Italiana di Scienza Politica*, vol. 53, no. 3, pp. 351–366, Nov. 2023, doi: 10.1017/ipo.2023.8.

[36] K. M. Roberts, "Populism and Polarization in Comparative Perspective: Constitutive, Spatial and Institutional Dimensions," *Government and Opposition*, vol. 57, no. 4, pp. 680–702, Oct. 2022, doi: 10.1017/gov.2021.14.

[37] M. A. C. G. van der Velden, A. L. Ortega, D. J. Roth, and P. Guldemond, "Populism and Polarization: A Nostalgic Narrative of 'Us' and 'Them'," *Government and Opposition*, vol. 60, no. 4, pp. 1408–1425, Oct. 2025, doi: 10.1017/gov.2025.10020.

[38] J. Benson, "Democracy and the Epistemic Problems of Political Polarization," *American Political Science Review*, vol. 118, no. 4, pp. 1719–1732, Nov. 2024, doi: 10.1017/S0003055423001089.

[39] C. F. Rostbøll, "Polarization and the Democratic System: Kinds, Reasons, and Sites," *Perspectives on Politics*, vol. 23, no. 1, pp. 76–92, Mar. 2025, doi: 10.1017/S1537592724000525.

[40] A. Schedler, "Rethinking Political Polarization," *Political Science Quarterly*, vol. 138, no. 3, pp. 335–359, Aug. 2023, doi: 10.1093/psquar/qqad038.

[41] M. H. Graham and M. W. Svolik, "Democracy in America? Partisanship, Polarization, and the Robustness of Support for Democracy in the United States," *American Political Science Review*, vol. 114, no. 2, pp. 392–409, May 2020, doi: 10.1017/S0003055420000052.

[42] G. Simonovits, J. McCoy, and L. Littvay, "Democratic Hypocrisy and Out-Group Threat: Explaining Citizen Support for Democratic Erosion," *The Journal of Politics*, vol. 84, no. 3, pp. 1806–1811, Jul. 2022, doi: 10.1086/719009.

[43] W. H. DuBay, "The Principles of Readability," 2004.

[44] E. Dale and J. S. Chall, "A Formula for Predicting Readability: Instructions," *Educational Research Bulletin*, vol. 27, no. 2, pp. 37–54, 1948, Accessed: Sep. 18, 2026. [Online]. Available: https://www.jstor.org/stable/1473669

[45] A. C. Graesser, D. S. McNamara, M. M. Louwerse, and Z. Cai, "Coh-Metrix: Analysis of Text on Cohesion and Language," *Behavior Research Methods, Instruments, & Computers*, vol. 36, no. 2, pp. 193–202, May 2004, doi: 10.3758/BF03195564.

[46] R. Flesch, "A New Readability Yardstick," *Journal of Applied Psychology*, vol. 32, no. 3, pp. 221–233, 1948, doi: 10.1037/h0057532.

[47] G. H. Mc Laughlin, "SMOG Grading-a New Readability Formula," *Journal of Reading*, vol. 12, no. 8, pp. 639–646, 1969, Accessed: Sep. 18, 2026. [Online]. Available: https://www.jstor.org/stable/40011226

[48] P. M. McCarthy, "An Assessment of the Range and Usefulness of Lexical Diversity Measures and the Potential of the Measure of Textual, Lexical Diversity (MTLD) - ProQuest." Accessed: Sep. 18, 2026. [Online]. Available: https://www.proquest.com/openview/860b2901fa90c6e68e46cd9111bd2d1c

[49] D. Lin, N. Zhao, D. Tian, and J. Li, "Large Language Models Reshape the Language of Science," 2025.

[50] Y. Bestgen, "Measuring Lexical Diversity in Texts: The Twofold Length Problem," *Language Learning*, vol. 74, no. 3, pp. 638–671, 2024, doi: 10.1111/lang.12630.

[51] D. J. Velasco and M. T. Roque, "Rethinking the Role of Text Complexity in Language Model Pretraining." Accessed: Sep. 17, 2026. [Online]. Available: http://arxiv.org/abs/2509.16551

[52] G. Penedo *et al.*, "The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale." Accessed: Sep. 22, 2026. [Online]. Available: http://arxiv.org/abs/2406.17557

[53] A. Grattafiori *et al.*, "The Llama 3 Herd of Models." Accessed: Sep. 22, 2026. [Online]. Available: http://arxiv.org/abs/2407.21783

[54] E. Amadori *et al.*, "Involvement Drives Complexity of Language in Online Debates." Accessed: Sep. 18, 2026. [Online]. Available: http://arxiv.org/abs/2506.22098

[55] C. U. Yule, *The Statistical Study of Literary Vocabulary*. Cambridge University Press, 2014.

[56] E. Parada-Cabaleiro *et al.*, "Song Lyrics Have Become Simpler and More Repetitive over the Last Five Decades," *Scientific Reports*, vol. 14, no. 1, p. 5531, Mar. 2024, doi: 10.1038/s41598-024-55742-x.

[57] A. Desiderio, A. Mancini, G. Cimini, and R. D. Clemente, "Recurring Patterns in Online Social Media Interactions during Highly Engaging Events." Accessed: Sep. 22, 2026. [Online]. Available: http://arxiv.org/abs/2306.14735

[58] N. Di Marco, E. Loru, A. Bonetti, A. O. G. Serra, M. Cinelli, and W. Quattrociocchi, "Patterns of Linguistic Simplification on Social Media Platforms over Time," *Proceedings of the National Academy of Sciences*, vol. 121, no. 50, p. e2412105121, Dec. 2024, doi: 10.1073/pnas.2412105121.

[59] D. R. Fredrick and L. Craven, "Lexical Diversity, Syntactic Complexity, and Readability: A Corpus-Based Analysis of ChatGPT and L2 Student Essays," *Frontiers in Education*, vol. 10, Sep. 2025, doi: 10.3389/feduc.2025.1616935.

[60] M. P. Fiorina and S. J. Abrams, "Political Polarization in the American Public," *Annual Review of Political Science*, vol. 11, pp. 563–588, 2008, doi: 10.1146/annurev.polisci.11.053106.153836.

[61] D. Baldassarri and A. Gelman, "Partisans without Constraint: Political Polarization and Trends in American Public Opinion1," *American Journal of Sociology*, vol. 114, no. 2, pp. 408–446, Sep. 2008, doi: 10.1086/590649.

[62] P. DiMaggio, J. Evans, and B. Bryson, "Have American's Social Attitudes Become More Polarized?," *American Journal of Sociology*, vol. 102, no. 3, pp. 690–755, Nov. 1996, doi: 10.1086/230995.

[63] S. Iyengar, Y. Lelkes, M. Levendusky, N. Malhotra, and S. J. Westwood, "The Origins and Consequences of Affective Polarization in the United States," *Annual Review of Political Science*, vol. 22, pp. 129–146, 2019, doi: 10.1146/annurev-polisci-051117-073034.

[64] E. G. Carmines, M. J. Ensley, and M. W. Wagner, "Who Fits the Left-Right Divide? Partisan Polarization in the American Electorate," *American Behavioral Scientist*, vol. 56, no. 12, pp. 1631–1653, Dec. 2012, doi: 10.1177/0002764212463353.

[65] G. C. Layman, T. M. Carsey, and J. M. Horowitz, "PARTY POLARIZATION IN AMERICAN POLITICS: Characteristics, Causes, and Consequences," *Annual Review of Political Science*, vol. 9, pp. 83–110, 2006, doi: 10.1146/annurev.polisci.9.070204.105138.

[66] M. J. Hetherington, "Review Article: Putting Polarization in Perspective," *British Journal of Political Science*, vol. 39, no. 2, pp. 413–448, Apr. 2009, doi: 10.1017/S0007123408000501.

[67] M. J. Hetherington and T. J. Rudolph, *Why Washington Won't Work: Polarization, Political Trust, and the Governing Crisis*. in Chicago Studies in American Politics. Chicago, IL: University of Chicago Press, 2015. Accessed: Sep. 23, 2026. [Online]. Available: https://press.uchicago.edu/ucp/books/book/chicago/W/bo21516007.html

[68] T. Orian Harel, I. Maoz, and E. Halperin, "A Conflict within a Conflict: Intragroup Ideological Polarization and Intergroup Intractable Conflict," *Current Opinion in Behavioral Sciences*, vol. 34, pp. 52–57, Aug. 2020, doi: 10.1016/j.cobeha.2019.11.013.

[69] B. Nettasinghe, A. G. Percus, and K. Lerman, "How Out-Group Animosity Can Shape Partisan Divisions: A Model of Affective Polarization," *PNAS Nexus*, vol. 4, no. 3, p. pgaf082, Mar. 2025, doi: 10.1093/pnasnexus/pgaf082.

[70] T. E. Mann and N. J. Ornstein, *It's Even Worse Than It Looks: How the American Constitutional System Collided with the New Politics of Extremism*. Basic Books, 2016.

[71] H. Hahm, D. Hilpert, and T. König, "Divided by Europe: Affective Polarisation in the Context of European Elections," Nov. 2022, doi: 10.6084/m9.figshare.21514722.v1.

[72] R. L. Claassen and B. Highton, "Policy Polarization among Party Elites and the Significance of Political Awareness in the Mass Public," *Political Research Quarterly*, vol. 62, no. 3, pp. 538–551, Sep. 2009, doi: 10.1177/1065912908322415.

[73] A. Geiger, "Partisanship and Political Animosity in 2016." Accessed: Sep. 23, 2026. [Online]. Available: https://www.pewresearch.org/politics/2016/06/22/partisanship-and-political-animosity-in-2016/

[74] M. P. García-Guadilla and A. Mallen, "Polarization, Participatory Democracy, and Democratic Erosion in Venezuela's Twenty-First Century Socialism," *The ANNALS of the American Academy of Political and Social Science*, vol. 681, no. 1, pp. 62–77, Jan. 2019, doi: 10.1177/0002716218817733.

[75] B. McMurtrie, M. Philipp, R. Hebden, and M. Williams, "Development and Validation of the Affective Polarization Scale," *International Review of Social Psychology*, vol. 37, no. 1, Jun. 2024, doi: 10.5334/irsp.926.

[76] M. Gentzkow, J. M. Shapiro, and M. Taddy, "Measuring Group Differences in High-Dimensional Choices: Method and Application to Congressional Speech," *Econometrica*, vol. 87, no. 4, pp. 1307–1340, 2019, doi: 10.3982/ECTA16566.

[77] B. Nettasinghe, A. Rao, B. Jiang, A. G. Percus, and K. Lerman, "In-Group Love, Out-Group Hate: A Framework to Measure Affective Polarization via Contentious Online Discussions," in *Proceedings of the ACM on Web Conference 2025*, in WWW '25. New York, NY, USA: Association for Computing Machinery, Apr. 2025, pp. 560–575. doi: 10.1145/3696410.3714935.

[78] L. McInnes, J. Healy, N. Saul, and L. Großberger, "UMAP: Uniform Manifold Approximation and Projection," *Journal of Open Source Software*, vol. 3, no. 29, p. 861, Sep. 2018, doi: 10.21105/joss.00861.

[79] F. Hamborg and K. Donnay, "NewsMTSC: A Dataset for (Multi-)Target-Dependent Sentiment Classification in Political News Articles," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, P. Merlo, J. Tiedemann, and R. Tsarfaty, Eds., Online: Association for Computational Linguistics, Apr. 2021, pp. 1663–1675. doi: 10.18653/v1/2021.eacl-main.142.

[80] J. Chen, S. Yan, and K.-C. Wong, "Verbal Aggression Detection on Twitter Comments: Convolutional Neural Network for Short-Text Sentiment Analysis," *Neural Computing and Applications*, vol. 32, no. 15, pp. 10809–10818, Aug. 2020, doi: 10.1007/s00521-018-3442-0.

[81] T. Wang, K. Lu, K. P. Chow, and Q. Zhu, "COVID-19 Sensing: Negative Sentiment Analysis on Social Media in China via BERT Model," *IEEE Access*, vol. 8, pp. 138162–138169, 2020, doi: 10.1109/ACCESS.2020.3012595.

[82] S. MacAvaney, H.-R. Yao, E. Yang, K. Russell, N. Goharian, and O. Frieder, "Hate Speech Detection: Challenges and Solutions," *PLOS ONE*, vol. 14, no. 8, p. e0221152, Aug. 2019, doi: 10.1371/journal.pone.0221152.

[83] S. Behdenna, F. Barigou, and G. Belalem, "Sentiment Analysis at Document Level," in *Smart Trends in Information Technology and Computer Communications*, A. Unal, M. Nayak, D. K. Mishra, D. Singh, and A. Joshi, Eds., Singapore: Springer Nature, 2016, pp. 159–168. doi: 10.1007/978-981-10-3433-6_20.

[84] H. H. Do, P. Prasad, A. Maag, and A. Alsadoon, "Deep Learning for Aspect-Based Sentiment Analysis: A Comparative Review," *Expert Systems with Applications*, vol. 118, pp. 272–299, Mar. 2019, doi: 10.1016/j.eswa.2018.10.003.

[85] Y. Mao, Y. Zhang, L. Jiao, and H. Zhang, "Document-Level Sentiment Analysis Using Attention-Based Bi-Directional Long Short-Term Memory Network and Two-Dimensional Convolutional Neural Network," *Electronics*, vol. 11, no. 12, p. 1906, Jan. 2022, doi: 10.3390/electronics11121906.

[86] B. Liu, *Sentiment Analysis and Opinion Mining*. in Synthesis Lectures on Human Language Technologies. Cham: Springer International Publishing, 2012. doi: 10.1007/978-3-031-02145-9.

[87] T. Chen, R. Xu, Y. He, and X. Wang, "Improving Sentiment Analysis via Sentence Type Classification Using BiLSTM-CRF and CNN," *Expert Systems with Applications*, vol. 72, pp. 221–230, Apr. 2017, doi: 10.1016/j.eswa.2016.10.065.

[88] J. Su, Q. Chen, Y. Wang, L. Zhang, W. Pan, and Z. Li, "Sentence-Level Sentiment Analysis Based on Supervised Gradual Machine Learning," *Scientific Reports*, vol. 13, no. 1, p. 14500, Sep. 2023, doi: 10.1038/s41598-023-41485-8.

[89] L. Wu, F. Morstatter, and H. Liu, "SlangSD: Building, Expanding and Using a Sentiment Dictionary of Slang Words for Short-Text Sentiment Classification," *Language Resources and Evaluation*, vol. 52, no. 3, pp. 839–852, Sep. 2018, doi: 10.1007/s10579-018-9416-0.

[90] B. Pang and L. Lee, "Opinion Mining and Sentiment Analysis," *Foundations and Trends in Information Retrieval*, vol. 2, no. 1–2, pp. 1–135, Jan. 2008, doi: 10.1561/1500000011.

[91] Z. Madhoushi, A. R. Hamdan, and S. Zainudin, "Sentiment Analysis Techniques in Recent Works," in *2015 Science and Information Conference (SAI)*, Jul. 2015, pp. 288–291. doi: 10.1109/SAI.2015.7237157.

[92] H. Sankar and V. Subramaniyaswamy, "Investigating Sentiment Analysis Using Machine Learning Approach," in *2017 International Conference on Intelligent Sustainable Systems (ICISS)*, Dec. 2017, pp. 87–92. doi: 10.1109/ISS1.2017.8389293.

[93] H. Thakkar and D. Patel, "Approaches for Sentiment Analysis on Twitter: A State-of-Art Study." Accessed: Sep. 28, 2026. [Online]. Available: http://arxiv.org/abs/1512.01043

[94] M. Taboada, J. Brooke, M. Tofiloski, K. Voll, and M. Stede, "Lexicon-Based Methods for Sentiment Analysis," *Computational Linguistics*, vol. 37, no. 2, pp. 267–307, Jun. 2011, doi: 10.1162/COLI_a_00049.

[95] H. Kang, S. J. Yoo, and D. Han, "Senti-Lexicon and Improved Naïve Bayes Algorithms for Sentiment Analysis of Restaurant Reviews," *Expert Systems with Applications*, vol. 39, no. 5, pp. 6000–6010, Apr. 2012, doi: 10.1016/j.eswa.2011.11.107.

[96] M. Ahmad, S. Aftab, and I. Ali, "Sentiment Analysis of Tweets Using SVM," *International Journal of Computer Applications*, vol. 177, no. 5, pp. 25–29, 2017, Accessed: Sep. 28, 2026. [Online]. Available: https://ijcaonline.org/archives/volume177/number5/28624-2017915758/

[97] N. O. F. Daeli, "Sentiment Analysis on Movie Reviews Using Information Gain and K-Nearest Neighbor," Jan. 2020, doi: 10.34818/JDSA.2020.3.22.

[98] Y. Chen, "Convolutional Neural Network for Sentence Classification," Aug. 2015, Accessed: Sep. 28, 2026. [Online]. Available: http://hdl.handle.net/10012/9592

[99] J. Xu, D. Chen, X. Qiu, and X. Huang, "Cached Long Short-Term Memory Neural Networks for Document-Level Sentiment Classification," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, J. Su, K. Duh, and X. Carreras, Eds., Austin, Texas: Association for Computational Linguistics, Nov. 2016, pp. 1660–1669. doi: 10.18653/v1/D16-1172.

[100] M. M.Abdelgwad, T. H. A Soliman, A. I.Taloba, and M. F. Farghaly, "Arabic Aspect Based Sentiment Analysis Using Bidirectional GRU Based Models," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 9, pp. 6652–6662, Oct. 2022, doi: 10.1016/j.jksuci.2021.08.030.

[101] J.-R. Chang, H.-Y. Liang, L.-S. Chen, and C.-W. Chang, "Novel Feature Selection Approaches for Improving the Performance of Sentiment Classification," *Journal of Ambient Intelligence and Humanized Computing*, Aug. 2020, doi: 10.1007/s12652-020-02468-z.

[102] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding." Accessed: Sep. 28, 2026. [Online]. Available: http://arxiv.org/abs/1810.04805

[103] M. Hoang, O. A. Bihorac, and J. Rouces, "Aspect-Based Sentiment Analysis Using BERT," in *Proceedings of the 22nd Nordic Conference on Computational Linguistics*, M. Hartmann and B. Plank, Eds., Turku, Finland: Linköping University Electronic Press, Sep. 2019, pp. 187–196. Accessed: Sep. 28, 2026. [Online]. Available: https://aclanthology.org/W19-6120/

[104] A. S. Talaat, "Sentiment Analysis Classification System Using Hybrid BERT Models," *Journal of Big Data*, vol. 10, no. 1, p. 110, Jun. 2023, doi: 10.1186/s40537-023-00781-w.

[105] A. Bello, S.-C. Ng, and M.-F. Leung, "A BERT Framework to Sentiment Analysis of Tweets," *Sensors*, vol. 23, no. 1, p. 506, Jan. 2023, doi: 10.3390/s23010506.

[106] X. Sun *et al.*, "Sentiment Analysis through LLM Negotiations." Accessed: Sep. 28, 2026. [Online]. Available: http://arxiv.org/abs/2311.01876

[107] P. S. Ghatora, S. E. Hosseini, S. Pervez, M. J. Iqbal, and N. Shaukat, "Sentiment Analysis of Product Reviews Using Machine Learning and Pre-Trained LLM," *Big Data and Cognitive Computing*, vol. 8, no. 12, p. 199, Dec. 2024, doi: 10.3390/bdcc8120199.

[108] W. Jin *et al.*, "Self-Adaptive LLM Instructions Optimization for Aspect-Based Sentiment Analysis by Incorporating Emotion-Oriented In-Contexts," *Computational Intelligence*, vol. 41, no. 5, p. e70129, 2025, doi: 10.1111/coin.70129.

[109] P. Koehn, "Europarl: A Parallel Corpus for Statistical Machine Translation," in *Proceedings of Machine Translation Summit X: Papers*, Phuket, Thailand, Sep. 2005, pp. 79–86. Accessed: Mar. 31, 2026. [Online]. Available: https://aclanthology.org/2005.mtsummit-papers.11/

[110] J. Cova, "A New Database for Italian Parliamentary Speeches: Introducing the \mkbibemph{ItaParlCorpus} Dataset," *Italian Political Science Review/Rivista Italiana di Scienza Politica*, vol. 55, no. 1, pp. 77–86, Mar. 2025, doi: 10.1017/ipo.2025.6.

[111] F. Cominetti, L. Gregori, E. Lombardi Vallauri, and A. Panunzi, "IMPAQTS: A Multimodal Corpus of Parliamentary and Other Political Speeches in Italy (1946-2023), Annotated with Implicit Strategies," in *Proceedings of the IV Workshop on Creating, Analysing, and Increasing Accessibility of Parliamentary Corpora (ParlaCLARIN) @ LREC-COLING 2024*, D. Fiser, M. Eskevich, and D. Bordon, Eds., Torino, Italia: ELRA and ICCL, May 2024, pp. 101–109. Accessed: Apr. 01, 2026. [Online]. Available: https://aclanthology.org/2024.parlaclarin-1.15/

[112] L. Giuliano, "Il corpus LP." Accessed: Apr. 28, 2026. [Online]. Available: https://www.paroladileader.com/p/blog-page.html

[113] T. E. Payne, "Describing Morphosyntax: A Guide for Field Linguists." Accessed: Apr. 28, 2026. [Online]. Available: https://www.cambridge.org/highereducation/books/describing-morphosyntax/85E720AD1CD573E24FDA36FEEA63783C

[114] T. Erjavec *et al.*, "The ParlaMint Corpora of Parliamentary Proceedings," *Language Resources and Evaluation*, vol. 57, no. 1, pp. 415–448, Mar. 2023, doi: 10.1007/s10579-021-09574-0.

[115] T. Erjavec *et al.*, "ParlaMint II: Advancing Comparable Parliamentary Corpora across Europe," *Language Resources and Evaluation*, vol. 59, no. 3, pp. 2071–2102, Sep. 2025, doi: 10.1007/s10579-024-09798-w.

[116] S. Tonelli, R. Sprugnoli, and G. Moretti, "Prendo La Parola in Questo Consesso Mondiale:," 2019, [Online]. Available: https://ceur-ws.org/Vol-2481/paper71.pdf

[117] D. Witten, M. L. Jockers, and C. S. Criddle, "Reassessing Authorship of the Book of Mormon Using Delta and Nearest Shrunken Centroid Classification," *Literary and Linguistic Computing*, Jan. 2008, doi: 10.1093/LLC/FQN040.

[118] M. L. Jockers and D. M. Witten, "A Comparative Study of Machine Learning Methods for Authorship Attribution," *Literary and Linguistic Computing*, vol. 25, no. 2, pp. 215–223, Jun. 2010, doi: 10.1093/llc/fqq001.

[119] M. H, S. Kumar, D. A. Dinesh, and V. Thenkanidiyoor, "Spoken Language Identification in Unseen Target Domain Using Centroid Similarity Loss With Adaptive Gradient Blending," *IEEE Access*, vol. 12, pp. 95959–95971, 2024, doi: 10.1109/ACCESS.2024.3422380.

[120] M. H. Protter and C. B. Morrey, *College Calculus with Analytic Geometry*, 2d ed. in Addison-Wesley Series in Mathematics. Reading, Mass: Addison-Wesley, 1970.

[121] DeepSeek-AI *et al.*, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," *Nature*, vol. 645, no. 8081, pp. 633–638, Sep. 2025, doi: 10.1038/s41586-025-09422-z.

[122] B. Herre, "Democracy Data: How Sources Differ and When to Use Which One," *Our World in Data*, Jun. 2025, Accessed: Apr. 10, 2026. [Online]. Available: https://ourworldindata.org/democracies-measurement

[123] B. Herre, "The 'Varieties of Democracy' Data: How Do Researchers Measure Democracy?," *Our World in Data*, Apr. 2025, Accessed: Apr. 10, 2026. [Online]. Available: https://ourworldindata.org/vdem-electoral-democracy-data